

Live Ninja — Formal Project Plan

Live Ninja — Product Requirements Document

Executive Summary

1. Vision & Goals; Non-Goals
2. Personas & Primary Use Cases
3. Functional Requirements by Surface
4. UX Requirements per Surface
5. System Architecture Overview
6. Voice / GPT-Realtime Experience Requirements
7. Authentication & Session Requirements
8. Data Model & Storage
9. Non-Functional Requirements
10. Security & Privacy
11. Success Metrics / KPIs
12. Assumptions, Dependencies, Risks
13. Delivery Roadmap
14. Open Questions with Chosen Defaults

UI Wireframe Specifications

UI Prototypes — Full-Screen Clickable Mockups

Live Ninja — Implementation Plan

1. Overview & how to read this plan
2. Workstream map
3. Architecture snapshot
4. Milestones
5. Environments & deploy
6. Testing & verification strategy per milestone
7. Risks & mitigations (execution / sequencing)
8. Implementation Notes

FORMAL PROJECT PLAN

Live Ninja

A GPT-Realtime voice-assistant platform — one backend, three surfaces

Version: 1.0 • **Date:** 2026-07-17

Owner: Jeremy Proffitt

Surfaces: Android • Website • M5Stack Tab5

Stack: AWS SAM • Go / Go-Fiber • Lambda (arm64) • DynamoDB • S3 • SES • IoT Core • OpenAI
GPT Realtime

Contents: Product Requirements Document + Implementation Plan

Live Ninja — Product Requirements Document

Version: 1.0 · **Date:** 2026-07-17 · **Owner:** Jeremy Proffitt · **Status:** Draft

Executive Summary

Live Ninja is a personal, always-available speech-to-speech AI assistant delivered across three Login-with-Amazon-gated surfaces — an Android app that becomes the phone’s default voice assistant, a responsive web application, and an ambient M5Stack Tab5 embedded voice terminal on the desk — all sharing a single serverless AWS backend built on Go-Fiber (Lambda Web Adapter, arm64/Graviton) and OpenAI’s GPT Realtime engine. The backend’s core responsibilities are to broker short-lived OpenAI Realtime ephemeral tokens so the API key never touches a client, run the LWA OAuth exchange and mint first-party session credentials with surface-specific lifetimes (30 days for web/Android, 10 years for the M5Stack), persist users/devices/sessions/wake-word configs/usage in a single-table DynamoDB with strict Query/GetItem-only access, and serve the web UI and device config assets. Wake-word detection is always on-device on every surface — a hard architectural privacy invariant — and is user-programmable (“Hey Live Ninja” by default). This document specifies the vision, personas, functional and non-functional requirements per surface, UX rules, system architecture, voice experience, authentication, data model, security/privacy posture, KPIs, risks, and the baked-in decision for every open question.

Metadata	Value
Document version	1.0
Date	2026-07-17
Owner	Jeremy Proffitt
Status	Draft
Product	Live Ninja
Domain	live.jeremy.ninja (aliases web.live.ninja / auth.live.ninja)
AWS account / region	759775734231 · us-east-1
Repo	JeremyProffittOrg/live-ninja
Cost center	voice-ai

1. Vision & Goals; Non-Goals

1.1 Vision

Live Ninja is a single, coherent voice assistant that follows the user across every context of their day — pocket (Android), desk (M5Stack Tab5), and browser (web) — with identical persona, memory, and capabilities regardless of surface. It is private by construction (wake detection never leaves the device), cheap to run (serverless, cost-guarded, no scans, no secrets managers), and conversationally instant (direct client→OpenAI audio, sub-second first response, barge-in). It is a personal product for a production-only, single-operator shop, engineered to the same rigor as a commercial platform.

1.2 Goals

Goal	Description
G-1	One shared backend and one persona/settings contract across all three surfaces.
G-2	Speech-to-speech conversation with barge-in and server-side tool execution.
G-3	On-device, user-programmable wake word on every surface; no always-on cloud audio.
G-4	LWA identity front-door with first-party sessions (30-day web/Android, 10-year M5Stack).
G-5	OpenAI API key never reaches any client; ephemeral tokens only.
G-6	Strict cost discipline: Query/GetItem-only DynamoDB, on-demand billing, per-user metering/quotas, budget alarms.
G-7	Production-grade security/privacy: encrypted-at-rest, revocable credentials, deletion rights, threat-modeled.
G-8	OIDC-only GitHub Actions deploys; no local deploys, no static keys, mandatory cost tags.

1.3 Non-Goals

- **N-1:** No multi-tenant SaaS, org/team management, or billing/subscription system (personal product).
- **N-2:** No telephony/SIP surface.
- **N-3:** No on-device ASR/LLM/TTS — all conversation runs through OpenAI Realtime (with a server-side chained fallback).
- **N-4:** No staging/dev environment — production-only, guarded by local smoke tests, alarms, and canary rollouts.
- **N-5:** No third-party analytics SDKs (privacy + cost); analytics derive from the self-hosted Athena event lake.
- **N-6:** No relaying of WebRTC/browser/Android audio through AWS — audio is direct to OpenAI.

- **N-7:** No secrets manager (SSM Parameter Store SecureString / KMS only).
-

2. Personas & Primary Use Cases

2.1 Phone-primary-assistant user (Ava)

Ava has made Live Ninja her phone's default digital assistant. She invokes it hands-free by saying "Hey Live Ninja" (on-device Porcupine wake), by the assist gesture, by long-press power, or from her Bluetooth headset — even over the lock screen. Primary use cases: dictate and send an email while driving, set timers/reminders, ask quick questions, control her M5Stack, all with barge-in so she can interrupt mid-answer.

2.2 Desktop / web user (Wei)

Wei uses Live Ninja in the browser at his desk. He signs in with Amazon, clicks the large mic button (or optionally enables in-browser WASM wake word), and has a speech-to-speech conversation with a live transcript and audio visualizer. He manages persona, voice, and wake-word settings from a schema-driven settings page. Primary use cases: longer working conversations, reviewing/exporting transcripts, tuning settings that then sync to his other surfaces.

2.3 Ambient M5Stack-on-desk user (Maya)

Maya has an M5Stack Tab5 sitting on her desk, paired once via its config page and persisting for years. It shows an idle "Say 'Hey Live Ninja'" screen. She speaks to it hands-free; the device does on-device wake + AEC, streams Opus audio over IoT Core MQTT to a Fargate relay that brokers the OpenAI session, and plays the response back with instant local barge-in. Primary use cases: glanceable ambient assistant, quick questions/timers/device actions without touching a phone, a persistent household terminal.

3. Functional Requirements by Surface

Acceptance criteria (AC) are testable. IDs are stable references.

3.1 Backend

ID	Requirement	Acceptance Criteria
FR-B01	Single Go-Fiber app on Lambda Web Adapter behind HTTP API Gateway v2, arm64.	Same binary runs locally on :8080 and in Lambda; \$default route forwards to Fiber; deployed via SAM.
FR-B02	realtime-broker mints OpenAI ephemeral client secrets from the SSM-held key.	GET /api/v1/realtime/session returns an ek_ ... token + session/persona/tool manifest; standing key never in response or client.
FR-B03	Single-table DynamoDB live-ninja, on-demand, Query/GetItem only.	No Scan on any serving path (code review + ConsumedReadCapacityUnits alarm); two GSIs cover inverse lookups.
FR-B04	S3 presigned upload/download with content-type allowlist, size cap, owner-namespaced keys.	POST /api/v1/uploads returns a presigned PUT with pinned Content-Type / Content-Length ; download authorizes key prefix == caller userId .
FR-B05	SES transactional email off the request path via SQS → email-dispatch .	Source is a DKIM-verified @jeremy.ninja identity, Reply-To proffitt.jeremy@gmail.com ; idempotency prevents duplicate sends.
FR-B06	IoT Core MQTT control-plane ingest for device telemetry/heartbeat.	IoT Rule → iot-ingest writes PutItem (never Scan) updating DEVICE# lastSeen/telemetry.
FR-B07	usage-rollup aggregates raw usage logs into daily rollups hourly.	Dashboards read ROLLUP# items via Query; no raw-log fan-out.
FR-B08	Secrets in SSM SecureString, cached in-memory with TTL; broker IAM-scoped to the OpenAI key.	Broker role has ssm:GetParameter on one ARN + kms:Decrypt only; secrets never logged.
FR-B09	Per-user rate limiting and budget gate at token mint.	Over-quota mint returns 402 / 429 ; token-bucket enforced via DynamoDB conditional writes.
FR-B10	Server-side tool execution via POST /api/v1/tools/invoke under LN-JWT.	Tool calls authorized against the calling user; side-effecting tools carry idempotency keys and audit records.

3.2 Voice / GPT-Realtime

ID	Requirement	Acceptance Criteria
FR-V01	Speech-to-speech via <code>gpt-realtime</code> , config-bound session at mint.	Session object (model, voice, instructions, tools, <code>semantic_vad</code> , audio format) fixed server-side; client cannot escalate tools/persona.
FR-V02	Direct client↔OpenAI audio (WebRTC web/Android; WSS relay for M5Stack).	AWS is never in the WebRTC media path; M5Stack audio flows device→IoT→Fargate relay→OpenAI.
FR-V03	Barge-in via <code>interrupt_response</code> .	On <code>speech_started</code> , client stops/flushes playback within ≤ 150 ms; relay issues <code>response.cancel</code> + buffer clear for M5Stack.
FR-V04	Tool/function calling routed server-side.	Web/Android forward calls over HTTPS; relay intercepts for M5Stack; <code>function_call_output</code> returned to the session.
FR-V05	Transcript capture both directions.	User ASR and assistant transcripts persisted as <code>TURN#</code> / <code>LOG#</code> items with TTL; rendered as labeled turns, never raw JSON.
FR-V06	Per-session usage metering from <code>response.done</code> .	Atomic <code>UpdateItem ADD</code> on <code>USAGE#</code> /monthly counters; EMF metrics emitted; enforced pre-spend at mint.
FR-V07	Fallback cascade on Realtime failure.	Retry+backoff → chained STT→LLM→TTS → text-only → graceful failure with side-effects queued; tools still function.
FR-V08	Idle/duration caps.	Session auto-closes after inactivity and at the OpenAI session cap; no billing on abandoned sessions.

3.3 Auth

ID	Requirement	Acceptance Criteria
FR-A01	LWA Authorization Code + PKCE, backend as confidential client/BFF.	Code exchanged server-side with client secret from SSM; clients never hold LWA client secret.
FR-A02	Token-substitution defense.	Backend validates <code>aud == our client_id</code> via <code>/tokeninfo</code> and derives identity (<code>user_id</code>) from <code>/user/profile</code> .
FR-A03	First-party hybrid session: stateless ES256 access JWT (15 min) + stateful rotating refresh token.	JWT verified at edge with no DB read; refresh stored hashed, rotated on use with reuse detection.
FR-A04	Web 30-day sliding session in <code>__Host-</code> HttpOnly cookie.	Refresh cookie <code>Secure</code> ; <code>HttpOnly</code> ; <code>SameSite=Lax</code> ; access JWT in memory only; TTL slides on each rotation.
FR-A05	Android 30-day session, refresh in Keystore-backed EncryptedSharedPreferences.	PKCE via Custom Tabs + App Link; backend confidential exchange; refresh in memory/Keystore split.
FR-A06	M5Stack device-hosted pairing, 10-year silently rotated credential lineage.	Device-generated PKCE + pairing nonce; backend-completed exchange; device claims via poll presenting verifier; 10-year family TTL reset on each rotation.
FR-A07	Revocation: logout, logout-everywhere, device revoke.	Session row delete kills refresh immediately; <code>tokensValidAfter</code> bump kills all JWTs within the 60 s authorizer cache; device revoke detaches IoT cert + family.
FR-A08	New-sign-in SES tripwire.	New device/surface login sends a security-alert email out-of-band.

3.4 Android

ID	Requirement	Acceptance Criteria
FR-N01	Implement <code>VoiceInteractionService / Session</code> assistant contract and acquire <code>ROLE_ASSISTANT</code> .	Manifest declares the services with <code>BIND_VOICE_INTERACTION</code> ; role requested via <code>RoleManager</code> with OEM-aware guided fallback; <code>isRoleHeld</code> re-checked each launch.
FR-N02	Own on-device wake engine (Porcupine primary, openWakeWord fallback) in a <code>microphone FGS</code> .	"Hey Live Ninja" + custom words detected on-device; persistent listening notification shown; works regardless of role or OEM DSP.
FR-N03	GPT Realtime via WebRTC using backend-brokered ephemeral key.	Mic track + <code>oai-events</code> datachannel; SDP exchange with <code>Bearer ek_</code> ; API key never on device.
FR-N04	AEC-backed barge-in.	Platform + WebRTC AEC; on user speech mid-playback, fade-stop ≤ 50 ms, flush jitter buffer, start new turn.
FR-N05	Locked-screen session gating.	Non-sensitive replies proceed over keyguard; sensitive actions gated behind <code>requestDismissKeyguard()</code> .
FR-N06	Battery discipline.	VAD pre-gate before neural model, no continuous wakelock, thermal/battery-saver duty-cycling; target $< 2\%$ /hr screen-off idle.
FR-N07	Boot restart of wake FGS when user-enabled.	<code>BOOT_COMPLETED</code> starts the mic FGS if enabled and permissions granted; graceful deferral to first unlock if blocked.

3.5 Web

ID	Requirement	Acceptance Criteria
FR-W01	Go-Fiber server-rendered HTML + progressive-enhancement ES modules (no SPA framework).	Unauthenticated users get server-rendered login with no app-chrome flash; one deployable serves HTML + <code>/api/*</code> .
FR-W02	WebRTC directly to OpenAI via ephemeral token.	<code>POST /api/realtime/session</code> mints token (persona resolved server-side from ID); browser establishes peer connection; key never client-side.
FR-W03	Click-to-talk always works; optional WASM wake word off by default.	Mic state machine functions without wake word; wake word (openWakeWord WASM / Porcupine) opt-in, lazy-loaded in AudioWorklet; degrades to <code>unsupported</code> .
FR-W04	Live transcript + visualizer.	Incremental deltas rendered via text nodes only (XSS-safe); <code>role="log" aria-live="polite"</code> ; canvas visualizer <code>aria-hidden</code> .
FR-W05	Schema-driven settings, zero blind text boxes.	Every enumerable field is a populated control; persona sent as ID; voice from enum with preview.
FR-W06	PWA with network-first HTML service worker.	HTML <code>no-cache</code> + network-first SW; hashed assets <code>immutable</code> stale-while-revalidate; versioned cache purged on activate; API/OpenAI traffic never intercepted.

3.6 M5Stack

ID	Requirement	Acceptance Criteria
FR-M01	ESP-IDF firmware on ESP32-P4 with C6 <code>esp-hosted</code> radio link; LVGL 9 UI.	Pinned IDF tag; framebuffers in PSRAM with PPA dirty-rect flushes; touch via GT911.
FR-M02	On-device wake: ESP-SR WakeNet primary, microWakeWord fallback for custom phrases.	AFE (AEC/NS/VAD) precedes the tap; shadow <code>wakeword.engine</code> selects one active engine; custom models hot-loaded from S3 with SHA-256 verify.
FR-M03	Backend-proxied audio over IoT Core MQTT to Fargate relay.	Device publishes Opus frames on <code>audio/up</code> ; relay transcodes to PCM16@24k and bridges the OpenAI WS; audio never on IoT-terminated OpenAI path.
FR-M04	Instant local barge-in.	Local VAD stops DAC immediately and publishes <code>barge_in</code> ; relay issues <code>response.cancel + input_audio_buffer.clear</code> .
FR-M05	Device-hosted config: SoftAP WiFi onboarding + browser LWA pairing.	SSID chosen from a scan list (no blind box); LWA runs in the phone browser; bind result delivered over mTLS IoT channel.
FR-M06	Secure storage + 10-year identity.	Flash encryption + Secure Boot v2 + NVS encryption; key in DS peripheral (optional ATECC608B); 10-year cert auto-rotated at year 8; refresh rotated every 24h/boot.
FR-M07	OTA via IoT Jobs with A/B partitions and rollback.	Signed images, anti-rollback eFuse, mark-valid-after-check-in, canary group first; coordinated P4↔C6 version gating.

3.7 Wake-word

ID	Requirement	Acceptance Criteria
FR-K01	Wake detection always on-device on all surfaces.	No audio leaves the device before a wake trigger; verified architecturally and disclosed in-product.
FR-K02	User-programmable keywords with a shared catalog.	Wake-word control is a combobox over catalog + user models; default "Hey Live Ninja"; never a blind free-text phrase box that silently does nothing.
FR-K03	Backend training pipeline (AWS Batch arm64 for openWakeWord; Picovoice Console for Porcupine).	<code>POST /api/v1/wakewords</code> validates phrase (length/phonemes/profanity/collision); job concurrency ≤ 2 , 20-min timeout, ≤ 3 /day/user; output to S3, <code>status=ready</code> , SES notify.
FR-K04	Per-platform, content-addressed model distribution + SHA-256 verified hot-swap.	<code>GET /api/v1/wakewords/{id}/model?platform=</code> → presigned/CloudFront-signed URL; client verifies SHA before swap; unsupported format → default model + report.
FR-K05	M5Stack constraint: select from curated flashable WakeNet set + oWW-ESP fallback.	Device selects, never trains; list-select UI with position indicator; no on-screen keyboard for wake phrase.
FR-K06	Tunable sensitivity.	Bounded slider (0–1, default 0.5/"medium"); never a text field.

3.8 Settings

ID	Requirement	Acceptance Criteria
FR-S01	One canonical versioned settings doc per user (<code>SETTINGS#v<n></code>).	Monotonic <code>version</code> ; writes are <code>UpdateItem</code> with <code>ConditionExpression version=expected</code> ; conflict → 409, client re-reads/re-applies.
FR-S02	Cross-surface sync fan-out.	Web via WebSocket control frame; Android via FCM data message; M5Stack via IoT Device Shadow desired/reported delta; higher version wins.
FR-S03	All controls populated from queryable sources.	voice/persona-preset from enums; wakeWordId combobox; volume slider; retentionDays radio; only <code>persona.systemInstructions</code> is free text (capped, with reset-to-preset).
FR-S04	Privacy controls: transcript/audio storage + retention.	<code>storeAudio</code> default false; <code>storeTranscripts</code> default true 30-day; <code>retentionDays</code> enum {0,7,30,90}; TTL enforces deletion.
FR-S05	Right-to-delete.	<code>DELETE /account</code> partition-scoped <code>Query + BatchWriteItem</code> (no Scan), empties S3 prefix, deletes IoT thing/cert, revokes LWA refresh, SES-confirms; per-item deletion also available.

4. UX Requirements per Surface

4.1 Rich-UI surfaces (Web + Android)

- **UX-R01 — No blind text boxes.** Every enumerable/queryable value is a populated control: wake word (combobox over catalog + user models), voice (radio/segmented from the OpenAI voice enum with inline preview), persona (select over catalog; “Custom” reveals a labeled prompt editor — the sole justified free text, case 6), locale (ISO/IANA picker), mic/output device (populated from `enumerateDevices()` / `getAvailableCommunicationDevices()`), turn detection (radio), theme (segmented), sensitivity (slider 0–1).
- **UX-R02 — Control selection by threshold.** Booleans → toggles; 2–5 mutually-exclusive → radio/segmented; 6–20 → select; 100+/live → async typeahead; multi from a set → checkbox group or multi-select chips; bounded numbers → slider/stepper.

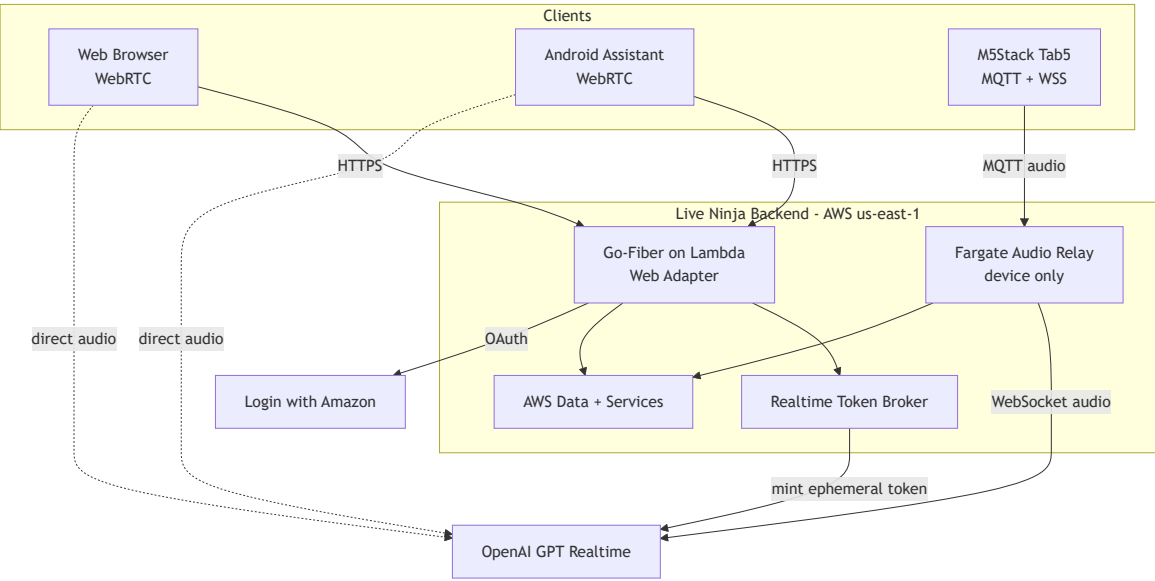
- **UX-R03 — Accessibility (WCAG AA, both themes).** Semantic landmarks, skip-to-content, programmatically associated `<label>`, `<fieldset>` / `<legend>` for groups, errors via `aria-describedby` + `role="alert"`, visible focus rings (never bare `outline:none`), ≥ 44 px targets, reflow at 200% zoom and minimum width, AA contrast (4.5:1 text / 3:1 large/UI) in light and dark, never state-by-color-alone. Android: Compose Material 3, TalkBack labels, `announceForAccessibility` for state changes.
- **UX-R04 — Data presentation.** Transcripts render as labeled turn bubbles (role label + timestamp), never raw JSON; usage/history in real tables with right-aligned numbers, sticky header, explicit empty/loading/error states, pagination $> \sim 50$ rows.
- **UX-R05 — Form flow.** Human-order fields, progressive disclosure (Porcupine engine reveals the AccessKey field), preserve valid input on error, inline real-time validation, action-specific button labels, destructive actions subordinate and guarded ("Sign out all devices" confirm).
- **UX-R06 — Voice-first accessible source of truth.** The text status region (`role="status"` `aria-live="polite"`) mirrors every audio state (Idle/Listening/Thinking/Speaking/Interrupted); the visualizer is decorative `aria-hidden`; `prefers-reduced-motion` respected.
- **UX-R07 — Mandatory agentic design pass.** The web/Android settings forms (non-trivial: ≥ 2 related fields, dependent reveal, enum-driven, destructive actions) go through the pre-build multi-persona flow-spec review (UX-flow / product / a11y / copy) before implementation.
- **UX-R08 — Android role UX.** Cannot self-assign the role; always a user action; OEM-aware walkthrough (Samsung/Pixel/Xiaomi/OnePlus copy + deep links); `isRoleHeld` polling on `onResume`; core UX works without the role.

4.2 M5Stack (LCD / embedded carve-out)

- **UX-L01 — Spirit preserved, form adapted.** No blind free-text except WiFi passphrase and device name (on-screen keyboard, most-constrained variant); everything enumerable is a list-select or segmented control populated from the shadow/firmware enums.
 - **UX-L02 — One primary value/decision per screen.** Idle ("Say 'Hey Live Ninja'"), Listening (live level meter + large Cancel), Speaking (orb + large always-visible Stop/Interrupt), Settings (master→detail list-select), Onboarding wizard, Error/reconnect (distinct, not silent).
 - **UX-L03 — Touch + rendering constraints.** ~ 48 – 64 px targets, no hover affordances, immediate tap feedback + debounce, dirty-rect PPA-accelerated flushes, "N of M" position indicators (no readable scrollbar), shallow widget trees, cheap/skippable animations.
 - **UX-L04 — Wake-word on device is select-only.** Full-screen single-column list-select of "Hey Live Ninja" + compilable user models; phrase creation is redirected to web/Android; no on-screen keyboard for wake phrases.
 - **UX-L05 — Config page is rich UI.** The device-hosted page, opened in a real phone/laptop browser, follows all rich-UI rules (populated SSID scan list, semantic, WCAG AA) — it is not an LCD UI.
 - **UX-L06 — Firmware UI state machine** governs transitions (see §6 state diagram): Boot → Provisioning/Idle → Listening → Thinking → Speaking, with barge-in, Config, and Error recovery paths.
-

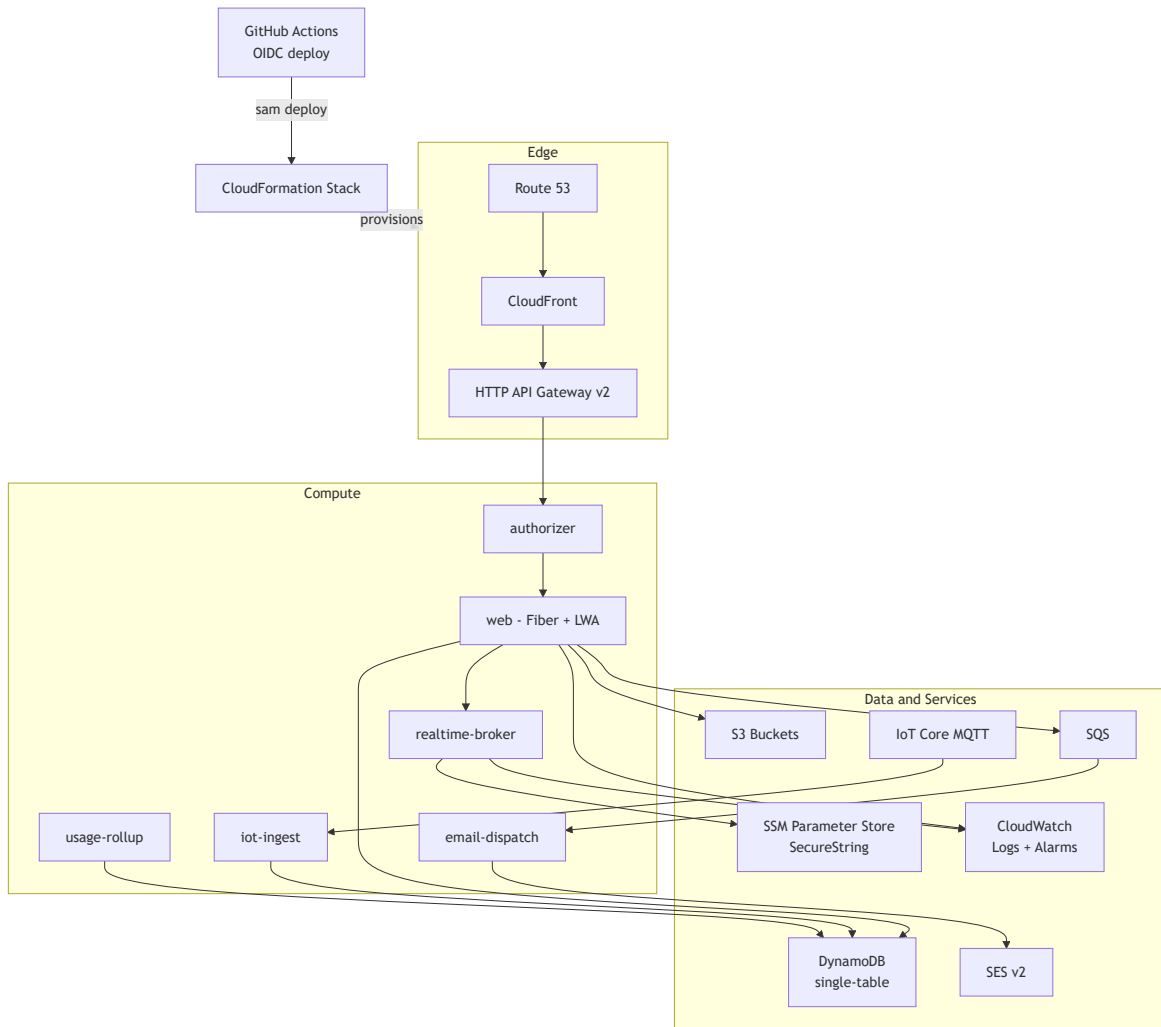
5. System Architecture Overview

Live Ninja is one Go-Fiber-on-Lambda-Web-Adapter service fronted by HTTP API v2 + CloudFront, with a small set of purpose-split Lambdas (`authorizer`, `realtime-broker`, `iot-ingest`, `usage-rollup`, `email-dispatch`) and one justified always-on component: the Fargate audio relay for the M5Stack. Web and Android connect audio directly to OpenAI via WebRTC; the M5Stack streams Opus over IoT Core MQTT to the relay, which holds the OpenAI WebSocket. The following system context diagram shows the three LWA-gated clients, the shared backend, OpenAI Realtime, and the AWS services behind them.



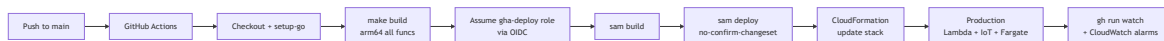
diagram

The concrete AWS resource topology — HTTP API + Fiber Lambda, the split-out Lambdas, DynamoDB, S3, SES, IoT Core, SSM, CloudWatch, and the OIDC deploy path — is shown below.



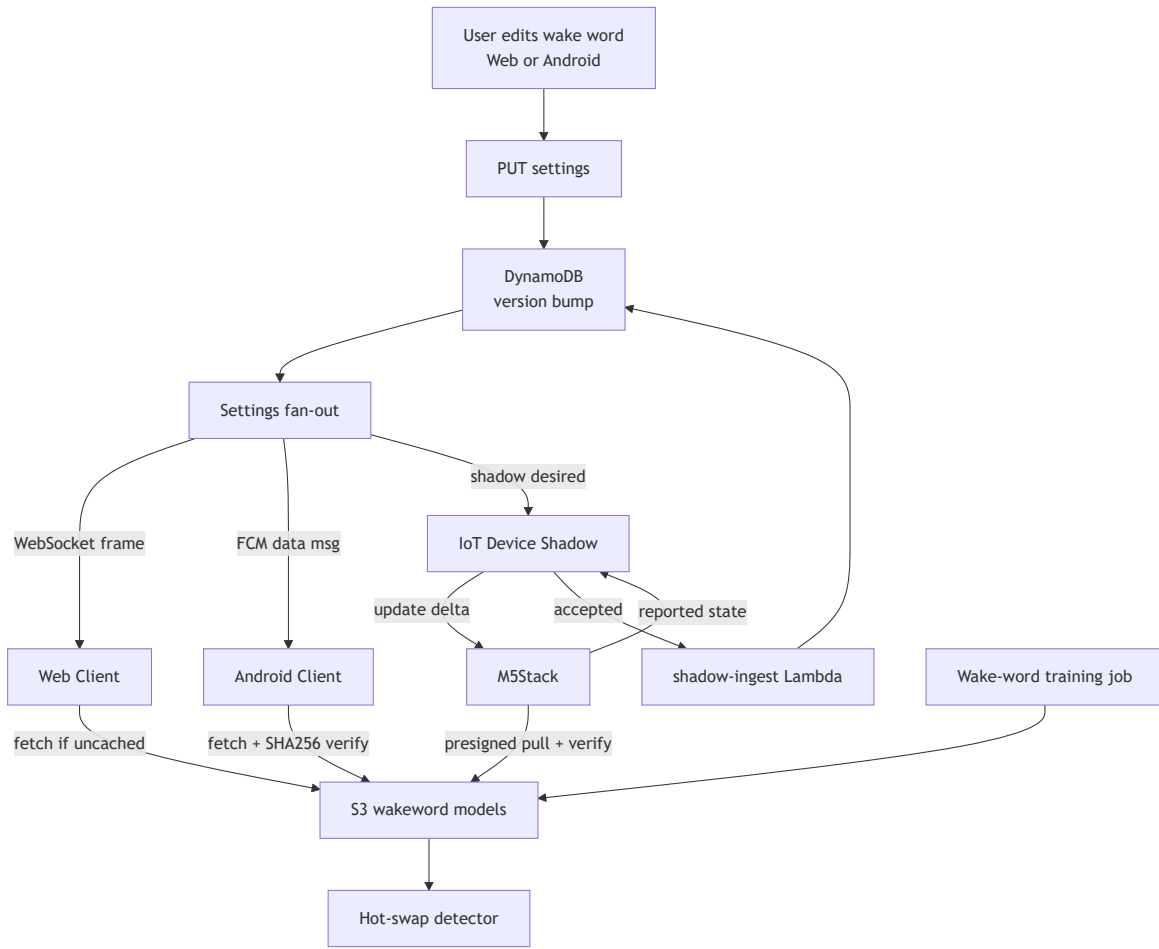
diagram

Deployment is push-to- `main` → GitHub Actions (OIDC-assumed `gha-deploy` role, no static keys) → `sam build` / `sam deploy` , arm64 throughout, with the six mandatory cost tags. The pipeline:



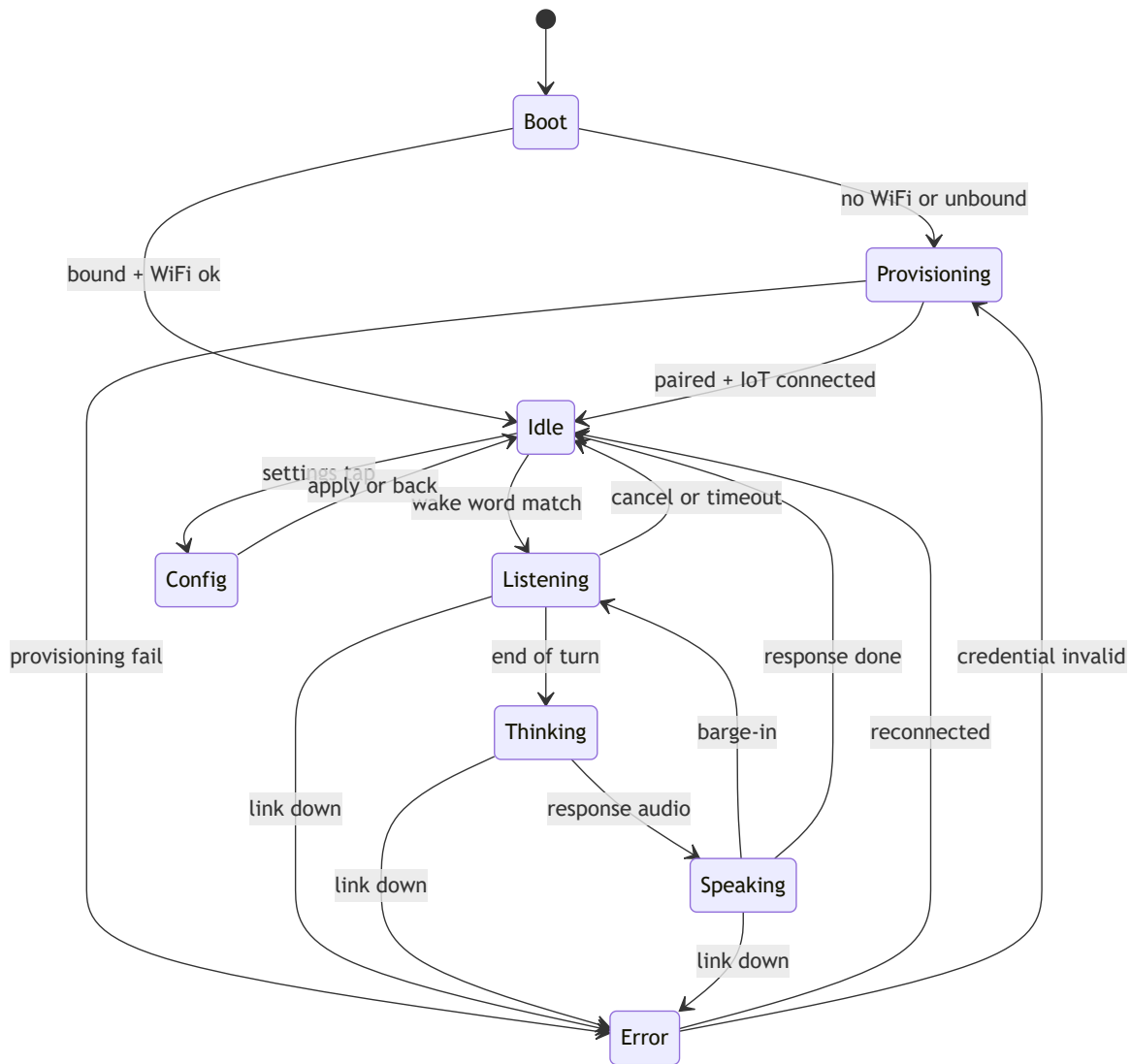
diagram

Wake-word config propagates across surfaces via push channels and the IoT device shadow:



diagram

The M5Stack firmware lifecycle from boot/provisioning through the conversational states and error recovery:



diagram

6. Voice / GPT-Realtime Experience Requirements

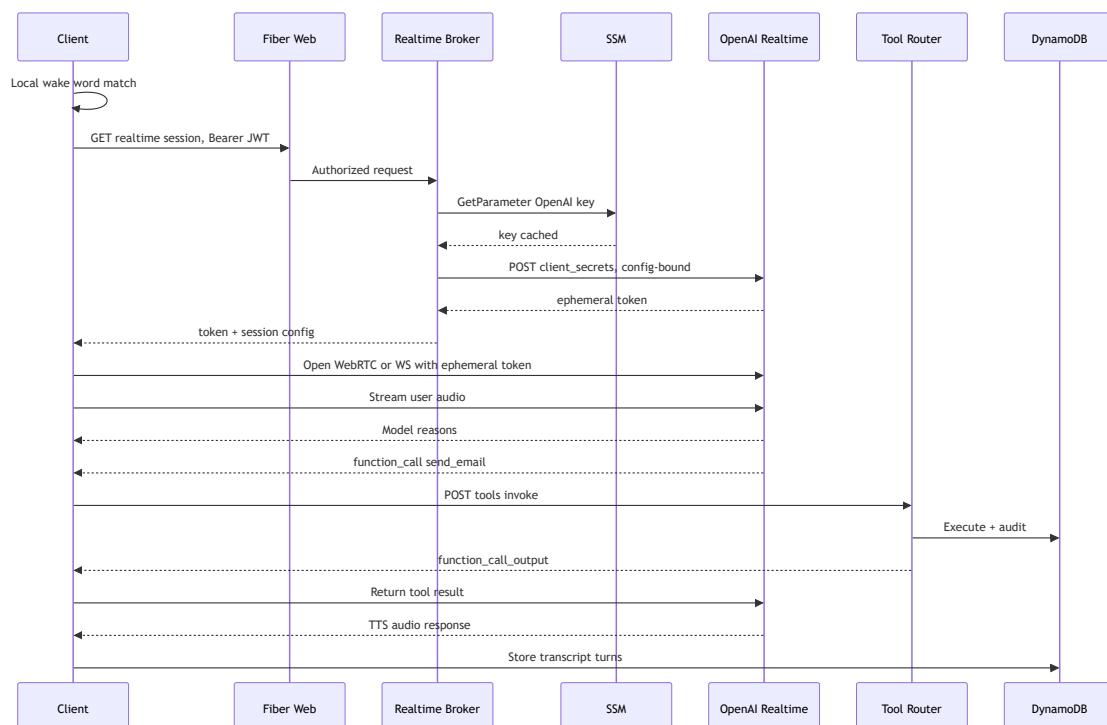
6.1 Latency & interaction targets

Metric	Target
User stops speaking → first assistant audio	≤ 800 ms (WebRTC); < 1.2 s (M5Stack relay)
Barge-in stop (playback silenced)	≤ 150 ms
Ephemeral token establish window	~60 s TTL, pre-minted during wake
OpenAI session duration cap	~30 min, transparently re-minted/reconnected

6.2 Requirements

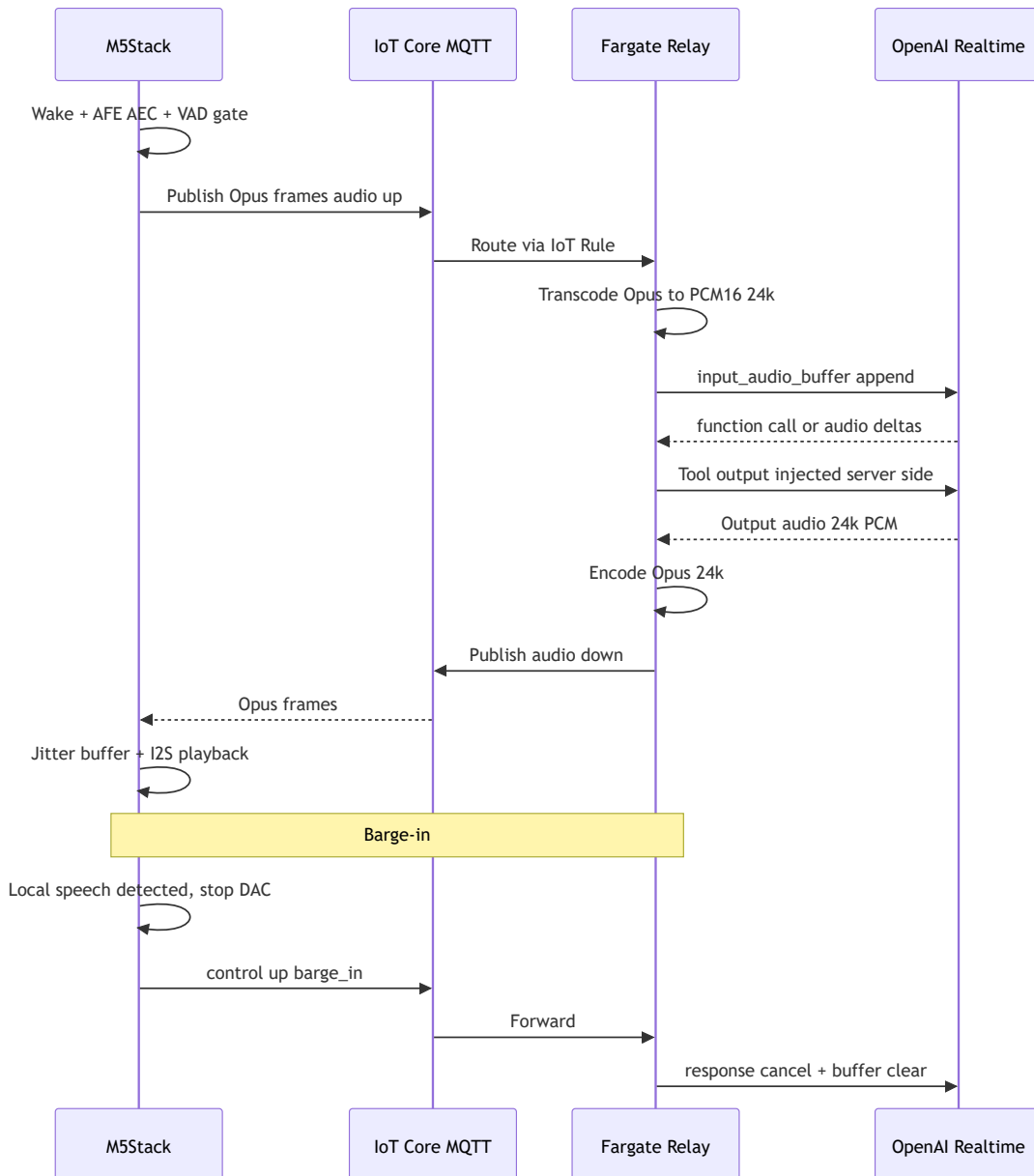
- **Config-bound sessions:** model `gpt-realtime`, `output_modalities:["audio"]`, `pcm16` audio (WebRTC negotiates Opus itself), `turn_detection: semantic_vad` with `interrupt_response:true` and `create_response:true`, voice default `cedar` (user-selectable from an enumerated picker), `temperature 0.7`, `max_output_tokens 4096`. Persona = shared base prompt + per-user overrides; never a blind box.
- **Barge-in:** on `input_audio_buffer.speech_started`, clients immediately duck/stop playback and flush buffers before the response-cancel round-trips; M5Stack stops the DAC locally and clears its jitter buffer.
- **Tool calling:** tools are declared at mint (config-bound); execution is always server-side in `fn-tool-router`, re-authorized against the LWA user each call, with enumerated tool args, confirm-before-send for external email, and idempotency keys. Initial catalog: `send_email`, `set_timer / set_reminder`, `device_control`, `get_weather / web_lookup`, `remember_note / recall_note`, `create_calendar_event` (phase 2).
- **Persona:** "You are Live Ninja, a concise, action-oriented voice assistant..." plus user name/timezone/units injected server-side; identical across surfaces.
- **Fallback:** retry+backoff → chained STT (`gpt-4o-transcribe`) → `gpt-4o-mini` → TTS (`gpt-4o-mini-tts`) → text-only → graceful failure with side-effects queued; tools work in the fallback chain.

The end-to-end voice turn — local wake, ephemeral token, Realtime session, tool call, spoken response, and transcript storage:



diagram

The M5Stack audio path bridges device MQTT through the Fargate relay to OpenAI and back, with local instant barge-in:



diagram

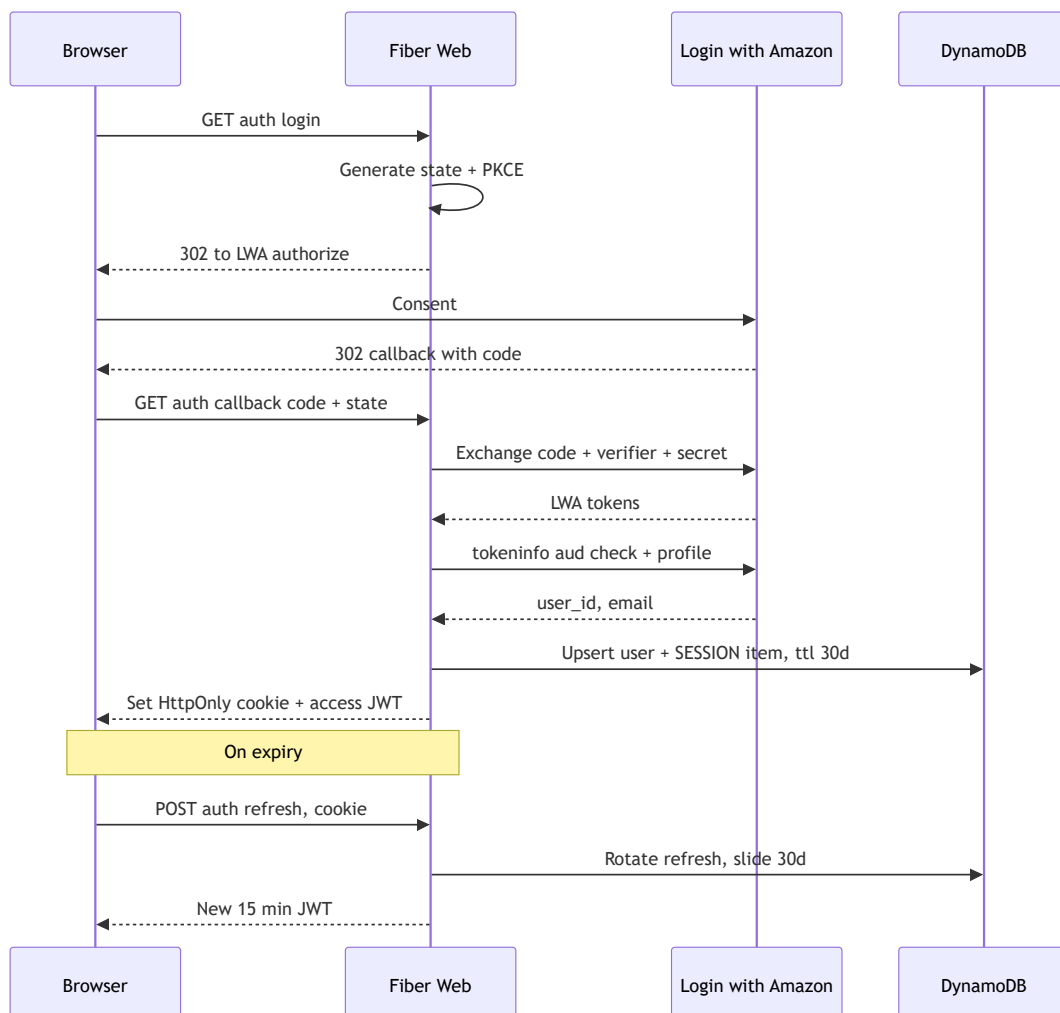
7. Authentication & Session Requirements

LWA establishes identity only; clients carry first-party credentials. The hybrid model — stateless ES256 access JWT (15 min, verified at the edge with no DB read) plus stateful opaque refresh token (hashed at rest, rotated on every use with reuse detection) — is shared machinery; only lifetime, storage medium, and second factor differ per surface.

Surface	LWA flow	Refresh lifetime	Refresh storage	Second factor
Web	Auth Code + PKCE (server-side confidential)	30 days, sliding	<u>__Host-</u> Secure HttpOnly cookie	SameSite + CSRF state
Android	Auth Code + PKCE (public → BFF exchange)	30 days, sliding	Keystore / EncryptedSharedPreferences	App Links + optional attestation
M5Stack	Device-hosted Auth Code + PKCE, backend-completed, device-pollled claim	10 years, silently rotated	Encrypted NVS (flash enc + secure boot)	AWS IoT X.509 client cert (mTLS)

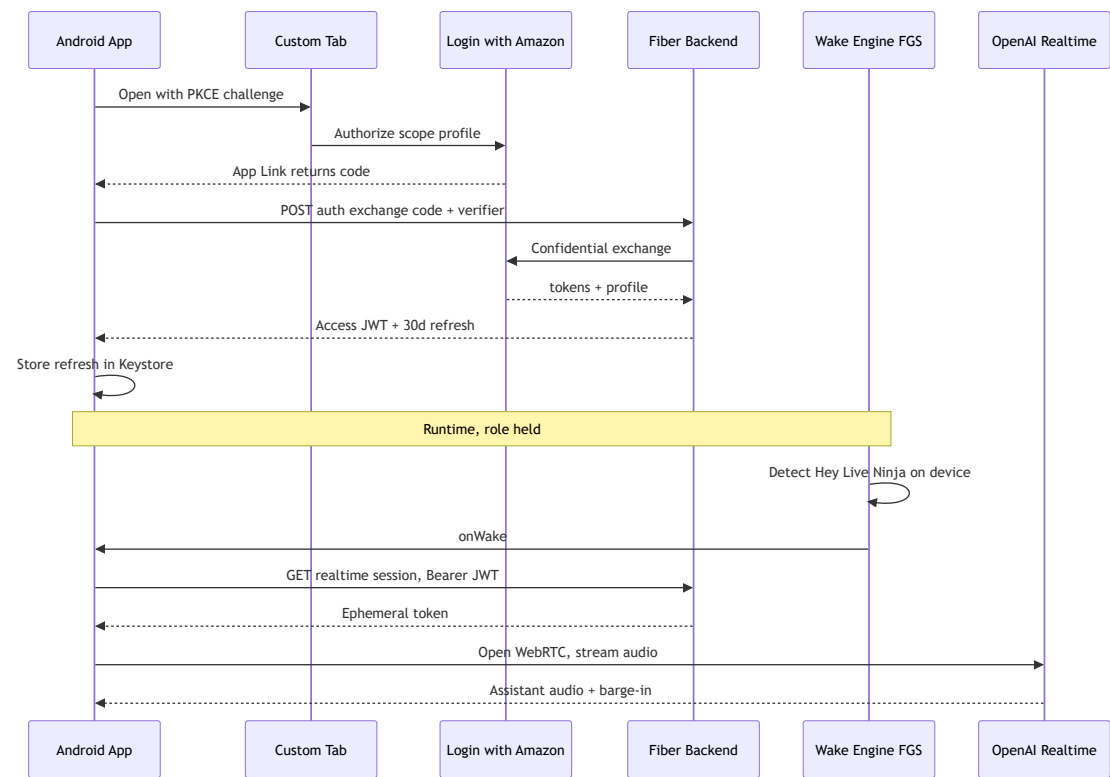
Revocation: logout deletes the session row (refresh dead immediately, JWT dies ≤ 15 min); logout-everywhere bumps `tokensValidAfter` (all JWTs rejected within the 60 s authorizer cache); device revoke detaches the IoT cert and revokes the family. Reuse of a rotated refresh token revokes the whole `familyId` and fires a security alert.

Web login — LWA Auth Code + PKCE, 30-day session



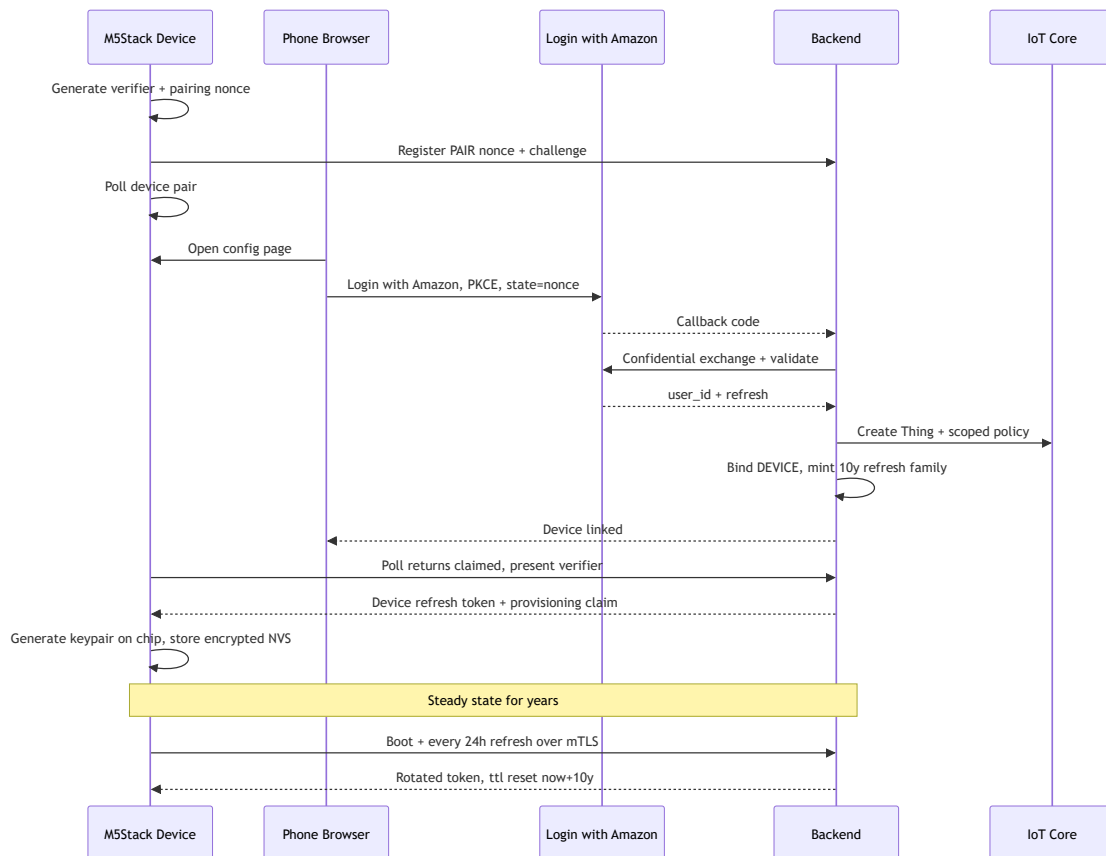
diagram

Android login + primary-assistant wake flow



diagram

M5Stack on-device config login + 10-year persistence



diagram

The 10-year “login” is a continuously, silently rotated credential lineage anchored by a hardware-bound X.509 cert (issued 10-year, auto-rotated at year 8), not a static 10-year bearer secret in flash.

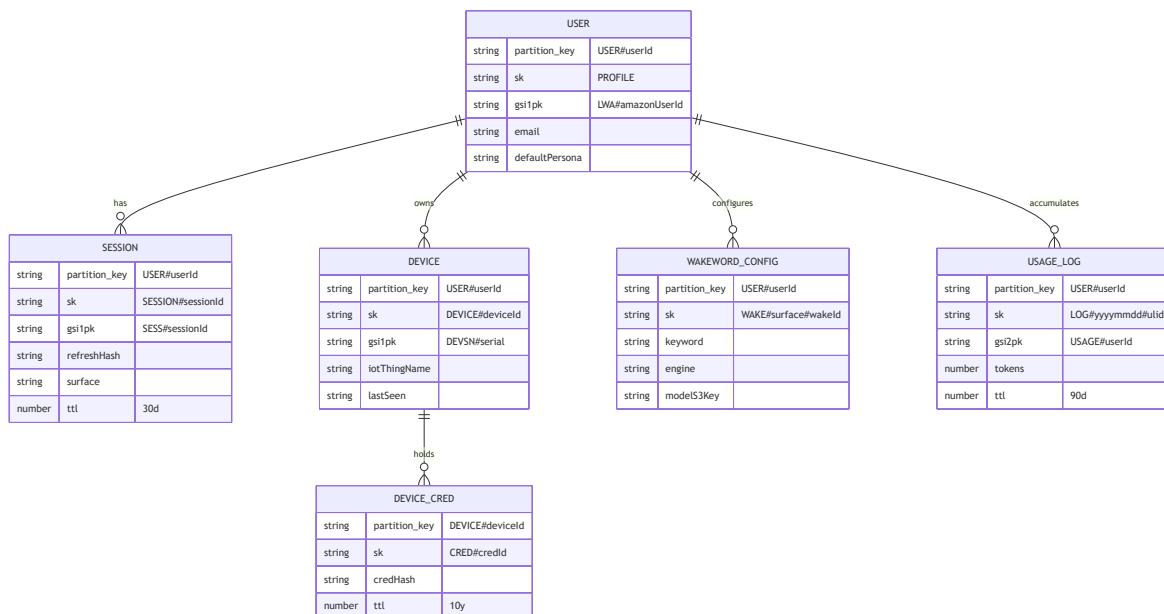
8. Data Model & Storage

8.1 DynamoDB — single table `live-ninja`

- **PK** `pk` (S), **SK** `sk` (S), on-demand (`PAY_PER_REQUEST`). **GSI1** (`gsi1pk` / `gsi1sk`) for inverse/external-id lookups (LWA user, device serial, session id, email); **GSI2** (`gsi2pk` / `gsi2sk`) for time-ordered feeds (usage by day, active sessions, device lastSeen). TTL on `ttl` auto-expires sessions/logs. Every access path is a `Query` / `GetItem` ; `Scan` is reserved for manual one-off migrations only.

Entity	PK	SK	GS1	GS2	TTL
User	USER# <userId>	PROFILE	LWA# <amazonUserId>	—	—
Session	USER# <userId>	SESSION# <sessionId>	SESS# <sessionId>	SESSACTIVE# <userId> / <expISO>	30d web/Android
Device	USER# <userId>	DEVICE# <deviceId>	DEVS# <serial>	DEVSEEN# <userId> / <lastSeen>	—
Device cred	DEVICE# <deviceId>	CRED#<credId>	—	—	10y
WakeWordConfig	USER# <userId>	WAKE# <surface># <wakeId>	—	—	—
Persona	USER# <userId>	PERSONA# <personaId>	—	—	—
UsageLog	USER# <userId>	LOG# <yyyymmdd># <ulid>	—	USAGE# <userId> / <epoch>	90d
UsageDailyRollup	USER# <userId>	ROLLUP# <yyyymmdd>	—	ROLLUPGLOBAL# <yyyymm>	400d
Settings	USER# <userId>	SETTINGS#v<n>	—	—	—
IdempotencyKey	IDEMP# <key>	IDEMP	—	—	24h

The entity model and GSI lookup relationships:



diagram

8.2 S3 buckets

Bucket	Purpose
<code>live-ninja-user-<acct></code>	user uploads/downloads, exported transcripts (<code>uploads/<userId>/...</code> , <code>transcripts/<userId>/<sid>.json</code>)
<code>live-ninja-wakewords-<acct></code>	custom wake-word model blobs (<code>.ppn</code> / <code>.onnx</code> / <code>.tflite</code>), per-user, per-platform
<code>live-ninja-assets-<acct></code>	web static assets + M5Stack config/firmware (CloudFront origin)
<code>live-ninja-logs-<acct></code>	access logs / rollup snapshots (lifecycle → Glacier → expire)
<code>live-ninja-analytics-<acct></code>	Firehose telemetry lake, date-partitioned, Athena-queried

All buckets: block-public-access on, SSE-S3/AES256, versioning on assets. Transfers use presigned URLs (15-min PUT, 5-min GET) so bytes never transit Lambda.

8.3 Retention

Audio not stored by default (`storeAudio=false`). Transcripts stored if `storeTranscripts=true` (default 30-day) with DynamoDB TTL = now + `retentionDays` {0,7,30,90}; S3 audio (opt-in) uses lifecycle expiration matching retention. OpenAI called with zero-data-retention requested where available. Right-to-delete removes all `USER#<uid>` items, S3 objects, IoT things/certs, and LWA refresh tokens.

9. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	First assistant audio ≤ 800 ms (WebRTC) / < 1.2 s (M5Stack); barge-in ≤ 150 ms; ephemeral mint p99 within alarm threshold; Android wake $< 2\%$ /hr screen-off battery.
NFR-02	Security	OpenAI key only in SSM, IAM-scoped to broker/relay; refresh tokens hashed at rest; JWT signing via KMS (non-extractable); mTLS for IoT; TLS everywhere; strict CSP; XSS-safe rendering.
NFR-03	Privacy	On-device wake as a hard invariant; audio off by default; transcripts TTL'd + user-deletable; ZDR requested; no PII harvesting; consent recorded.
NFR-04	Cost	On-demand DynamoDB, Query/GetItem only; per-user metering enforced pre-spend; daily/monthly quotas; AWS Budgets at \$20/\$50/\$100 on <code>Project=live-ninja</code> ; CloudFront caching for static; direct-to-OpenAI audio (no AWS in media path).
NFR-05	Reliability	Fallback cascade on Realtime failure; WebRTC ICE-restart; M5Stack reconnect state machine with backoff; A/B OTA with rollback; idempotency on side-effecting operations.
NFR-06	Observability	Structured slog JSON with <code>requestId / userId / surface / route / latencyMs</code> ; CloudWatch EMF metrics; X-Ray on hot-path Lambdas; <code>live-ninja-ops</code> dashboard; alarms $\rightarrow$ SNS $\rightarrow$ SES.
NFR-07	Scalability	Serverless auto-scales; Fargate relay autoscaled on connection count; single-table + GSIs; S3/CloudFront snapshots for global lists; virtualized/paginated large lists.
NFR-08	Compatibility	Path-versioned <code>/v1</code> ; capability negotiation via <code>X-LN-Client / X-LN-Server</code> ; forward-compatible settings schema; <code>/v1</code> kept alive for the 10-year M5Stack field lifetime.

10. Security & Privacy

- **Always-listening disclosure:** on-device wake detection is architectural, not just policy — between wakes audio never leaves the device. Android shows a persistent `microphone`-FGS notification; web shows an always-visible mic-status chip and only accesses the mic after explicit `getUserMedia`; the

M5Stack shows a persistent on-screen listening indicator. First-run consent (timestamp + version) recorded at `CONSENT#<ts>`; Play data-safety declaration and prominent disclosure required.

- **On-device wake:** the load-bearing privacy control; only the post-wake utterance is streamed to OpenAI.
- **Data retention/deletion:** audio off by default; transcripts TTL'd with user-configurable retention; `DELETE /account` performs a partition-scoped Step Functions purge (no Scan) across DynamoDB, S3, IoT, and LWA refresh tokens with SES confirmation; per-item deletion available.
- **Credential protection:** refresh tokens stored only as hashes; LWA refresh tokens KMS-encrypted server-side; JWT signing key non-extractable in KMS; M5Stack key behind the DS peripheral (optional ATECC608B) with flash encryption + Secure Boot v2 + NVS encryption.

Threat model summary

Threat	Mitigation
OpenAI key leak to client	Key in SSM, broker-only IAM; 60 s config-bound ephemeral tokens; never in logs/env of <code>web</code> .
Auth-code interception	PKCE S256 (Android/M5Stack); confidential exchange (web).
Token substitution / confused deputy	Validate <code>aud == our client_id</code> via <code>/tokeninfo</code> ; identity from <code>/user/profile</code> .
Refresh-token theft + replay	Rotation with reuse detection → whole family revoked + SES alert.
Access-token theft	15-min TTL; <code>sid / did</code> binding; <code>tokensValidAfter</code> kill-switch.
XSS (web)	Refresh in <code>__Host-</code> HttpOnly cookie; JWT in memory; text-node rendering; strict CSP <code>script-src 'self'</code> .
CSRF (web)	<code>SameSite=Lax</code> + login <code>state</code> + Origin checks + double-submit token.
Device compromise / flash readout	Flash encryption + Secure Boot v2 + optional ATECC608; cert as second factor; rotation caps window; per-device IoT policy.
Cert theft / clone	Cert alone insufficient — rotating first-party token also required; reuse detection + IoT anomaly.
DynamoDB Scan / read-cost blowout	Query/GetItem only; <code>ConsumedReadCapacityUnits</code> alarm; rollups + S3 snapshots.
Realtime cost runaway	Pre-spend quota gate; idle/duration caps; anomaly auto-suspend + budget alarms.
Pairing hijack	<code>PAIR#</code> bound to device-generated nonce + PKCE verifier, 600 s TTL, single-use.
SES DMARC drop	Source is DKIM-verified <code>@jeremy.ninja</code> ; Reply-To gmail; bounce/complaint suppression.

11. Success Metrics / KPIs

ID	KPI	Target
KPI-01	First-audio latency (p95)	≤800 ms WebRTC / <1.2 s M5Stack
KPI-02	Barge-in stop latency (p95)	≤150 ms
KPI-03	Ephemeral token mint success rate	≥99.5%
KPI-04	Wake-word FRR @ fixed FAR (per engine, CI-gated)	No regression vs. baseline corpus
KPI-05	Android wake battery draw (screen-off idle)	<2%/hr
KPI-06	Realtime session availability (incl. fallback)	≥99.9% answered turns
KPI-07	Monthly AWS infra cost (<code>Project=live-ninja</code>)	Under budget alarms (\$20/\$50/\$100 tiers)
KPI-08	Per-user OpenAI cost	Within monthly ceiling (default ~\$15); zero over-cap sessions
KPI-09	DynamoDB read pattern	Zero <code>Scan</code> on serving paths; read units within baseline
KPI-10	Cross-surface settings sync convergence	<5 s for online clients; next-boot/apply for M5Stack
KPI-11	Deploy health	Push-to-main deploy green; alarms catch regressions within minutes
KPI-12	Accessibility	WCAG AA verified (axe/Lighthouse) in light + dark on web/Android

12. Assumptions, Dependencies, Risks

12.1 Assumptions

- Production-only shop; changes go straight to production guarded by local smoke tests, alarms, and canary rollouts.
- Single operator/user (Jeremy); no multi-tenant scale requirements.
- OpenAI GPT Realtime (`gpt-realtime`) remains available with WebRTC + WebSocket transports and ephemeral client secrets.
- `@jeremy.ninja` SES identity is DKIM-verified; SES production access obtained at launch.
- M5Stack Tab5 hardware (ESP32-P4 + C6) with the documented peripherals is the target device.

12.2 Dependencies

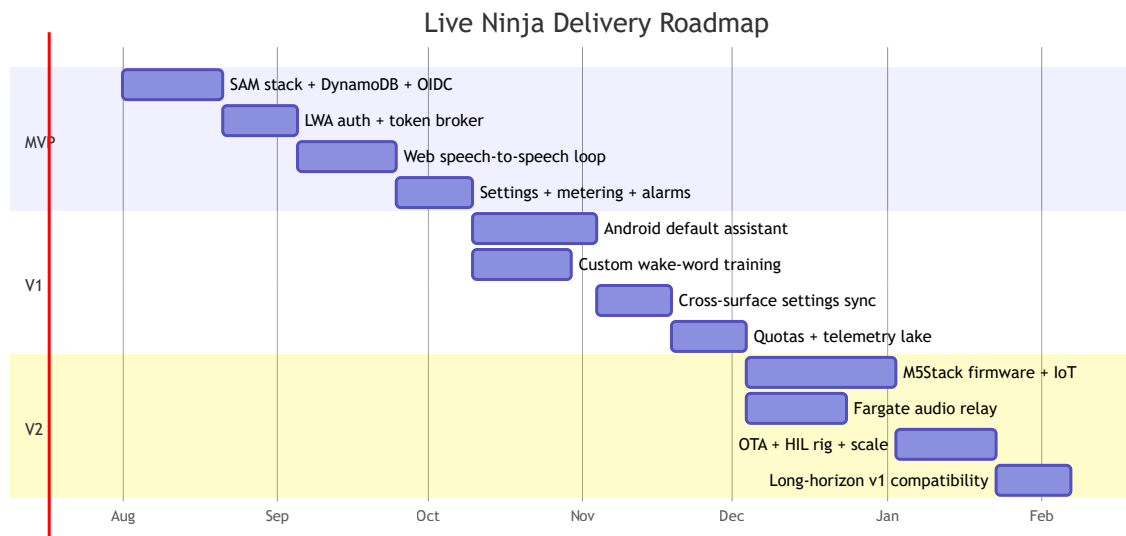
- OpenAI Realtime API; Login with Amazon (single Security Profile with registered return URLs); AWS (Lambda, HTTP API, DynamoDB, S3, SES, SSM, KMS, IoT Core, ECS Fargate, CloudFront, Route 53, EventBridge, SQS, CloudWatch/X-Ray).
- GitHub Actions OIDC `gha-deploy` role (`vars.AWS_DEPLOY_ROLE_ARN`); ACM cert, hosted zone, artifact bucket via `vars`.
- Picovoice Console (Porcupine training) — optional; openWakeWord (Apache-2.0) is the never-blocked fallback.

12.3 Risks

Risk	Impact	Mitigation
OpenAI API key leakage to clients	Severe (cost/abuse)	Key only in SSM, broker-scoped; short-TTL config-bound ephemeral tokens; never in logs/env of <code>web</code> .
DynamoDB Scan cost blowout	High (silent bill)	Single-table Query/GetItem; <code>ConsumedReadCapacityUnits</code> alarm; rollups + S3 snapshots; Scan only for manual migrations.
Realtime audio latency/cost if relayed through AWS	High	Direct client↔OpenAI WebRTC/WS; AWS never in audio path.
ESP32-P4 can't run WebRTC well	High (M5Stack blocked)	Backend-proxied WSS + Opus relay on Fargate; device runs libopus + WSS + esp-sr only.
10-year M5Stack credential compromise	High	Store only peppered hash / KMS-encrypted refresh; short-lived working tokens; cert second factor; rotation; instant server-side revoke.
SES mail silently dropped (DMARC)	Medium (missed alerts)	DKIM-verified <code>@jeremy.ninja</code> Source, Reply-To gmail; never send from gmail identity; suppression list.
Runaway OpenAI/AWS cost (abandoned/always-listening)	High	Local wake (no continuous cloud audio); idle/duration caps; pre-spend quotas; org hard limit; budget + cost alarms.
Android <code>ROLE_ASSISTANT</code> not honored on OEMs	Medium (UX friction)	OEM-aware guided walkthrough + deep links + <code>isRoleHeld</code> polling; wake word does not depend on the role.
Play Store rejection (background mic + assistant + FGS-mic)	Medium	Default to signed-APK sideload/internal testing; prominent disclosure + consent logging; full declarations only if going public.
Session-token theft (30-day web/Android)	Medium	15-min access JWT + rotating refresh with reuse detection; <code>HttpOnly</code> cookies; new-login SES alert; per-session revoke.
Stale HTML after deploy (web)	Low/Medium	Network-first HTML SW + <code>no-cache</code> headers + content-hashed immutable assets + versioned cache purge.

Risk	Impact	Mitigation
Settings sync conflicts across 3 devices	Medium	Monotonic version + <code>ConditionExpression</code> optimistic concurrency; DynamoDB source of truth; higher version wins.
Bad OTA bricks the M5Stack	Medium	A/B partitions, signed images, anti-rollback eFuse, mark-valid-after-check-in, canary group, coordinated P4→C6 gating.
OpenAI outage / 429	Medium	Backoff → chained STT→LLM→TTS → text-only → graceful failure with side-effects queued; alarm on broker error rate.
Long-lived M5Stack devices outrun API changes	Medium	<code>/v1</code> kept alive for field lifetime; capability negotiation + <code>/compat</code> ; forward-compatible schema.
No staging (production-only)	Medium	<code>dynamodb-local</code> + mocked OpenAI/IoT integration tests gate deploy; HIL rig for M5Stack; fast SES-alerting alarms; additive-only API changes.

13. Delivery Roadmap



diagram

14. Open Questions with Chosen Defaults

#	Question	Chosen default (baked in)
Q-1	Fiber-on-Lambda via LWA or the proxy adapter?	Lambda Web Adapter — one idiomatic Fiber codebase, streaming/SSE support, local/prod parity, native arm64.
Q-2	HTTP API v2 or REST API v1?	HTTP API v2 — ~70% cheaper, native JWT authorizer, lower latency; <code>\$default</code> → Fiber.
Q-3	Single-table or multi-table DynamoDB?	Single table <code>live-ninja</code> with two GSIs; all access by known keys; Query/GetItem only.
Q-4	Audio transport through AWS or direct to OpenAI?	Direct client ↔ OpenAI (WebRTC web/Android, WSS relay for M5Stack); AWS never in the media path.
Q-5	Secrets store?	SSM Parameter Store SecureString (+ KMS for signing/refresh encryption); no secrets manager.
Q-6	M5Stack Realtime transport?	Backend-proxied WSS + Opus via a Fargate relay — on-device WebRTC on the radio-split P4 is impractical.
Q-7	Are broker/authorizer separate Lambdas from day one?	Yes — broker holds the crown-jewel key policy; authorizer is on every hot path (keep tiny for cold-start).
Q-8	Android wake engine?	Porcupine primary, openWakeWord fallback , behind a <code>WakeWordEngine</code> interface; own FGS engine, not the DSP hotword.
Q-9	Web wake word default state?	Off by default, opt-in ; <code>openWakeWord</code> WASM default, Porcupine optional; click-to-talk always available.
Q-10	Android distribution?	Signed-APK sideload / internal-testing to sidestep Play assistant/always-on-mic review; full declarations only if going public.
Q-11	Web front-end framework?	Go-Fiber server-rendered HTML + progressive-enhancement ES modules — no SPA framework.

#	Question	Chosen default (baked in)
Q-12	M5Stack custom wake phrases?	Select-only on device (curated flashable WakeNet set + oWW-ESP fallback); phrase creation on web/Android.
Q-13	Session signing algorithm?	ES256 via non-extractable KMS CMK , JWKS published; access JWT 15 min.
Q-14	Default voice / persona?	Voice cedar (user-selectable from enum); shared "concise, action-oriented" persona with per-user overrides.
Q-15	Transcript storage default?	On, 30-day TTL (assistant needs short-term memory); audio off by default.
Q-16	Per-user OpenAI quota?	~30 min realtime audio/day + monthly token ceiling (~\$15) , enforced pre-spend at mint; 10-min hard session cap; soft-warn at 80%.
Q-17	Confirm-before-send for email tool?	Yes for external recipients , with per-user daily send cap and optional allowlist; idempotency keys.
Q-18	M5Stack secure element?	ATECC608B recommended default on production units for the 10-year field credential.
Q-19	Telemetry/control transport for M5Stack?	IoT Core MQTT (control-plane only); audio uses the WSS relay, never IoT.
Q-20	Cost center tag?	voice-ai ; full mandatory tag set applied once at stack level in <code>samconfig.toml</code> .

UI Wireframe Specifications

Screen-by-screen wireframes and control/data-source specs for every function on each surface.

I'll write the markdown section directly as requested.

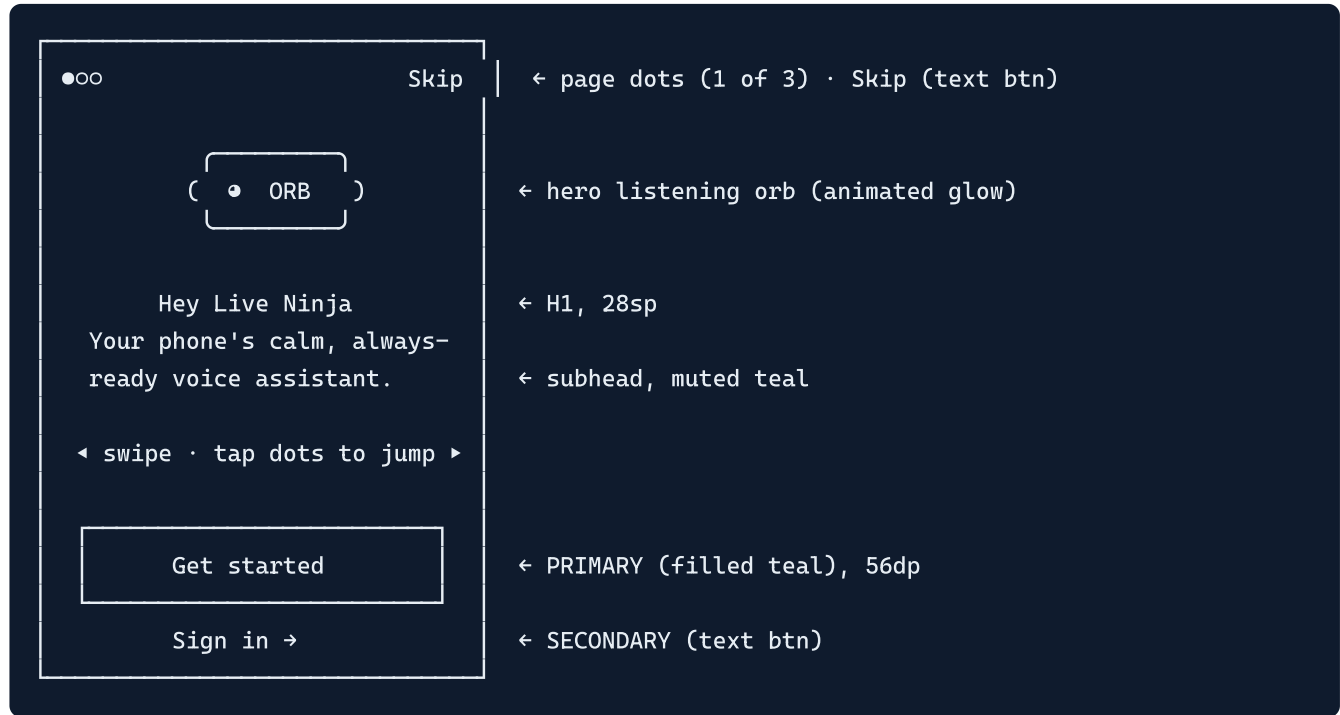
Android (mobile, primary assistant)

Rich-UI surface. Deep-navy canvas (#0A1230), electric-teal/cyan accent (#22E3D6), a persistent "listening orb" motif. Full-screen per screen; bottom tab bar (Home · History · Wake Words · Settings) is global once onboarded. All controls are native Android/Material semantic widgets: real `Button` , `Switch` ,

`Spinner` / `ExposedDropDownMenu`, `Slider`, `RadioGroup`, with visible labels, WCAG-AA contrast in the dark theme, visible focus rings, and ≥ 48 dp touch targets. Login with Amazon gates everything below the onboarding carousel.

1. Onboarding — First-run welcome carousel

Purpose: Introduce Live Ninja as the phone's primary voice assistant and route to sign-in or setup.



- **Carousel** — `ViewPager2`, 3 swipeable slides (What it is / Wake words / Privacy). DATA SOURCE: static slide array. Page dots show `N of 3`.
- **Get started** — primary filled `Button` → Login with Amazon (screen 2).
- **Sign in** — secondary text `Button` → same LWA screen (returning users).
- **Skip** — text `Button`, top-right → LWA.
- States: none (static content).
- A11y: each slide has a heading + `contentDescription`; orb is decorative (`importantForAccessibility="no"`); dots announce "Page N of 3"; swipe has button-equivalent (dots are tappable); focus order top→bottom.

2. Login with Amazon

Purpose: Authenticate via LWA only (no email/password), set expectations for the 30-day session.



Live Ninja
Sign in to continue

a Continue with Amazon

🔒 Your session stays active
for 30 days on this phone.

We use Login with Amazon only
to identify you. We never see
your password.

By continuing you agree to
the Terms & Privacy Policy →

← H1

← PRIMARY (Amazon-brand btn), 56dp

← info row

← privacy note

← link (opens legal)

- **Continue with Amazon** — single primary `Button` launching the LWA OAuth flow. DATA SOURCE: LWA SDK. No email/password fields by design.
- **Terms & Privacy Policy** — text link → in-app browser.
- States: **loading** (button shows spinner + “Connecting to Amazon...”, disabled to block double-tap); **error** (inline banner above button: “Couldn’t reach Amazon. Check your connection and try again.” + Retry, `role="alert"`).
- A11y: button has accessible name “Continue with Amazon”; lock/privacy text is real text (not baked into an image); link is a focusable control with visible focus; error announced via `aria-live / announceForAccessibility`.

3. Set as Default Assistant

Purpose: Guide the user to grant `ROLE_ASSISTANT` so “Hey Live Ninja” works system-wide.

← Set as default assistant

Status: ● Not set

Make Live Ninja your phone's assistant so it answers from anywhere.

- ✓ 1 Open Android settings
- 2 Tap "Digital assistant app"
- 3 Choose Live Ninja

Open Android settings

I'll do this later

← top app bar

← status badge (amber=Not set / teal=Active) – live from RoleManager

← stepper checklist

← PRIMARY (Intent → settings)

← SECONDARY (text)

- **Status badge** — read-only chip; DATA SOURCE: `RoleManager.isRoleHeld(ROLE_ASSISTANT)`, refreshed on resume. Values from a fixed enum: `Not set` / `Active`.
- **Stepper checklist** — ordered step list; items auto-check when the role is detected on return. DATA SOURCE: role state.
- **Open Android settings** — primary `Button` firing `ACTION_VOICE_INPUT_SETTINGS` / role-request intent.
- **I'll do this later** — secondary text `Button` → Home (with a reminder chip on Home).
- States: **Active** → badge turns teal, primary swaps to "Done" and steps show all-checked; **Not set** → amber badge persists.
- A11y: badge conveys state with icon + text (not color alone); steps are a labeled list announcing "Step 2 of 3, not done"; live status change announced.

4. Permissions

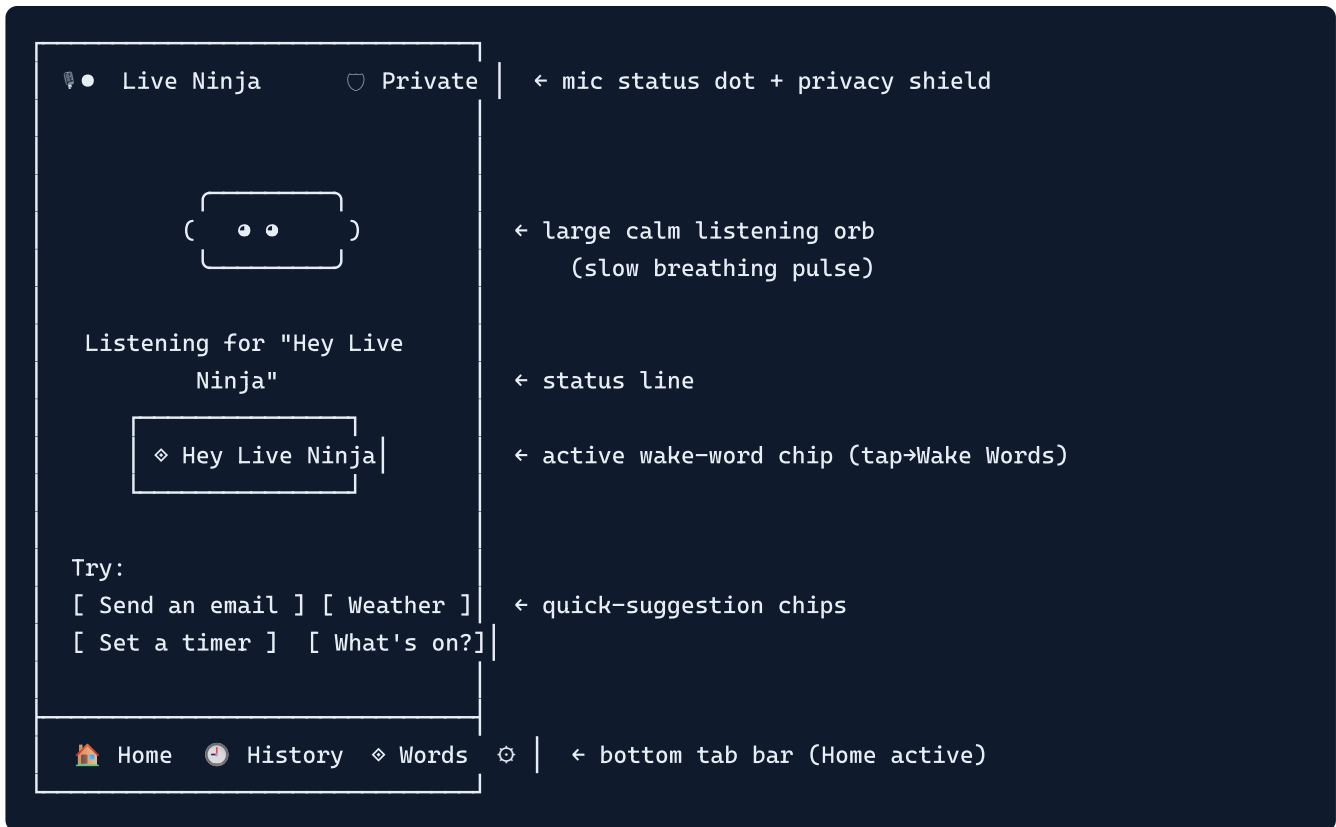
Purpose: Explain and request the three runtime permissions in plain language before asking the OS.



- **Per-permission card** — each pairs a plain-language “why” with a **Grant** Button that flips to a Granted state (or “Open settings” if permanently denied). DATA SOURCE: `ContextCompat.checkSelfPermission` per permission, live.
- **Continue** — primary Button, enabled once Microphone (the hard requirement) is granted; foreground-service + notifications are recommended-but-optional and noted as such.
- States: each card = *Not granted* / *Granted* / *Denied* → *Open app settings*; mic denied disables Continue with helper text “Microphone is required for wake-word listening.”
- A11y: each card is a `fieldset`-style group with heading + description tied to the button via `aria-describedby`; state shown by icon + text; buttons announce result (“Microphone granted”); AA contrast on cards over navy.

5. Home / Idle (wake-listening)

Purpose: Primary resting screen — calm proof that the assistant is listening, plus quick entry points.



- **Listening orb** — animated status visual; DATA SOURCE: wake-engine state (idle→pulsing). Tap = start conversation manually (barge-in without wake word).
- **Mic status dot / privacy shield** — indicators; DATA SOURCE: mic capture state + on-device-processing flag. Tapping shield opens Privacy settings.
- **Active wake-word chip** — read-only chip showing the current active phrase; DATA SOURCE: wake-word store (active record). Tap → Wake-Word Manager (screen 7).
- **Quick-suggestion chips** — `ChipGroup`; DATA SOURCE: curated capability list + user's recent intents (known set, not free text). Tap runs the suggested action.
- **Bottom tab bar** — 4 destinations, always visible.
- States: **listening** (default), **muted** (orb greyed, "Muted — tap to resume", from mic-mute toggle), **role-not-set** (amber reminder chip → screen 3).
- A11y: orb decorative; status line is the live source of truth (`aria-live="polite"` on state change); mic/shield have text labels; tab bar items have labels + selected state; 48dp targets.

6. Active Conversation

Purpose: Show a live, trustworthy conversation with visible tool results and easy interruption.



- **State pill** — read-only status; DATA SOURCE: realtime session state enum (`Listening` / `Thinking` / `Speaking`). Icon + text.
- **Live transcript** — chat-style `RecyclerView`; DATA SOURCE: streaming ASR (user) + model tokens (assistant). User bubbles right, assistant left; auto-scroll with "jump to latest" if scrolled up.
- **Tool-call result card** — structured card (not raw JSON); DATA SOURCE: tool-invocation result payload rendered to labeled fields (channel icon, status ✓/X, key params). Tap → detail (e.g., open sent email summary).
- **Stop** — primary destructive-weight `Button` ends the turn/session. **Mute** — toggle `Button` mutes mic mid-conversation.
- **Barge-in hint** — static helper text; interruption is by voice or tapping the orb.
- States: **thinking** (orb spins, pill "Thinking", typing indicator bubble); **error** (assistant bubble error style: "That tool failed — want me to retry?" + Retry); **tool running** (card skeleton "Sending..." → resolves to ✓/X).
- A11y: transcript bubbles labeled by speaker for screen readers; state pill `aria-live="assertive"`; tool card has a text status ("Email sent, success"); Stop/Mute have distinct accessible names and are the largest targets; barge-in hint associated with the orb control.

7. Wake-Word Manager

Purpose: Manage the set of programmable wake words; each can be toggled, tested, or edited.



- **Wake-word list** — `RecyclerView` of cards; DATA SOURCE: wake-word store (user's saved words). Each card: name (label), **sensitivity badge** (fixed enum `Low / Balanced / High`), **Active** `Switch`, **Test** `Button`. No free-text here — words come from the saved set.
- **Test** — per-item `Button` plays a mic-listen check and shows "Detected ✓ / Not detected" inline.
- **Create custom wake word** — primary `Button` → screen 8.
- **Row overflow** (long-press / `:`): Edit, Delete (guarded confirm).
- States: **empty** (illustration + "No wake words yet. Create your first one." + primary CTA); **loading** (3 skeleton cards); **test running** (card shows listening spinner).
- A11y: each card is a group; Switch announces "Hey Live Ninja, Active, on/off"; sensitivity badge is text; Test result announced live; header shows count ("Showing 4").

8. Create / Train Wake Word

Purpose: Create a wake phrase from suggestions or a novel phrase, tune sensitivity, and train by voice sample.

← New wake word

Choose a phrase

Suggested:

- Hey Ninja
- OK Ninja
- Hey Live One
- 🔍 Use my own phrase...

Sensitivity

Low ————— High

Balanced

Train your voice (2 of 3)

☒ Sample 1 ☒ Sample 2
☐ Sample 3 [☐ Record]

← section label

← autocomplete combobox (case 4)

← list-select from suggested set

← reveals validated text field

← Slider (discrete: Low/Balanced/High)

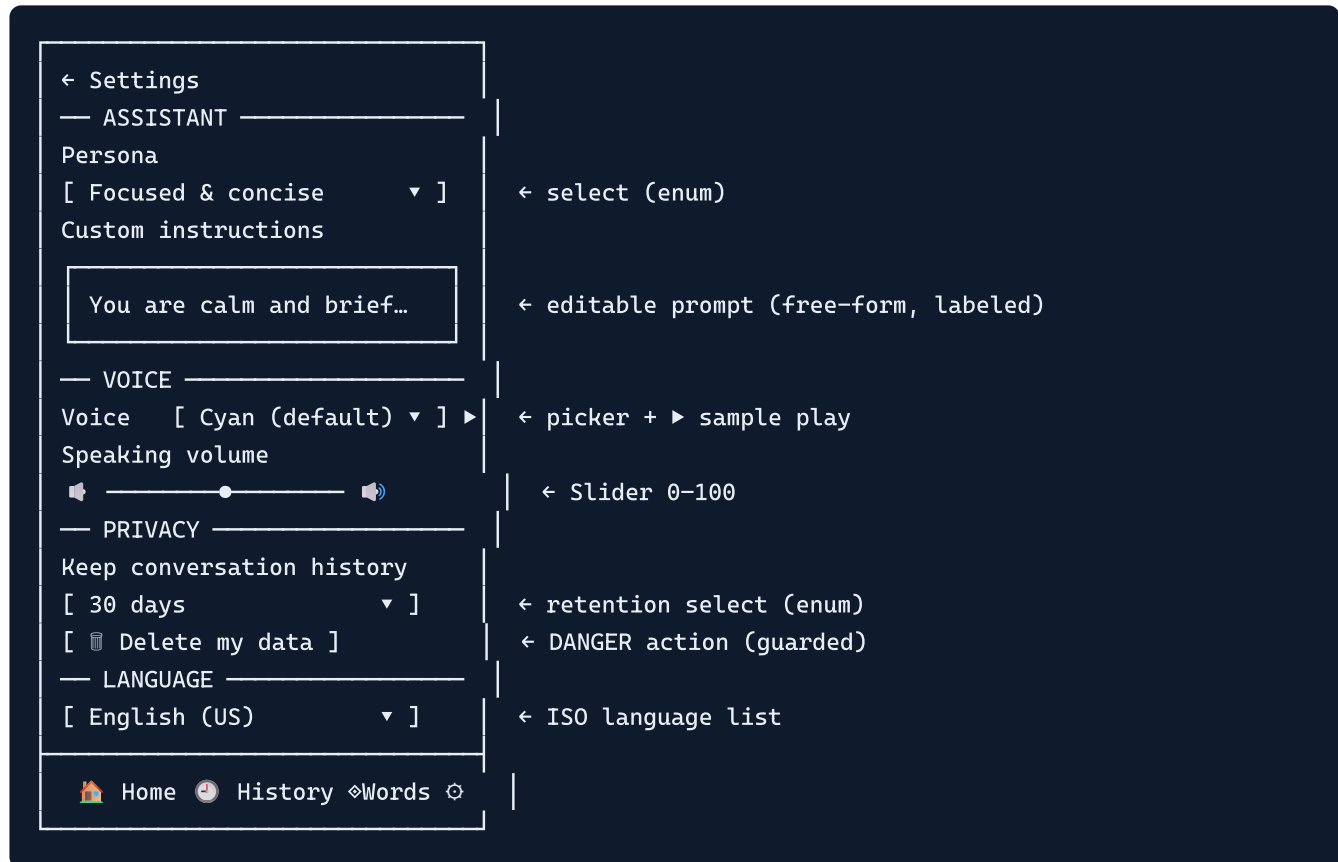
← current value label

← stepper w/ progress

← Test (2ndary) · Save (PRIMARY)

- **Phrase picker** — autocomplete combobox (Prime Directive case 4): DATA SOURCE = suggested-phrase list; surfaces suggestions *and* accepts a novel phrase. “Use my own phrase” reveals a labeled text field (the one legitimate free-text, for a truly novel value) with inline validation.
- **Suggested phrases** — `RadioGroup` list-select; DATA SOURCE: curated suggestion set + phonetically-safe recommendations.
- **Sensitivity** — discrete `Slider` (3 stops, value label under thumb). DATA SOURCE: fixed enum, defaults to `Balanced`.
- **Train stepper** — 3-sample recorder with `N of 3` progress; each **Record** captures a sample → ✓. DATA SOURCE: on-device training buffer.
- **Test** — secondary `Button` (enabled after ≥1 sample). **Save** — primary `Button`, disabled until phrase valid + 3 samples captured.
- Validation (inline): phrase length 2–4 words, “Too short — use at least two syllables”, duplicate-of-existing blocked with message; errors adjacent to field.
- States: **recording** (waveform + “Speak now...”); **sample failed** (“Too quiet, tap to re-record”); **save disabled** with reason text.
- A11y: combobox full keyboard nav (arrows+Enter); slider announces “Balanced”; stepper announces “Sample 3 of 3, recording”; Save disabled reason via `aria-describedby`; all inputs have visible `<label>`.

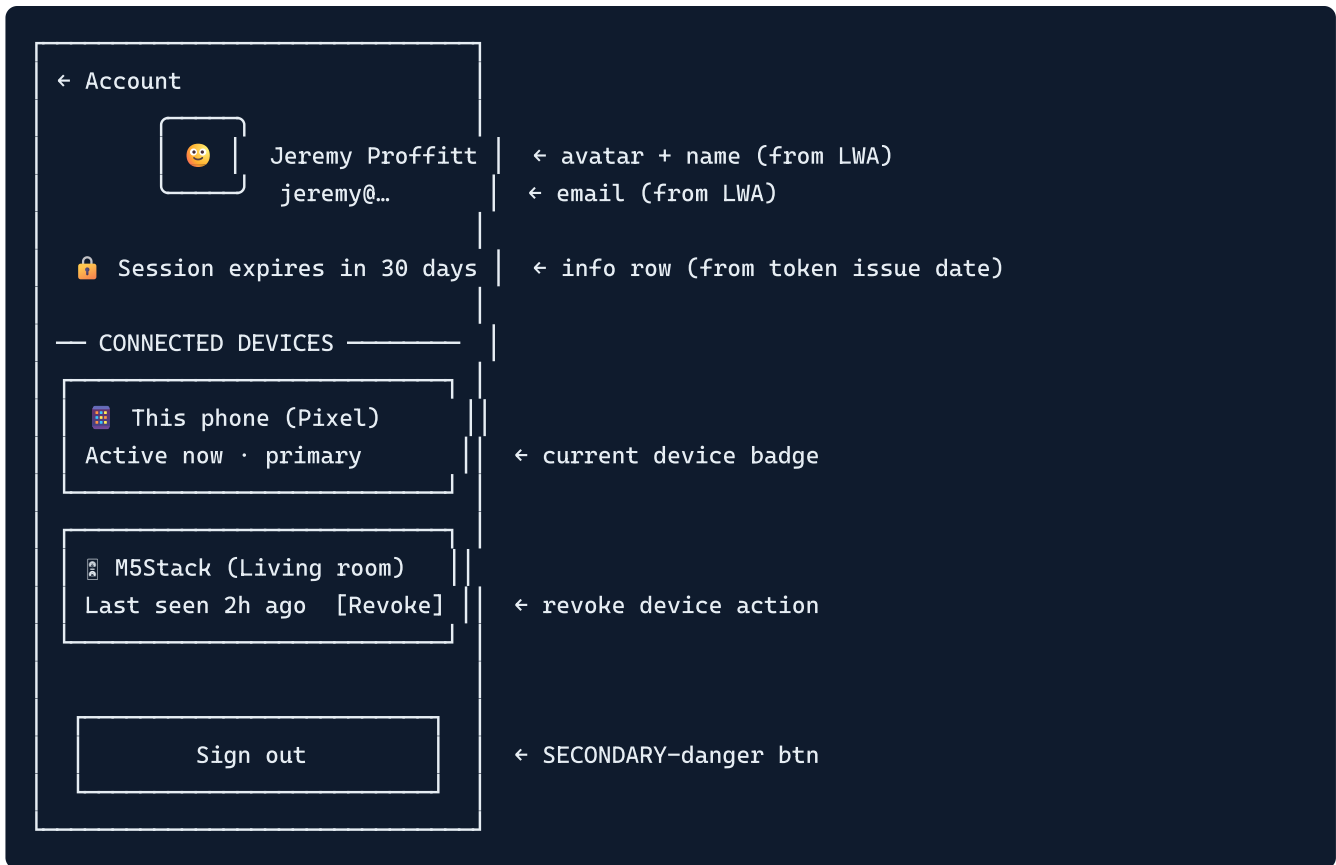
Purpose: Configure persona, voice, volume, privacy, and language — every control populated from a known set.



- **Persona** — `Spinner` /dropdown; DATA SOURCE: persona enum (Focused & concise / Friendly / Professional / Playful / Custom). **Custom instructions** — labeled multiline field (legitimate free text) editable when Persona=Custom or to refine any preset.
- **Voice** — dropdown of available TTS voices with an inline `► sample` `Button`; DATA SOURCE: GPT-Realtime voice catalog. Shows human-readable names, not IDs.
- **Speaking volume** — `Slider` 0-100 with icons; live-effect setting.
- **Keep history / retention** — `select`; DATA SOURCE: fixed options (Off / 7 days / 30 days / 1 year / Forever), default 30 days.
- **Delete my data** — destructive `Button`, guarded: confirmation dialog requiring typed "DELETE" for irreversible wipe.
- **Language** — `select` from ISO 639 / locale list; DATA SOURCE: supported-locale list, default = device locale.
- States: voice **sample loading/playing** (button spinner→pause); **saved** confirmation toast per change (autosave); delete flow shows pending + explicit success.
- A11y: every control has a persistent visible `<label>`; sections use headed groups; slider/select fully keyboard-operable; destructive action visually subordinate + confirmed; AA contrast for section dividers and danger red on navy.

10. Account & Sessions

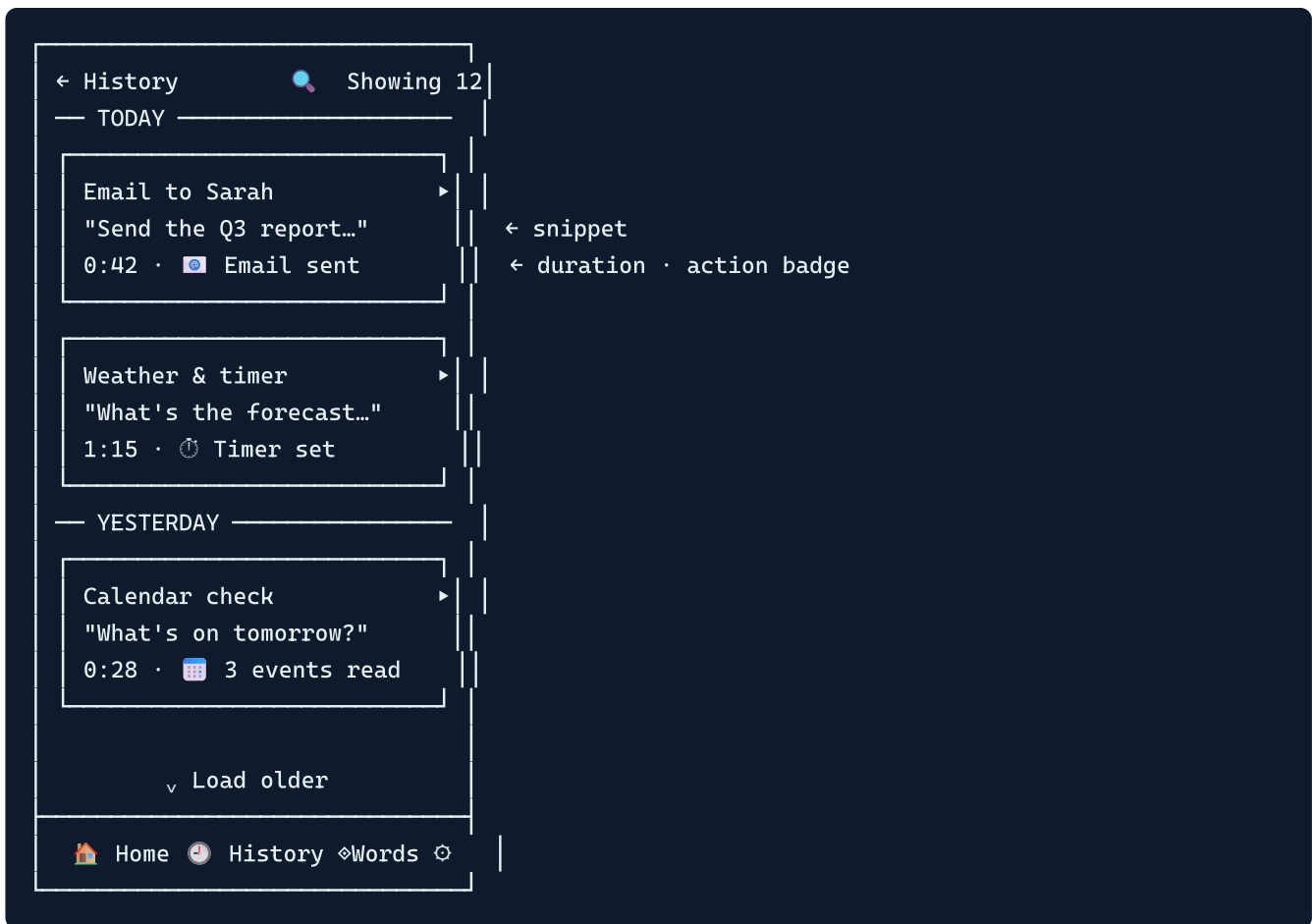
Purpose: Show the LWA identity, session lifetime, and connected devices with revoke/sign-out.



- **Profile** — read-only avatar/name/email; DATA SOURCE: LWA profile response. Avatar has alt text.
- **Session info** — read-only row; DATA SOURCE: token issue time + 30-day policy → computed countdown.
- **Connected devices** — structured list/cards; DATA SOURCE: session registry (this phone + M5Stack). Each shows name, location/last-seen, status badge. Current device marked “Active now · primary” and is not revokable from itself (only sign-out).
- **Revoke** — per-device destructive `Button` with confirm (“Revoke M5Stack? It’ll need to sign in again.”).
- **Sign out** — danger-weight secondary `Button`, confirm dialog; clears the 30-day session.
- States: **loading** (skeleton profile + 2 device rows); **error** (“Couldn’t load account” + retry); **empty devices** (just this phone).
- A11y: status badges are icon+text; revoke buttons have device-specific accessible names (“Revoke M5Stack”); sign-out confirmation traps focus; countdown text is real text, updated `aria-live` politely.

11. Conversation History

Purpose: Browse past conversations grouped by date, with snippets, duration, and actions taken.



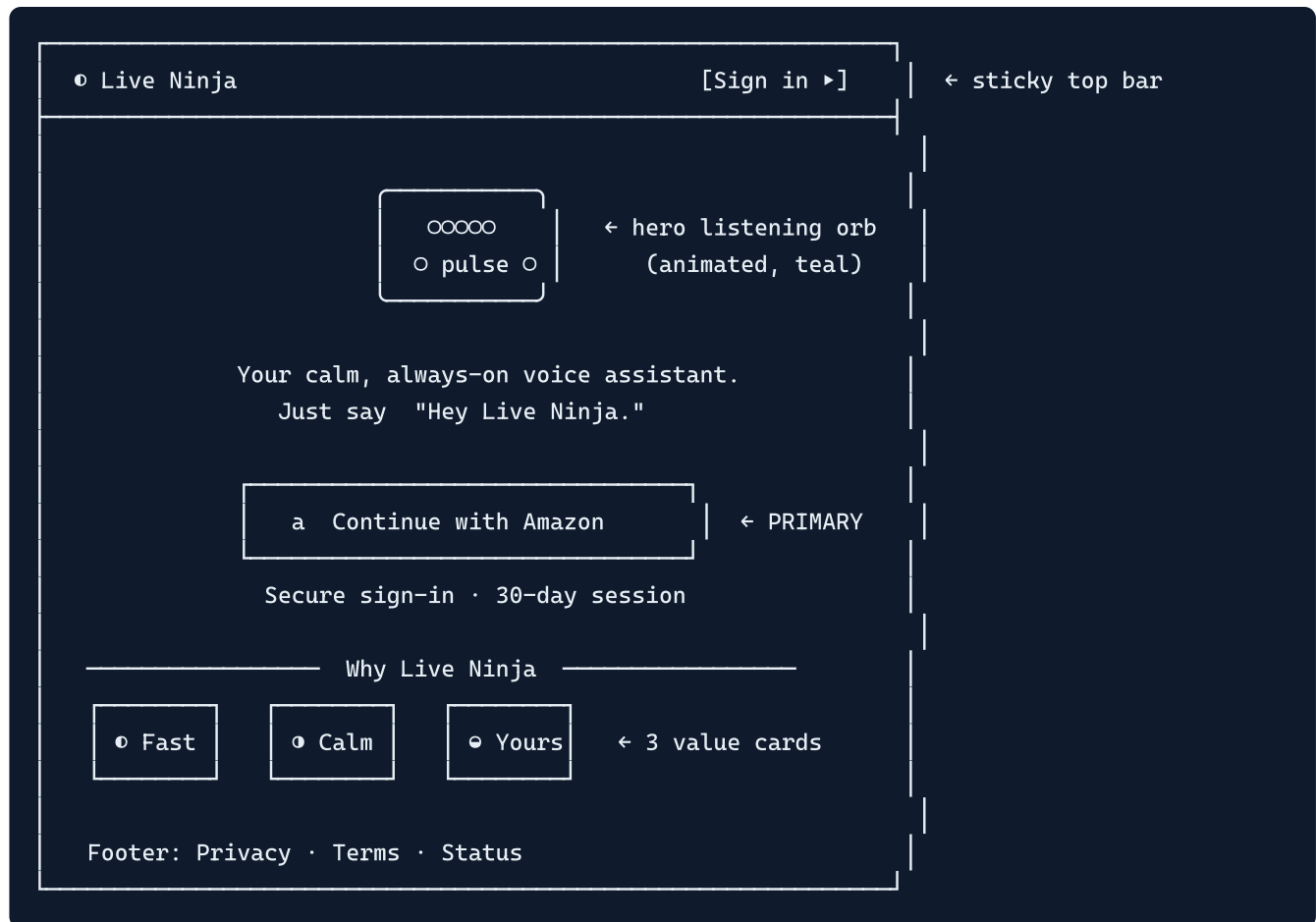
- **Grouped list** — `RecyclerView` with sticky date headers (Today / Yesterday / date); DATA SOURCE: conversation history store, sorted desc, paginated. Each row: title, snippet, duration, and **action badges** (fixed enum of tool types: Email / Timer / Calendar / Search / etc.).
- **Tap row** — expands/opens full transcript detail (user + assistant bubbles, same styling as screen 6, read-only) with the tool-result cards inline.
- **Search** — opens a query field (legitimate free-text search over history).
- **Load older** — pagination control; virtualized list, "Showing 12".
- States: **empty** (illustration + "No conversations yet. Say 'Hey Live Ninja' to start."); **loading** (skeleton rows under a date header); **error** ("Couldn't load history" + Retry).
- A11y: date headers are real headings for screen-reader navigation; each row is a single focusable item announcing "Email to Sarah, 42 seconds, email sent"; action badges are icon+text; expand control states announced (collapsed/expanded); search field has visible label.

Website (Go-Fiber served)

Full-screen rich-UI surface (Go-Fiber server-rendered templates + progressive JS). Deep-navy canvas, electric-teal/cyan accent, recurring "listening orb" motif. Login with Amazon (LWA, 30-day cookie) gates every route except Landing. Semantic HTML, WCAG AA in the navy theme, visible focus rings, ~44px targets, no blind free-text where a value set is known.

1. Landing / Login

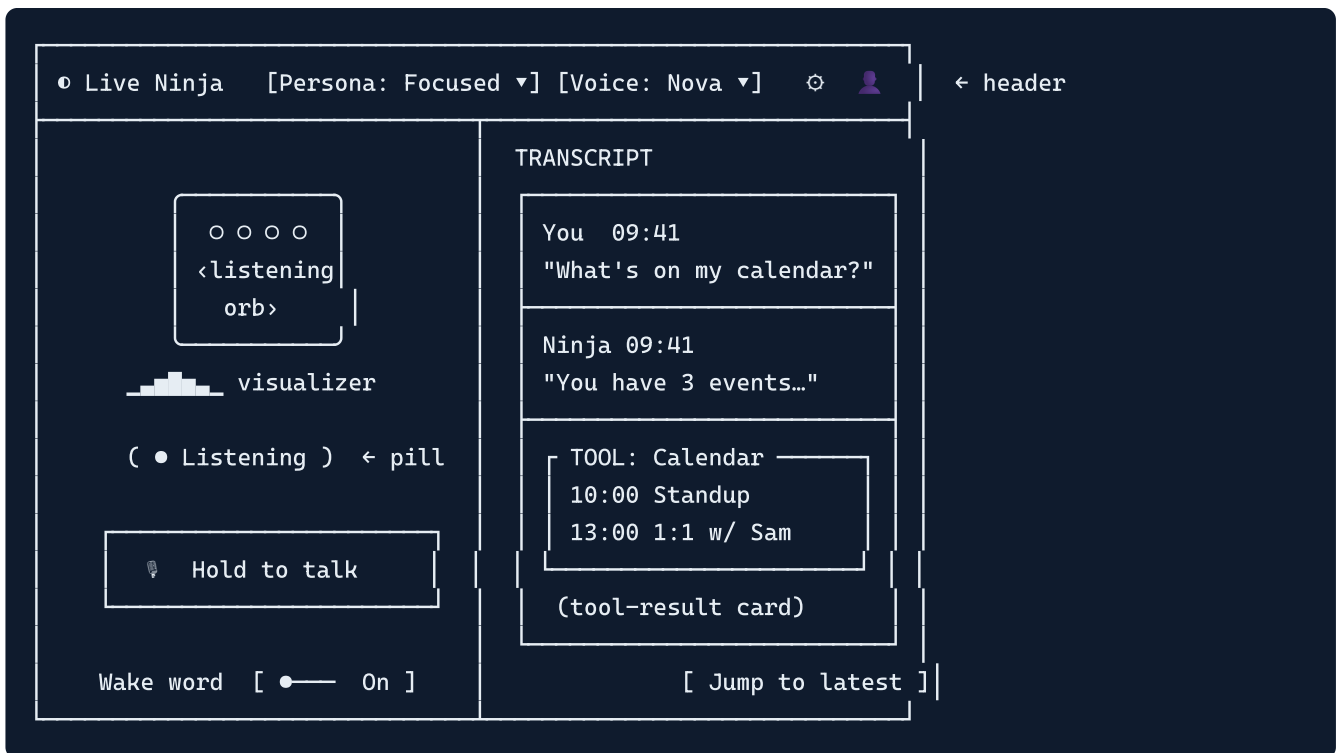
Purpose: Unauthenticated marketing-lite entry; single conversion goal — sign in with Amazon.



- **Continue with Amazon** — `<button>` /branded LWA anchor (PRIMARY). Data source: LWA OAuth endpoint. Same button appears in top bar (secondary emphasis) and hero (dominant). One conversion action, two placements — consistent label.
- **Actions:** Primary = LWA sign-in. Secondary (footer) = Privacy / Terms / Status links.
- **States:** Post-click → button shows pending spinner + disabled (blocks double-submit). OAuth error → inline banner above button: "Sign-in didn't complete. Try again." with retry. Already-authed visitor → redirect to Conversation.
- **Mobile reflow:** Single column; value cards stack vertically; orb scales down (~40vw); top-bar CTA collapses into the hero button only to avoid duplicate targets; hero button spans full width (min 44px height).
- **A11y:** `<main>` landmark, one `<h1>` (value prop), orb is decorative (`aria-hidden` , respects `prefers-reduced-motion` → static). LWA button has accessible name "Continue with Amazon"; visible focus ring; logical tab order (bar CTA → hero CTA → cards → footer).

2. Conversation

Purpose: The core voice loop — talk, watch state, read transcript, act on tool results.



- **Push-to-talk** — `<button>` "Hold to talk" (press-and-hold / Space-bar hold). PRIMARY action. Immediate visual + pressed state.
- **Wake-word toggle** — toggle switch (live-effect boolean). Data source: user wake-word config (see #3); off → disables always-listening, PTT still works.
- **State pill** — status badge, values from a fixed enum {Idle, Listening, Thinking, Speaking, Error} — color + icon + text label (never color alone).
- **Persona quick-switch** — `<select>` (6–20 range). Data source: personas list from Settings.
- **Voice quick-switch** — `<select>`. Data source: available TTS voice catalog.
- **Tool-result cards** — structured cards (calendar/list/etc.), never raw JSON; each shows source label + human-readable rows.
- **Actions:** Primary = talk (PTT). Secondary = wake toggle, persona/voice switch, "Jump to latest," per-message copy.
- **States:** Empty → orb idle + hint "Hold the button or say 'Hey Live Ninja.'" Loading/Thinking → orb pulse + pill "Thinking..." + skeleton transcript row. Mic-permission denied → inline banner with "Enable microphone" retry. Network drop → pill "Reconnecting..." (distinct error style).
- **Mobile reflow:** Transcript moves below the orb (single column, scroll); header switches persona/voice into a "... menu; PTT pinned as full-width bottom bar.
- **A11y:** Transcript is an `aria-live="polite"` log; state pill announced via `aria-live`; PTT operable by keyboard (Space hold) with accessible name; orb + visualizer `aria-hidden`, reduced-motion honored; focus never trapped in the live region.

3. Wake-Word Settings

Purpose: Manage the set of wake phrases as real data — view, choose the active one, add/edit/remove.

< Settings / Wake Words [+ Add wake word]				
Active	Name	Sensitivity	Status	Actions
☒	Hey Live Ninja	High	● Active	Edit ...
☐	Ninja	Medium	○ Ready	Edit ...
☐	Yo Assistant	Low	○ Ready	Edit ...
Showing 3 of 3 wake words				

— "+ Add wake word" opens dialog —

Add wake word ×

Phrase * [Hey Live Ninja ▼] ← picker
suggests: "Hey Ninja", "OK Ninja"...
+ type a new phrase (autocomplete)

Sensitivity Low ————●———— High [Medium]

Mic test [🗨 Test in browser] ▮ detected ✓
[Cancel] [Save wake word]

- **Active** — radio group (mutually exclusive, one active). Data source: the wake-word table rows.
- **Status** — badge from enum {Active, Ready, Training, Error}.
- **Phrase (add form)** — autocomplete combobox: suggests curated phrases from a known preset list **and** allows a novel phrase (case 4 — open-ended but suggestible). Never a bare text box.
- **Sensitivity** — slider (bounded range Low↔High) with visible current value label; per-row shown as read-only badge.
- **Mic test** — `<button>` browser mic capture; shows live meter + detected ✓/X against the phrase. Data source: WebAudio + local detector.
- **Actions:** Primary = "Add wake word" / "Save wake word." Row: Edit, Delete (guarded confirm). Radio = set active (autosave).
- **States:** Empty → "No wake words yet. Add one to start talking." + primary CTA. Save pending → disabled button + spinner. Mic denied → inline "Allow mic to test" with retry. Duplicate phrase → inline validation on the field.
- **A11y:** Real `<table>` with `<th scope>`; active radios in a `<fieldset><legend>Active wake word`; dialog traps focus, Esc closes and restores focus; slider has `aria-valuenow/min/max`; mic-test result announced via `aria-live`.

4. Settings

Purpose: Configure persona, voice, audio, language, privacy — the assistant's behavior.

< Back
Settings

PERSONA

Persona *
[Focused ▼]

Instructions

You are calm, concise, and proactive...

VOICE & AUDIO

Voice *
[Nova ▼]
[▶ Preview]

Volume

70%

Language *
[English (US) - en-US ▼]

PRIVACY

Retention *
[Keep 30 days ▼]

DANGER ZONE

Delete all conversation data ...
[Delete my data]

[Cancel]
[Save changes]

- **Persona** — `<select>` (choose preset) + editable `<textarea>` for instructions (select+editable per rule). Data source: persona presets; textarea pre-filled from the selected persona, user-editable.
- **Voice** — `<select>` + "Preview" `<button>` that plays a sample. Data source: TTS voice catalog.
- **Volume** — slider, `min=0 max=100 step=5`, visible % value.
- **Language** — `<select>` from ISO/IANA locale list (e.g. en-US, es-ES). Never free text.
- **Retention** — `<select>` from enum {Keep forever, Keep 90 days, Keep 30 days, Keep 7 days, Don't store}.
- **Delete my data** — destructive `<button>`, subordinate styling; opens typed-"DELETE" confirmation modal.
- **Actions:** Primary = "Save changes." Secondary = Cancel, Preview. Guarded = data deletion.
- **States:** Inline validation on blur (persona name required, textarea non-empty). Save → pending + disabled, then success toast "Settings saved." Preview loading → button spinner. Preserve all input on validation error.
- **A11y:** Every field a visible `<label for>`; sections as `<fieldset>` / `<legend>` on consistent spacing; errors via `aria-describedby` + `role="alert"`; slider keyboard-operable; delete confirm traps focus.

5. Account & Devices

Purpose: Show the signed-in account and every connected device; revoke sessions; pair new devices.

< Back Account & Devices				
 Jeremy P. · signed in with Amazon · [Sign out]				
Connected devices			[+ Pair a device]	
Device	Type	Last seen	Session	Actions
Living Room M5	M5Stack	2 min ago	10 years	Revoke
Jeremy's Pixel	Phone	1 hr ago	30 days	Revoke
Chrome · Windows	Web	now (this)	30 days	—
Showing 3 of 3 devices				

- **Devices table** — real `<table>`: Device (identity, leftmost), Type badge {M5Stack, Phone, Web}, Last seen (relative, one canonical format), Session/persistence (10 years for M5Stack, 30 days for phone/web), Actions (rightmost).
- **Revoke** — per-row destructive `<button>`; current session row shows “—” (can’t revoke self; use Sign out). Guarded confirm: “Revoke Living Room M5? It will need to pair again.”
- **Pair a device** — PRIMARY `<button>` → pairing flow (QR + short code). Data source: pairing service.
- **Sign out** — secondary `<button>` (ends current 30-day cookie).
- **Actions:** Primary = Pair a device. Row = Revoke. Header = Sign out.
- **States:** Empty (unlikely — current device always present) → single row + prominent Pair CTA. Revoke pending → row spinner, optimistic removal on success. Error → inline row error with retry.
- **A11y:** `<table>` with `<th scope="col">`; Type/Session as badges not raw values; relative timestamps carry `<time datetime>` with absolute tooltip; revoke buttons have row-specific accessible names (“Revoke Living Room M5”); confirm modal traps focus.

6. History

Purpose: Browse past conversations; drill into any one’s full transcript.

< Back		History		[🔍 Search]		[Date ▼]	
Date ▼	Title	Dur.	Actions	DETAIL DRAWER			
Jul 17 09:41	Calendar check	1:12	 	Calendar check			
Jul 16 20:03	Timer + music	0:44	 	Jul17 09:41			
Jul 16 08:15	Weather brief	0:31					
Jul 15 18:22	Add shopping	2:05	 				
...				You: "..."			
				Ninja: "..."			
Showing 1–25 of 214		[< Prev]		[Next >]		[tool card]	

- **History table** — sortable `<table>`: Date (left, sortable, default sort desc), Title (identity), Duration (right-aligned, m:ss), Actions-taken badges from a known enum {Calendar, Email, Timer, Music, Shopping, Weather, ...}. Row click → detail drawer.
- **Search** — `<input type="search">` (genuinely open-ended query — case 6, allowed), debounced 300ms.
- **Date filter** — `<select>` /date-range picker from preset ranges {Today, 7 days, 30 days, Custom}. Not free text.
- **Detail drawer** — master-detail: full transcript + tool-result cards for the selected row; structured, never raw dump.
- **Actions:** Primary = open detail (row). Secondary = sort columns, filter, search, paginate, per-row Delete/Export in drawer (guarded delete).
- **States:** Empty → “No conversations yet.” Loading → skeleton rows scoped to the table. Error → distinct banner with retry. Pagination shows “Showing 1–25 of 214” and disables Prev on page 1.
- **Mobile reflow:** Detail becomes a full-screen drawer over the list (not side-by-side); columns collapse to Title + Date + badge summary, duration in row-expand.
- **A11y:** `<table>` with sortable `<th aria-sort>`; badges have text labels/ `title` (icon not alone); drawer is a labeled region, Esc closes and restores focus to the originating row; pagination buttons have accessible names and disabled states; live region announces “Showing X–Y of N” on page change.

M5Stack Tab5 (native LCD firmware + device-hosted config portal)

Surface: M5Stack Tab5 — 1280×720 landscape capacitive touch, ESP32-P4. Screens 1–9 are native LCD firmware (embedded rules: one decision per screen, big type, segmented rows / rollers / full-screen list-select, 48–64px targets, “N of M” paging, on-screen keyboard only for WiFi passphrase / device name). Screens 10–12 are rich-UI web pages the device serves over its SoftAP, opened on a phone (semantic HTML, real controls, WCAG AA, visible focus, ~44px targets, populated controls, visible labels). Palette: deep navy #0B1220 field, electric teal/cyan accent #22E3D6, listening-orb motif throughout. Every surface is gated by Login with Amazon.

1. Boot / Provisioning (native LCD)

Purpose: First-boot welcome — get the user onto the device SoftAP and into the config page.



- **QR block** — control: static QR image (encodes the SoftAP join + `http://192.168.4.1`). Data source: fixed AP SSID `LiveNinja-Setup` + device captive-portal URL baked into firmware.
- **Primary action:** `Set up Wi-Fi on device` → Screen 2 (on-LCD path). **Secondary:** `I've opened it →` (advances the “N of 3” stepper once a phone client hits the portal).
- **States:** *waiting* (default, orb slow-breathes); *client-connected* (banner “Phone connected — continue on your phone”, secondary button enabled). No error state here (pre-network).
- **A11y:** single primary decision; step indicator “1 of 3” top-right; 64px buttons; QR paired with human-readable URL text (never QR-only); high-contrast teal-on-navy $\geq 4.5:1$.

2. WiFi Setup — on-screen keyboard (native LCD)

Purpose: Join a network directly on the device — full-screen SSID list-select, then the one permitted free-text keyboard for the passphrase.

STEP A – Pick a network

Networks 1–7 of 12

← Back

Choose Wi-Fi

▶ Proffitt-5G

🔒 📶

▶ Proffitt-Guest

🔒 📶

▶ ninja-lab

🔒 📶

▶ CenturyLink9921

🔒 📶

▶ <Join hidden network...>

◀ Page 1 of 2 ▶

[🔍 Rescan]

STEP B – Passphrase for "Proffitt-5G"

Password: ●●●●●●●●

[👁 Show]

q w e r t y u i o p

a s d f g h j k l

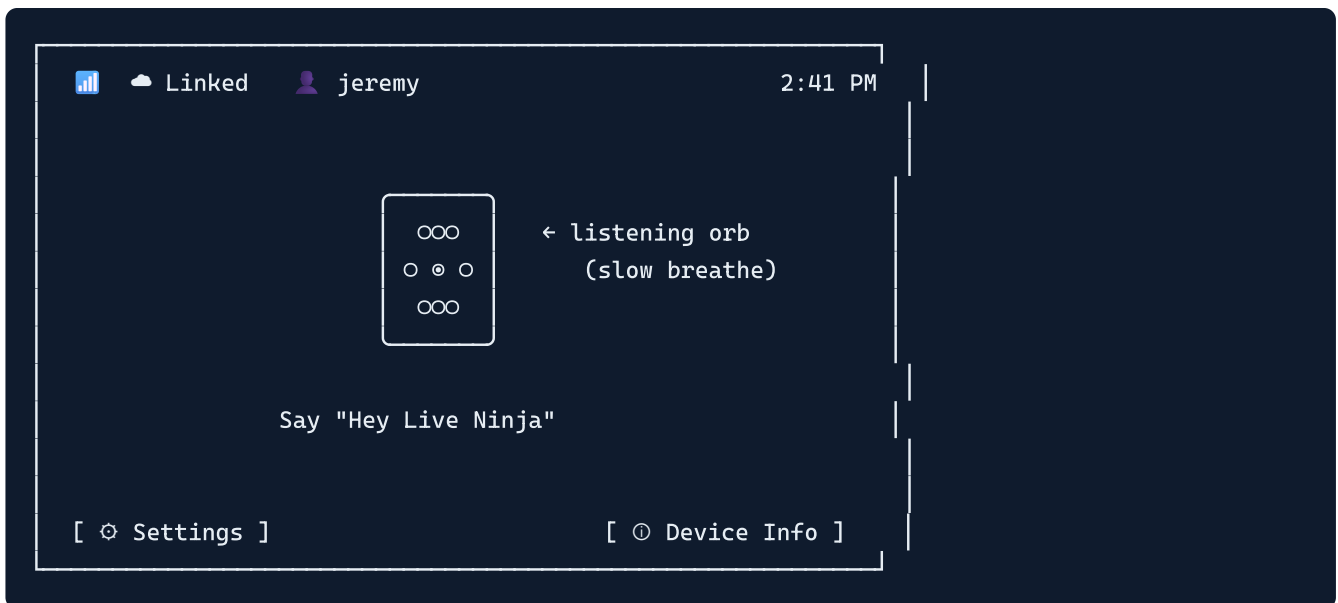
↑ z x c v b n m ☒

[123] [space] [Join →]

- **SSID list** — control: full-screen single-column list-select (NOT a dropdown). Data source: live Wi-Fi scan results; each row shows lock + signal-bar glyphs. <Join hidden network...> row routes to a name-entry keyboard.
- **Passphrase** — control: on-screen keyboard (the ONLY sanctioned free-text field on this surface). 48–64px keys; ↑ case toggle, [123] symbol layer, 👁 Show/Hide reveal toggle.
- **Primary:** Join → . **Secondary:** 🔍 Rescan , ← Back , page pager.
- **States:** *scanning* (skeleton rows + "Scanning..."); *empty* ("No networks found — Rescan"); *joining* (spinner "Connecting to Proffitt-5G..."); *auth-error* ("Wrong password — try again", passphrase preserved, field re-focused).
- **A11y:** "Networks 1–7 of 12" + "Page 1 of 2" position indicators; 64px keys; show/hide instead of blind entry; selected SSID echoed in Step B header so the target is never ambiguous.

3. Idle — wake-listening (native LCD)

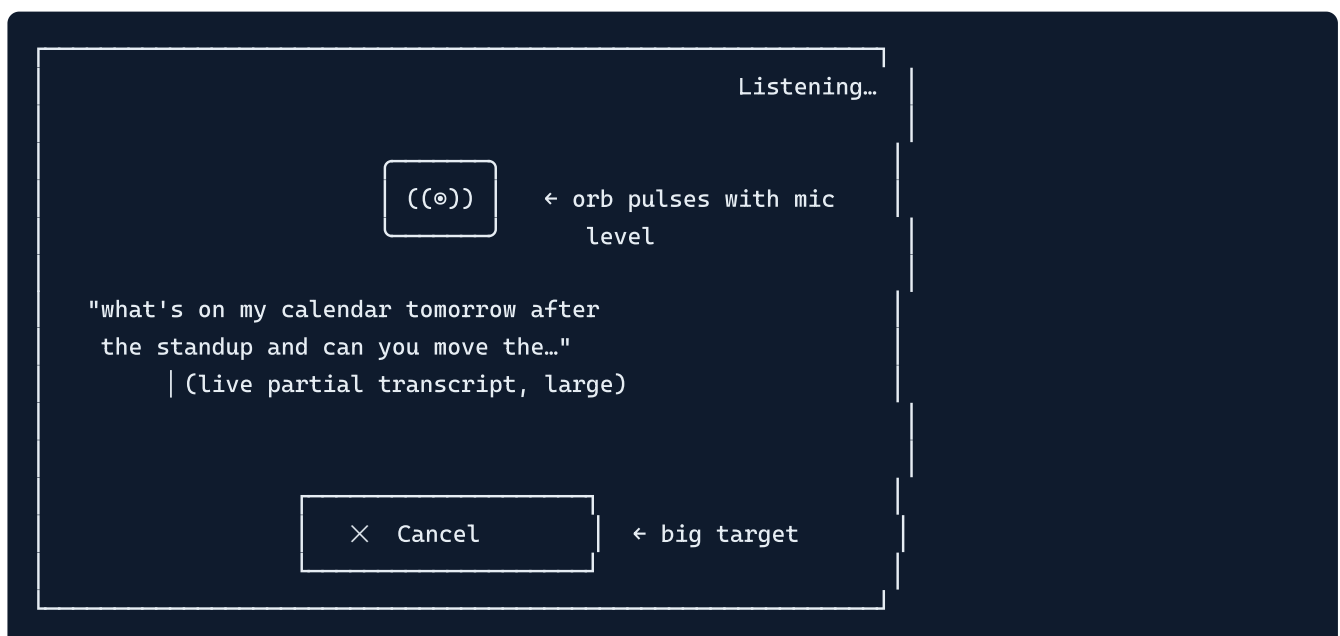
Purpose: Calm ambient home screen — device is armed and waiting for the wake phrase.



- **Primary value:** the single large listening orb (breathing animation) + wake-phrase prompt. One decision per screen.
- **Status row** — control: read-only badge strip. Data source: Wi-Fi RSSI (📶), AWS IoT link (☁ Linked), signed-in account label from on-device LWA (👤 jeremy). Clock from NTP/RTC.
- **Actions:** ⚙ Settings (→ 7), ⓘ Device Info (→ 8) — always-visible tap affordances, no hover.
- **States:** *linked* (default); *degraded* (badge turns amber, "☁ Reconnecting..."); *offline* routes to Screen 9 on wake attempt.
- **A11y:** big legible clock/prompt; badges pair icon + word (never color-only); 64px corner buttons.

4. Listening (native LCD)

Purpose: Active capture after wake — show we heard you and stream the partial transcript.

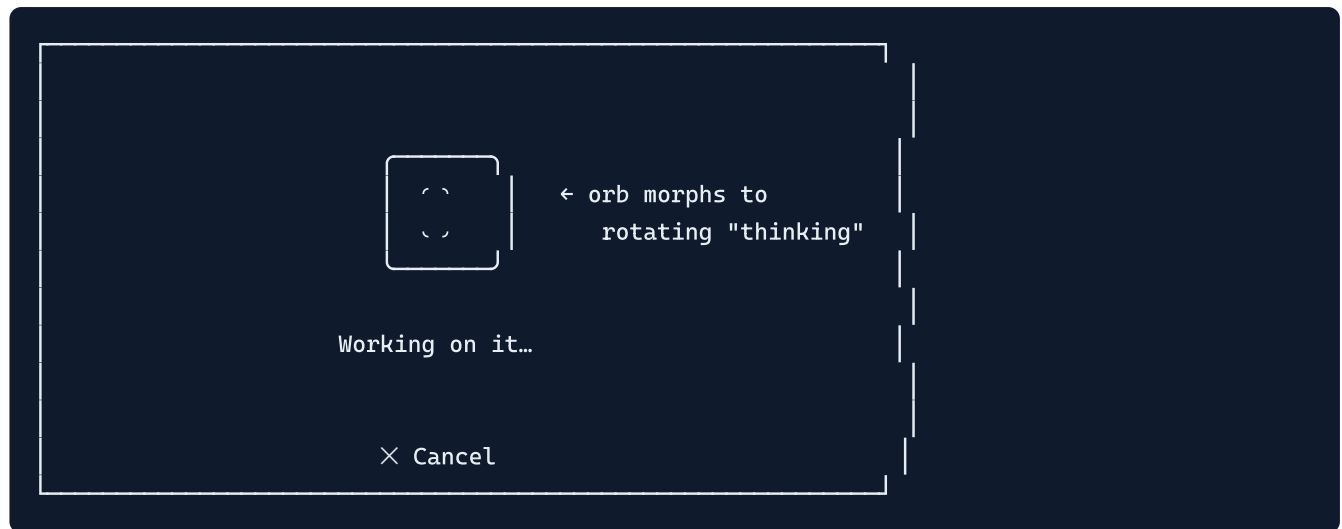


- **Primary value:** pulsing orb (amplitude-reactive) = immediate visual feedback that mic is live.

- **Transcript** — control: read-only large streaming text (interim results, caret cursor). Data source: realtime STT partials.
- **Primary action:** `× Cancel` — oversized ($\geq 64\text{px}$) to abort capture and return to Idle.
- **States:** *capturing* (default); *no-speech* (after silence timeout → “Didn’t catch that” then back to Idle).
- **A11y:** one dominant action; large transcript type; orb motion is decorative — the “Listening...” label carries the state in text.

5. Thinking (native LCD)

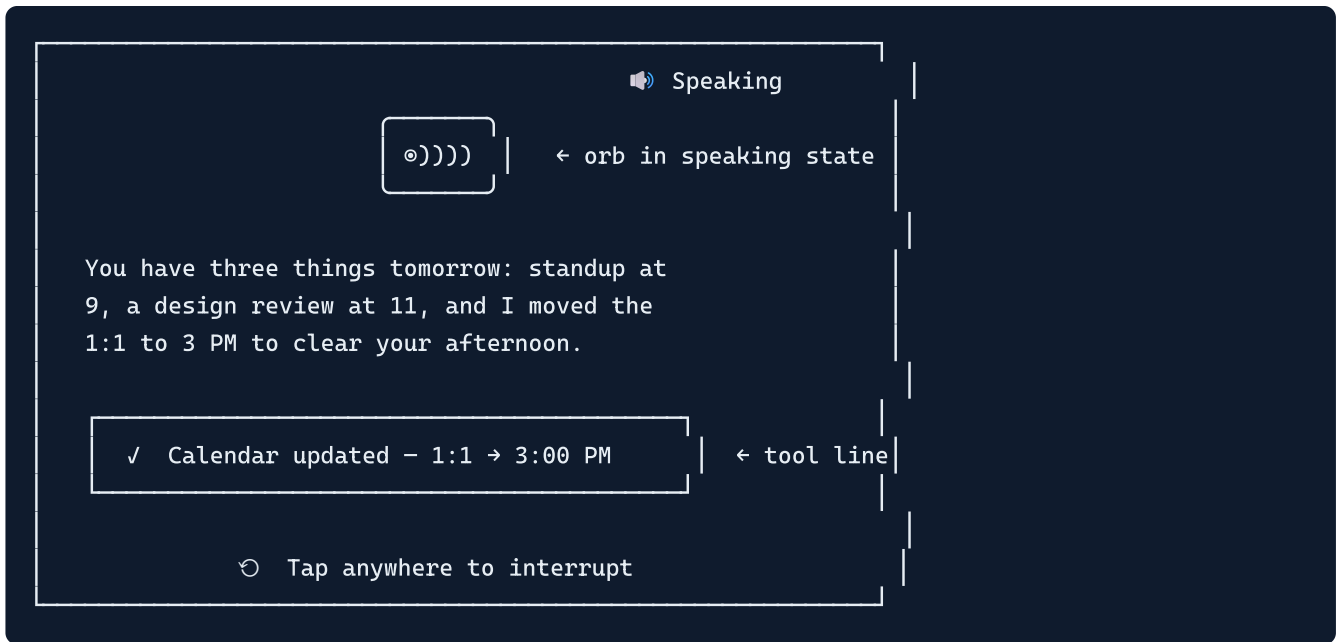
Purpose: Single unambiguous processing state between speech and response.



- **Primary value:** morphing orb + “Working on it...” — minimal chrome, one clear state.
- **Action:** `× Cancel` (secondary, still $\geq 48\text{px}$) to abandon the request.
- **States:** *thinking* (default); if it stalls $> \sim 8\text{s}$ show sub-line “Still working...”; failure transitions to Screen 9.
- **A11y:** state named in text, not color/motion alone; no competing controls.

6. Speaking + Transcript (native LCD)

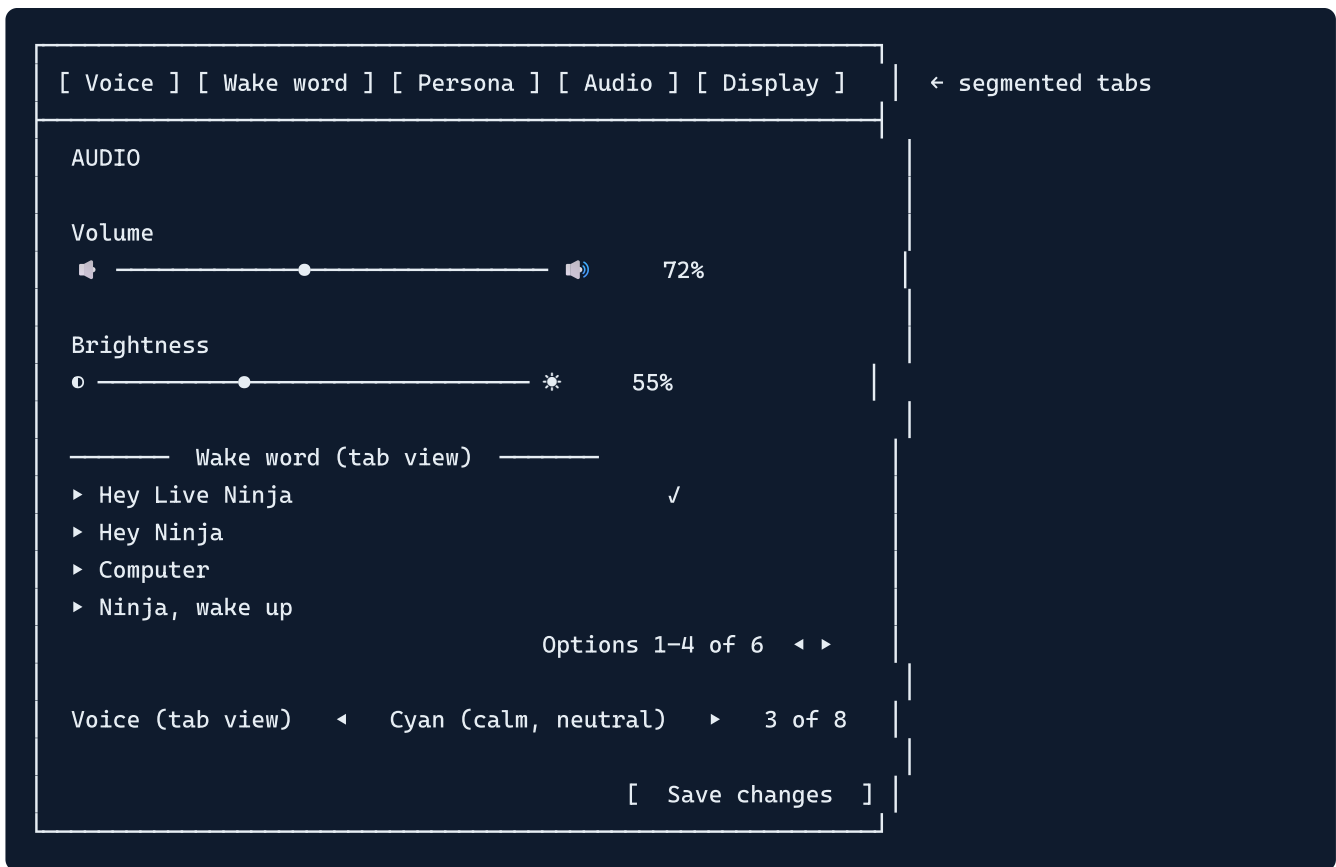
Purpose: Assistant is replying aloud — show the response text, any tool result, and allow barge-in.



- **Primary value:** speaking-state orb + large assistant response text. Data source: realtime TTS transcript stream.
- **Tool-result line** — control: read-only badge card ("✓ Email sent", "✓ Calendar updated"). Data source: tool-call outcome from the agent. Rendered as a badge, never raw JSON.
- **Barge-in:** entire screen is a tap target — "Tap anywhere to interrupt" → jumps back to Listening (Screen 4).
- **States:** *speaking* (default); *tool-pending* (badge shows spinner "Sending..." then resolves to ✓); *interrupted* (orb snaps to listening).
- **A11y:** big legible type; full-screen interrupt affordance is always visible and always available (no hover); tool status uses icon + words.

7. Settings — embedded controls (native LCD)

Purpose: Change device behavior on-device using embedded controls only — no dropdowns anywhere.



- **Tabs** — control: segmented button row across top (Voice / Wake word / Persona / Audio / Display). No dropdown.
- **Volume, Brightness** — control: large horizontal sliders with live % value and min/max glyph anchors. Data source: current device settings.
- **Wake word** — control: full-screen list-select, current selection ✓, paged “Options 1–4 of 6”. Data source: built-in wake-model catalog (fixed enum). NOT free text.
- **Voice** — control: roller / prev-next stepper with “3 of 8” position. Data source: TTS voice catalog.
- **Persona** — control: list-select (built-in personas). Data source: persona presets (novel/custom persona is a rich-UI-only affordance — see Screen 11).
- **Primary:** Save changes (persists to NVS). **Secondary:** per-tab defaults remembered.
- **States:** *loaded* (reflects stored values); *saving* (button → “Saving...”); *saved* (toast “Saved ✓”).
- **A11y:** every value has a visible label and printed numeric/label readout (no orphaned sliders); segmented tabs + “N of M” paging replace all dropdowns; 64px targets.

8. Device Info / Account (native LCD)

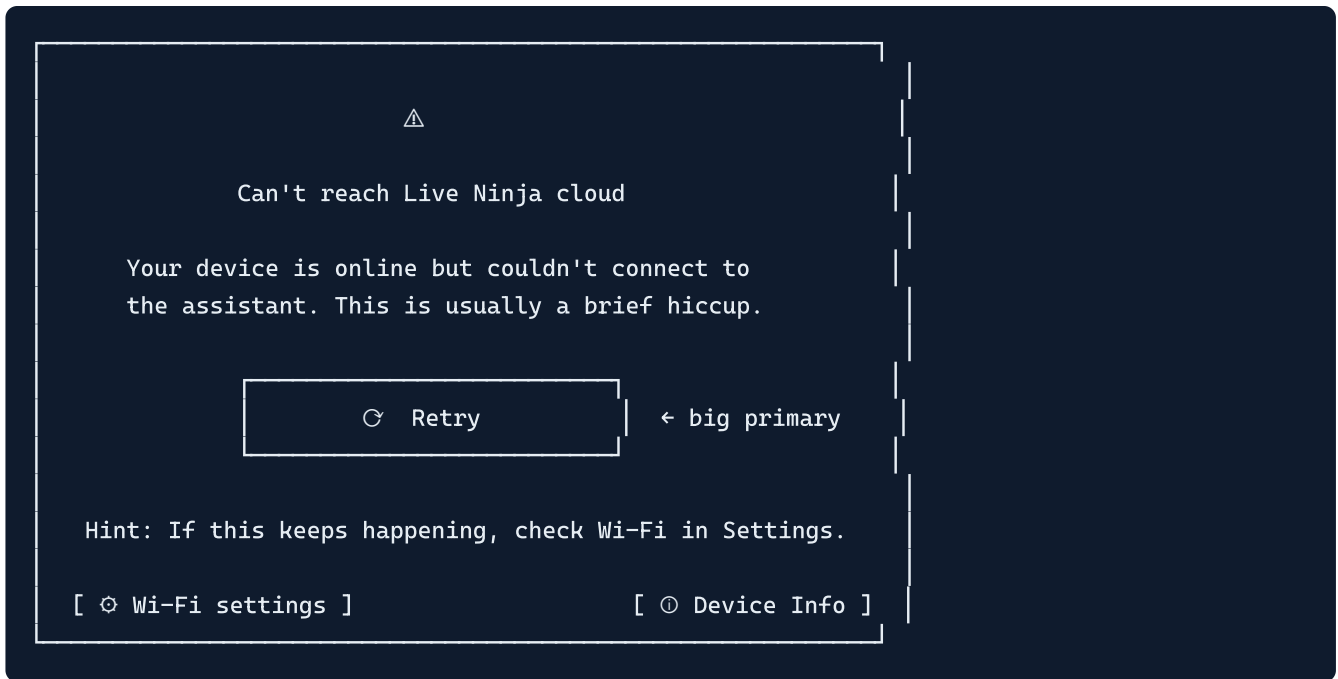
Purpose: At-a-glance device health and account, plus OTA and the guarded factory reset.



- **Account / IDs / firmware** — control: read-only labeled key/value list (definition list, not prose). Data source: on-device LWA profile, provisioned Thing name, eFuse MAC, firmware build string.
- **Health badges** — control: status badges (icon + word + detail). Data source: Wi-Fi link state/RSSI, AWS IoT MQTT connection.
- **Primary action:** ⌂ Check for updates (OTA). **Danger action:** ⚠ Factory reset... — subordinate styling, opens a confirm sheet requiring an explicit hold-to-confirm (“Hold 3s to erase & reset”) since a physical keyboard for typing “DELETE” is impractical here.
- **States:** OTA — *idle / checking... / up to date ✓ / update available → Install*. Health — green/amber/red badges.
- **A11y:** every value labeled; connection state shown by icon + text (not color alone); destructive action visually separated and guarded by a deliberate hold gesture.

9. Error / Offline (native LCD)

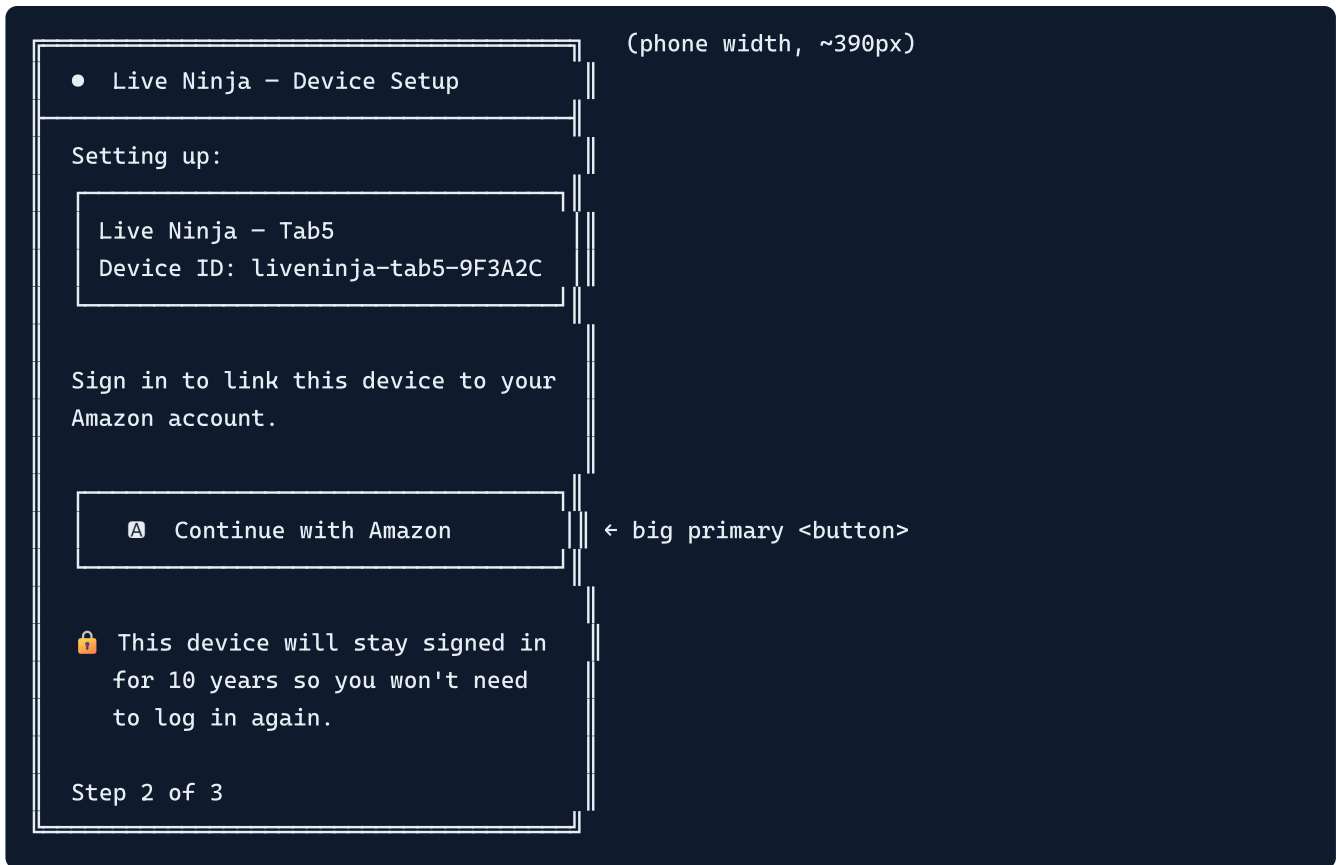
Purpose: A distinct, unmistakable failure state — plain language, one big recovery action.



- **Primary value:** warning icon + plain-language cause. Data source: connection subsystem error class (Wi-Fi down vs. cloud unreachable vs. auth expired) — message text is selected per class, not a raw error dump.
- **Primary action:** 🔄 Retry (oversized). **Secondary:** 🔄 Wi-Fi settings (→ 2), ⓘ Device Info (→ 8).
- **Variants:** *Wi-Fi down* → “Not connected to Wi-Fi”; *cloud unreachable* → shown above; *session expired* → “Please sign in again” routing to the config portal.
- **A11y:** visually distinct from all normal states (amber warning field, dedicated icon); message is specific and actionable; retry is the single dominant control.

10. Login with Amazon — device-served web page (rich UI, phone)

Purpose: The setup page the device hosts over SoftAP; opened on a phone — gate every surface behind LWA.



- **Device identity card** — control: read-only labeled card. Data source: firmware-reported device name + Thing/device ID.
- **Primary action:** `Continue with Amazon` — real semantic `<button>`, launches the LWA authorization flow (on-device token exchange). Full-width, ~48px min height, dominant.
- **Persistence note:** static helper text — “signed in for 10 years” (refresh-token longevity).
- **States:** *ready* (default); *authorizing* (button → spinner “Opening Amazon...”, disabled to block double-submit); *error* (inline alert “Couldn’t reach Amazon — Try again”, `role="alert"`).
- **A11y:** semantic `<h1>` header; device card is labeled key/value, not prose; visible focus ring on the button; WCAG AA teal-on-navy; icon-button has accessible name; page reflows at 320px.

11. Configure — device-served config form (rich UI, phone)

Purpose: One structured form (served by the device) to set wake word, voice, persona, volume, timezone — populated controls, visible labels, no blind text boxes.

- Live Ninja – Configure

linked as jeremy@...

Wake word *

Hey Live Ninja

▼ / 📄

← combobox: known set + allow novel value

Choose a preset or type your own.

Assistant voice *

Cyan – calm, neutral

▼

← <select> (8 voices)
[▶ Preview]

Persona *

Focused Assistant

▼

← select + "Custom..."

▶ (Custom) Persona instructions

Be concise, techy, and calm...

← textarea (only when "Custom" chosen)

Default volume

🔊

70%

← range slider

Timezone *

America/Denver

▼ 🔍

← searchable combobox (IANA list)

Save configuration

← primary <button>

- **Wake word** — control: autocomplete combobox (known presets surfaced, novel value allowed per open-ended-but-suggestible rule). Data source: built-in wake-model catalog; default = "Hey Live Ninja".
- **Assistant voice** — control: `<select>` (8 options, in the 6–20 range) with `▶ Preview`. Data source: TTS voice catalog.
- **Persona** — control: `<select>` with a `Custom...` option that progressively discloses a `<textarea>` for instructions. Data source: persona presets; custom = the sole free-text field, justified (genuinely open natural language).
- **Default volume** — control: `<input type="range">` min 0 / max 100 / step 5 with live % readout.
- **Timezone** — control: searchable combobox (typeahead, LIMIT-ed results). Data source: IANA tz database — never free text; default pre-filled from device locale/geo-IP guess.
- **Primary:** `Save configuration`. **Persistence:** account already known from Screen 10 — not re-asked.

- **States:** *loaded* (defaults pre-selected); *validating* (inline, on blur); *saving* (button pending + disabled); *error* (field-adjacent messages + summary banner, all valid input preserved).
- **A11y:** every field has a persistent `<label for>` (* marks required, legended once); helper text via `aria-describedby`; errors `role="alert"`; logical tab order; visible focus; native controls so mobile keyboards/pickers work; reflows at 320px.

12. Success — device-served confirmation (rich UI, phone)

Purpose: Confirm setup completed, echo the chosen settings, and hand off to voice use — preserving entered values.



- **Settings summary** — control: read-only labeled key/value list (definition list, structured — not a prose dump). Data source: the values just saved in Screen 11 + the linked account from Screen 10.
- **Primary action:** `Done` (releases the phone; device drops to Idle / Screen 3). **Secondary:** `← Edit settings` returns to Screen 11 with all values preserved (never cleared).
- **Persistence line:** “Stays signed in — 10 years” reaffirms the long-lived session.
- **States:** *success* (default); if a value failed to persist, an inline warning on that row with a `Retry` affordance rather than a silent success.
- **A11y:** success announced (`role="status"`); summary is labeled rows (no orphaned values); explicit confirmation of completion; primary action dominant, edit path subordinate; reflows at 320px; step “3 of 3” closes the provisioning stepper started on Screen 1.

UI Prototypes — Full-Screen Clickable Mockups

Every function has a self-contained, offline HTML prototype under `mockups/`, all sharing one design system (deep-navy + electric-teal dark theme, listening-orb motif). Open `mockups/index.html` for the gallery. Screens are wrapped in device frames (phone bezel, browser chrome, or the 1280×720 M5Stack LCD).

Android app (mobile · primary assistant)

- Onboarding
- Login Lwa
- Set Default Assistant
- Permissions
- Home Idle
- Conversation
- Wakeword Manager
- Wakeword Create
- Settings
- Account Sessions
- History

Website (Go-Fiber served)

- Landing Login
- Conversation
- Wakeword Settings
- Settings
- Account Devices
- History

M5Stack Tab5 — native LCD firmware

- Boot Provisioning
- Idle Listening
- Listening
- Thinking
- Speaking
- Settings
- Device Info
- Error Offline

M5Stack Tab5 — device-hosted config portal

- Portal Login
- Portal Config
- Portal Success

Implementation Plan

Live Ninja — Implementation Plan

Status: authoring / not yet started · **Owner:** jeremy · **Repo:** JeremyProffittOrg/live-ninja **Domain:** live.jeremy.ninja · **AWS account:** 759775734231 · **Region:** us-east-1 **Stack:** AWS SAM · Go 1.22 on provided.al2023 · **arm64/Graviton** · Go-Fiber via Lambda Web Adapter

Live Ninja is one AWS backend serving three LWA-gated client surfaces — a responsive **Web** app, an **Android** primary-assistant app, and an **M5Stack Tab5** embedded terminal — around OpenAI’s GPT Realtime speech-to-speech engine. This document is the formal, self-updating implementation plan.

1. Overview & how to read this plan

This plan is organized into **parallel workstreams** executed by **agentic teams of subagents**, sequenced across **milestones M0–M8**. Each milestone has a **Definition of Done** and an ordered task list. Every milestone and task carries a **status marker** and a **model-routing** annotation. Tasks cross-reference **FR IDs** from the PRD where natural.

1.1 Status markers (updated in place as work proceeds)

Marker	Meaning
[]	todo — not started
[~]	in progress
[x]	done
[!]	blocked (note the blocker inline)

All milestones and tasks below start at [] .

1.2 Model-routing legend

Route each task to the **cheapest capable model** (per the machine-global Sub-Agents & Model Routing policy). If **Fable** is unavailable, promote to **Opus** — never drop to Sonnet.

Tag	Model	Use for
H	Haiku	light / mechanical: scaffolding, boilerplate, config edits, doc stubs, wiring, renames
S	Sonnet	general engineering: handlers, CRUD, UI screens, tests, IaC of moderate complexity
F	Fable	hard reasoning: tricky protocol/state work, wake-word/audio pipelines, concurrency, sync/conflict logic
O	Opus	hardest reasoning / architecture: security-critical auth flows, cross-surface contracts, cost/consistency invariants, threat modeling

1.3 Execution rules (from house style)

- **Built for autonomy.** Every decision, default, and fallback is baked in below; execution runs straight through milestones without check-ins. The only reason to pause is a genuine, un-pre-decided blocker.
- **Verbose implementation notes.** As work proceeds, append detailed notes inline under each task/milestone (decisions, files touched, commands, gotchas) — verbose enough for a fresh agent to resume from the plan alone. See §8.
- **Production-only shop.** No staging. `sam deploy` change-set previewed in logs; local Fiber smoke-test (LWA parity) pre-push; alarms catch regressions fast.
- **Deploy = push to `main`.** No local AWS deploys, no static keys, OIDC only. Secrets set by the user via `scripts/set-secret.sh` / manual `aws ssm put-parameter`; agents never see secret values.

2. Workstream map

Six workstreams run in parallel wherever dependencies allow. **WS-A (Platform/Infra)** is the critical path that unblocks everything; **WS-B (Auth)** and **WS-C (Realtime)** unblock the three client workstreams (**WS-D/E/F**); **WS-G (Platform Cross-Cut)** threads through all of them.

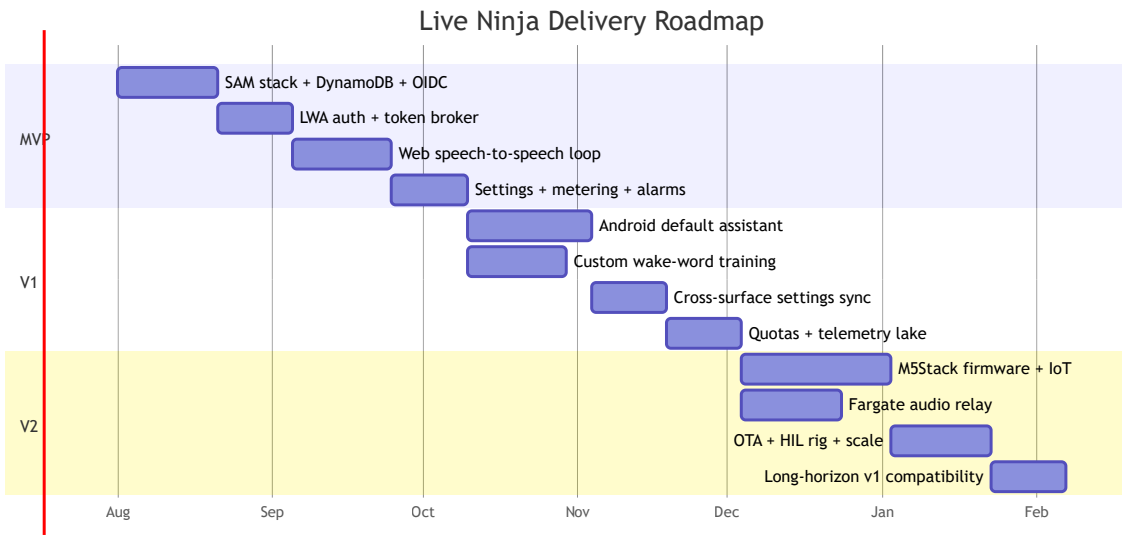
WS	Name	Owns	Primary models	Depends on	Runs parallel with
WS-A	Platform & Infra	SAM stack, OIDC pipeline, DynamoDB single-table, SSM, S3, CloudFront/R53, tagging, IoT Core, Fargate relay infra	O (arch), S (IaC), H (config)	—	all (foundation)
WS-B	Identity & Auth	LWA OAuth (BFF), first-party JWT (KMS ES256) + rotating refresh, device 10-yr flow, authorizer	O (flows/threat), S (handlers)	WS-A (M0)	WS-C
WS-C	Realtime Voice	Token broker, session config, tool router, Fargate audio relay, fallback cascade, metering gate	O (broker/relay), F (audio), S (tools)	WS-A, WS-B	WS-B
WS-D	Web Client	Fiber SSR UI, WebRTC-to-OpenAI, transcript/visualizer, settings, PWA/SW, WASM wake word	S (UI), F (realtime.mjs)	WS-B, WS-C	WS-E, WS-F
WS-E	Android Client	VoiceInteractionService, ROLE_ASSISTANT flow, wake-word FGS, WebRTC, Custom Tabs LWA	F (assistant/wake), S (UI)	WS-B, WS-C	WS-D, WS-F
WS-F	M5Stack Firmware	ESP-IDF firmware, ESP-SR wake, IoT MQTT audio, device-hosted config/LWA, 10-yr cert, OTA	O (P4/C6 arch), F (audio/wake), S (LVGL UI)	WS-A, WS-B, WS-C relay	WS-D, WS-E
WS-G	Cross-Cut Platform	Settings schema/sync, wake-word training+distribution, privacy/retention, observability, cost/quota, versioning/compat, testing	O (contracts/quota), F (sync), S (tests)	WS-A	all

2.1 CI/CD pipeline (WS-A)



diagram

2.2 Delivery roadmap (milestone gantt)



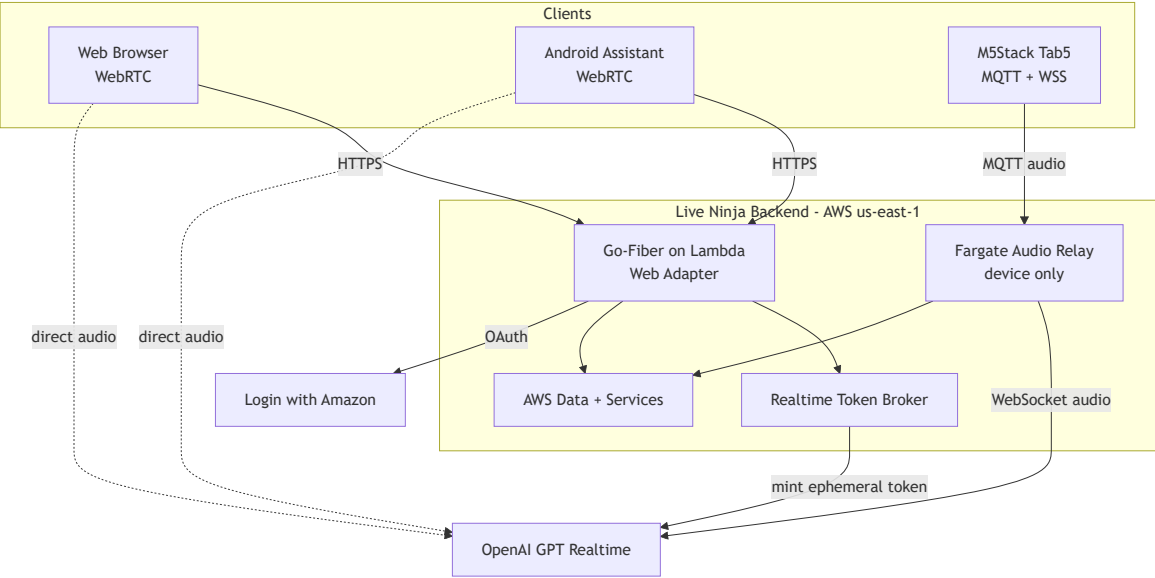
diagram

2.3 Milestone → workstream matrix

Milestone	WS-A	WS-B	WS-C	WS-D	WS-E	WS-F	WS-G
M0 Bootstrap/Infra	●						○
M1 Auth	○	●					
M2 Realtime backend		○	●				○
M3 Web client			○	●			○
M4 Android client		○	○		●		○
M5 M5Stack firmware+IoT	○	○	○			●	○
M6 Programmable wake + sync				○	○	○	●
M7 Hardening/observability/cost/privacy	○	○	○	○	○	○	●
M8 Launch	●	○	○	○	○	○	●

● = lead workstream · ○ = contributing

3. Architecture snapshot



diagram

Locked decisions (see research briefs): Lambda Web Adapter for the Fiber app (not proxy shims); **single-table DynamoDB** (`Query` / `GetItem` only, never `Scan` on a serving path); **direct client→OpenAI WebRTC/WS for audio** (browser/Android) with a **Fargate relay only for the M5Stack**; **SSM Parameter Store SecureString** for secrets (no Secrets Manager); **KMS ES256** JWT signing (private key non-extractable); GitHub Actions + OIDC as the only deploy path; six mandatory cost tags at stack level.

4. Milestones

Each task line: [status] · **model** · task · (FR refs / note). FR IDs reference the PRD's functional requirements; where the PRD numbering is not yet fixed, the bracketed area tag (e.g. [AUTH] , [RT] , [WAKE]) stands in and is reconciled during M0's contract pass.

M0 — Bootstrap / Infrastructure [] (WS-A lead, WS-G support)

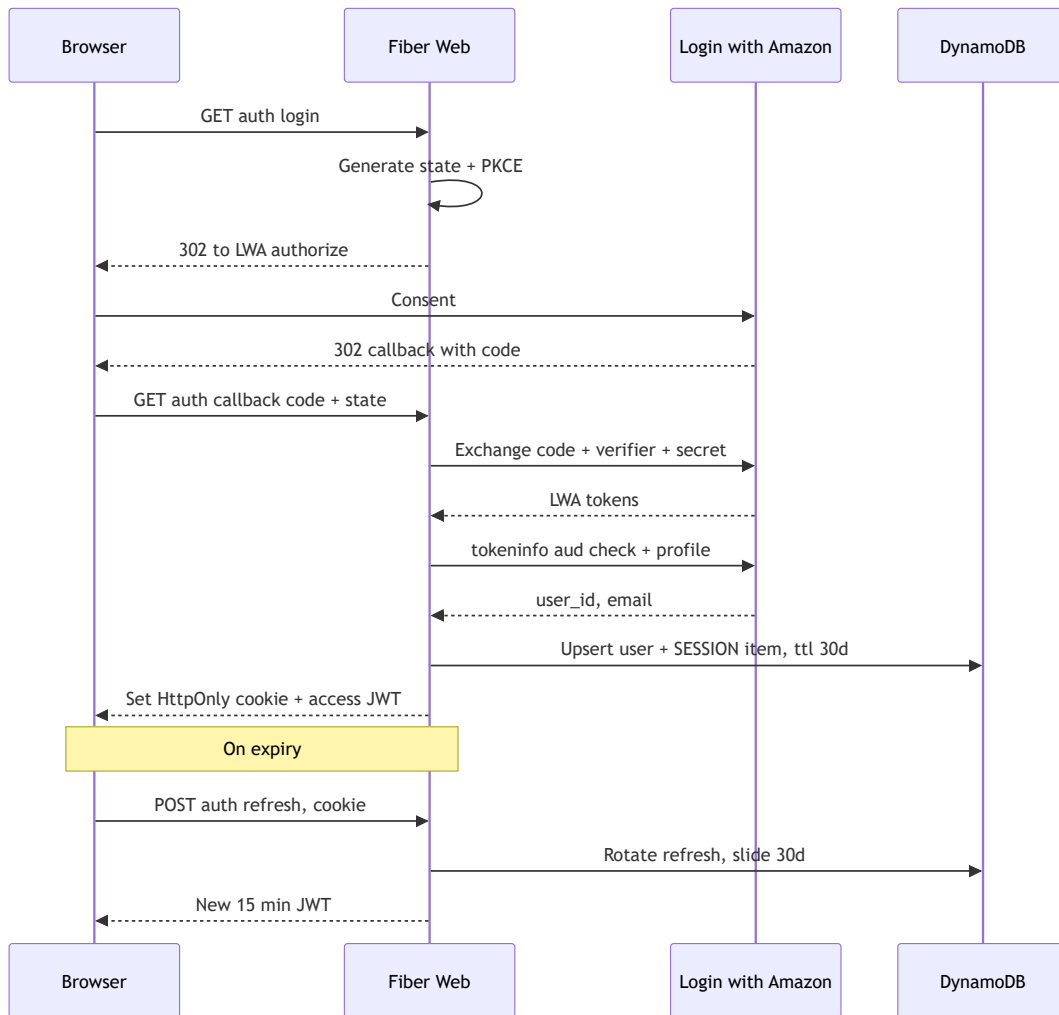
Definition of Done: An empty-but-real SAM stack deploys to 759775734231 via GitHub Actions + OIDC on push to main ; the single-table DynamoDB (live-ninja) with both GSIs + TTL exists; all SSM parameter slots exist (values set out-of-band by the user); S3 buckets, CloudFront + Route 53 for live.jeremy.ninja , and the six cost tags are in place; /healthz returns 200 through CloudFront; the DynamoDB ConsumedReadCapacityUnits alarm and AWS Budgets (\$20/\$50/\$100 on Project=live-ninja) are armed. **Cost Allocation Tags Project + CostCenter activated in Billing (non-retroactive — do early).**

Ordered tasks: - [] **O** — Author template.yaml skeleton: HTTP API v2 + web Fiber Lambda (arm64, LWA layer), per-function least-privilege roles, authorizer , realtime-broker , iot-ingest , usage-rollup , email-dispatch function stubs wired but minimal. (\$2.1 backend brief) - [] **S** — DynamoDB live-ninja

table: pk / sk, **GSI1** (gsi1pk / gsi1sk), **GSI2** (gsi2pk / gsi2sk), TTL on ttl, PAY_PER_REQUEST, PITR on. (DynamoDB Data Model diagram) - [] **H** — samconfig.toml with stack tags Project=live-ninja CostCenter=voice-ai Environment=prod ManagedBy=sam DeployedVia=github-actions Owner=jeremy; arm64 defaults; artifact bucket vars.CLOUDFORMATION_S3_BUCKET. (§12) - [] **H** — Makefile build targets: GOOS=linux GOARCH=arm64 CGO_ENABLED=0 go build -tags lambda.norpc -o bootstrap ./cmd/<fn> per function. - [] **S** — .github/workflows/deploy.yml per deploy.md: OIDC role-to-assume: vars.AWS_DEPLOY_ROLE_ARN, id-token: write, make build → sam build → sam deploy --no-confirm-changeset --no-fail-on-empty-changeset. - [] **H** — SSM SecureString/String parameter slots: /live-ninja/prod/openai/api_key, /lwa/client_id, /lwa/client_secret, /session/jwt_signing_key (or KMS key alias), /device/cred_pepper. Values set out-of-band by user. (§8) - [] **O** — KMS CMKs: alias/live-ninja-auth (envelope-encrypt LWA refresh tokens) + ECC_NIST_P256 SIGN_VERIFY CMK for first-party JWT signing (private key never leaves KMS). (Auth brief §2.3) - [] **S** — S3 buckets (block-public, SSE-S3, versioning on assets): live-ninja-user-<acct>, -wakewords-<acct>, -assets-<acct>, -logs-<acct>, -analytics-<acct>. (§5 backend) - [] **S** — Edge: CloudFront distro for live.jeremy.ninja (ACM vars.CERTIFICATE_ARN), Route 53 alias (vars.HOSTED_ZONE_ID), cache /static/* immutable, pass /api/auth no-cache; security headers (HSTS/CSP/X-CTO/Referrer). - [] **H** — Fiber /healthz + minimal internal/config SSM loader (cached, 5-min TTL) and internal/observ slog JSON logger. - [] **S** — IoT Core baseline: Thing Type liveninja-tab5, Thing Group fleet, provisioning template + claim-cert policy (empty policies scoped for M5). (§10 backend, §5 M5 brief) - [] **S** — Alarms/Budgets: DynamoDB ConsumedReadCapacityUnits / WriteCapacityUnits / ThrottledRequests; AWS Budgets \$20/\$50/\$100 on tag; EstimatedCharges backstop; SNS→SES notify. (§13) - [] **H** — **User action gate**: document the one-time manual steps (SSM put-parameter values; activate Cost Allocation Tags). (house rule: agents never see secret values) - [] **O** — **Contract freeze (WS-G)**: publish the six integration-seam contracts (settings JSON-schema+ version, shadow doc, wake-word manifest GET /wakeword/<id>/model?platform=, telemetry event schema, X-LN-Client / X-LN-Server headers, metering/quota gate) into /contracts and reconcile FR IDs. (Crosscut §closing)

M1 — Auth (LWA BFF + first-party sessions) [] (WS-B lead)

Definition of Done: All three surfaces can complete Login with Amazon through the backend BFF; the backend mints a first-party **ES256 access JWT (15 min, KMS-signed) + opaque rotating refresh token (hash-only in Dynamo, reuse-detected)**; web gets a __Host- HttpOnly cookie (30-day sliding), Android gets JWT+refresh (30-day sliding), M5Stack gets the device 10-year credential lineage; the Lambda authorizer validates JWTs against JWKS with a tokensValidAfter kill-switch; logout + “log out everywhere” + device revoke all work; a new-sign-in SES alert fires. (FR [AUTH])



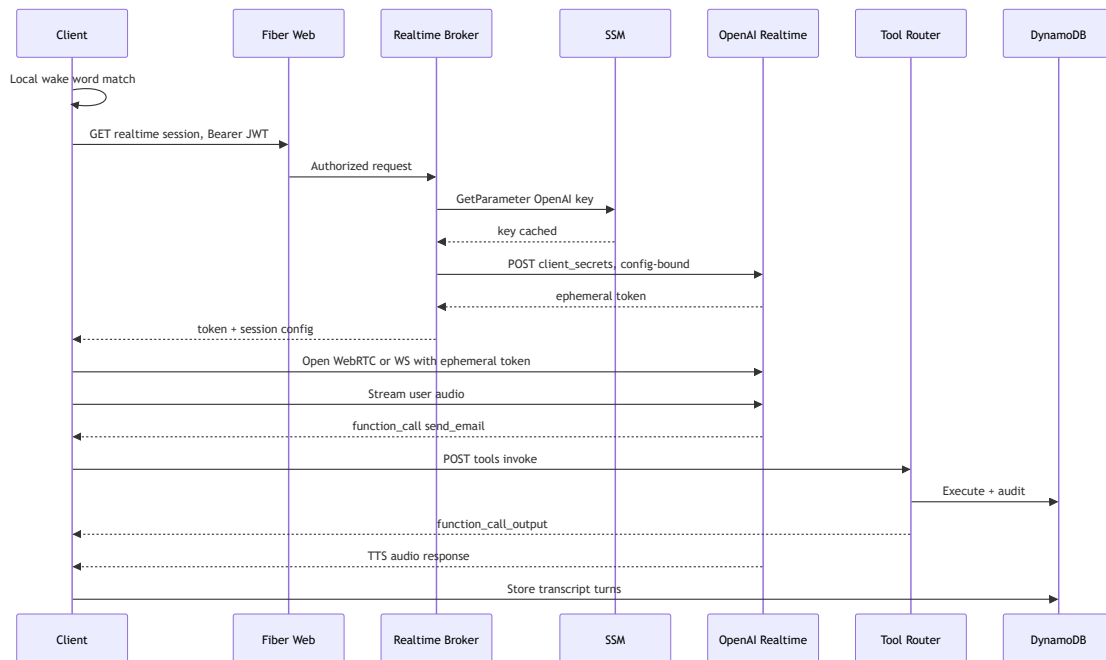
diagram

Ordered tasks: - [] **O** — `internal/auth/lwa.go`: authorize URL + PKCE, code exchange at `api.amazon.com/auth/o2/token`, **two-check validation** (`/tokeninfo` `aud == client_id` + `/user/profile`), `user_id` as canonical subject. (Auth §2.2) - [] **O** — `internal/auth/session.go`: ES256 JWT via `kms:Sign`, JWKS at `/.well-known/jwks.json` (from `kms:GetPublicKey`, 24h cache), claims `iss/sub/aud/sid/did/iat/exp/jti/scope`. (Auth §2.3) - [] **O** — Rotating refresh token: 256-bit random, SHA-256 hash stored, **TransactWriteItems rotate-on-use with reuse detection** → **family revoke + SES alert**. (Auth §2.4) - [] **S** — `store/users.go` + `store/sessions.go`: User upsert via GSI1 `LWA#<amazonUserId>`; SESSION item (`refreshHash`, `surface`, `familyId`, `ttl`); GSI1 `SESS#<sessionId>` lookup; GSI2 active-session feed. (DynamoDB model) - [] **O** — `authorizer` Lambda: verify JWT signature+ `exp` + `surface` against JWKS, reject `iat < user.tokensValidAfter` (60s cached), inject `userId/deviceId/surface` context; public-route bypass. (§9.3) - [] **S** — Web cookie flow: `/auth/lwa/login` + `/auth/lwa/callback`, `__Host-ln_rt` `Secure;HttpOnly;SameSite=Lax`, access JWT in JSON, OAuth state→verifier in Dynamo (TTL 10 min), CSRF double-submit. (Web §4) - [] **S** — Android exchange: `POST /api/v1/auth/lwa/exchange {code, code_verifier}` → JWT + 30-day refresh. (Android §6) - [] **O** — Device 10-yr flow (`internal/auth/device.go`): `PAIR#<nonce>` register, `/auth/device/callback` browser leg, `code_verifier` device-claim binding, `devices/pair` bind + IoT Thing/cert provision, 10-yr refresh family, silent 24h rotation. (Auth §6, M5 §6) - [] **S** — Revocation surface: `/auth/logout`, “log out everywhere” (`tokensValidAfter=now`), `DELETE /devices/{id}` (revoke family +

detach IoT cert). (Auth §7) - [] S — email-dispatch + SES templates for new-device-login / security-alert ; enqueue via SQS off request path; IDEMP# idempotency. (§6 backend) - [] F — Auth tests: PKCE, aud substitution rejection, refresh reuse-detection, tokensValidAfter kill within 60s, cookie flags, device claim binding. dynamodb-local + mocked LWA. (Crosscut §7)

M2 — Realtime voice backend (broker + tool-calling) [] (WS-C lead)

Definition of Done: An authenticated client can GET /api/v1/realtime/session and receive a **config-bound OpenAI ephemeral token** (~60s) plus the resolved persona/tool manifest; the OpenAI key lives only in SSM read by the broker's isolated role; server-side **tool router** (POST /api/v1/tools/invoke) executes tools re-authorized per call with idempotency; the **metering/quota gate** rejects over-cap mints pre-spend; the **fallback cascade** (retry → chained STT→LLM→TTS → text-only) is implemented; the **Fargate audio relay** service exists (for M5) even though no device connects yet. (FR [RT])



diagram

Ordered tasks: - [] O — realtime-broker Lambda (isolated IAM: ssm:GetParameter one ARN + kms:Decrypt): load persona/tools, mint POST /v1/realtime/client_secrets config-bound (model gpt-realtime , voice , instructions , semantic_vad interrupt_response , tools , pcm16). (§7 backend, Voice §2/§6) - [] O — Quota/metering gate at mint: read USAGE#<month> + daily counter, per-user token-bucket (1 mint/5s burst 3), soft-cap warn / hard-cap 402/429; write session ledger stub. (Crosscut §6, Voice §13) - [] O — internal/realtime session config + persona resolution (clients send persona ID, server resolves instructions — anti-injection). (Web §2.1) - [] S — Tool router (fn-tool-router / /api/v1/tools/invoke): re-authorize LWA user per call, enumerated-arg schemas, idempotency keys; tool catalog send_email (SES jeremy@jeremy.ninja / Reply-To gmail, confirm-before-send external), set_timer/reminder (EventBridge Scheduler), device_control (owner-scoped MQTT), get_weather/web_lookup , remember/recall_note . (Voice §7) - [] F — Fargate audio relay (Go arm64, ALB+WSS): validate device JWT, budget-gate, hold OpenAI WS, Opus↔PCM16@24k transcode, barge-in (response.cancel + input_audio_buffer.clear), tool-call intercept server-side, transcript/usage sink. ECS service+autoscaling in SAM. (Voice §3.3/§3.4, M5 §4) - [] F — Fallback cascade fn-fallback-turn : 2×

backoff retry → STT (gpt-4o-transcribe) → gpt-4o-mini → TTS (gpt-4o-mini-tts) → text-only → graceful hard-down (side-effects queued). (Voice §12) - [] S — Transcript sink + usage rollup: USER#<uid>/LOG#... turns (TTL 90d), usage-rollup hourly EventBridge → daily/monthly rollups (Query only). (§4, §13 backend) - [] S — EMF metrics: SessionsBrokered , EphemeralTokenMintLatency , ToolInvocations , per-surface counts; X-Ray on broker/authorizer/web. (§13) - [] F — Broker/relay/tool tests: mocked OpenAI WS + REST, quota gate pre-spend enforcement, tool re-authz, confirm-before-send, relay Opus transcode + barge-in cancel. (Crosscut §7)

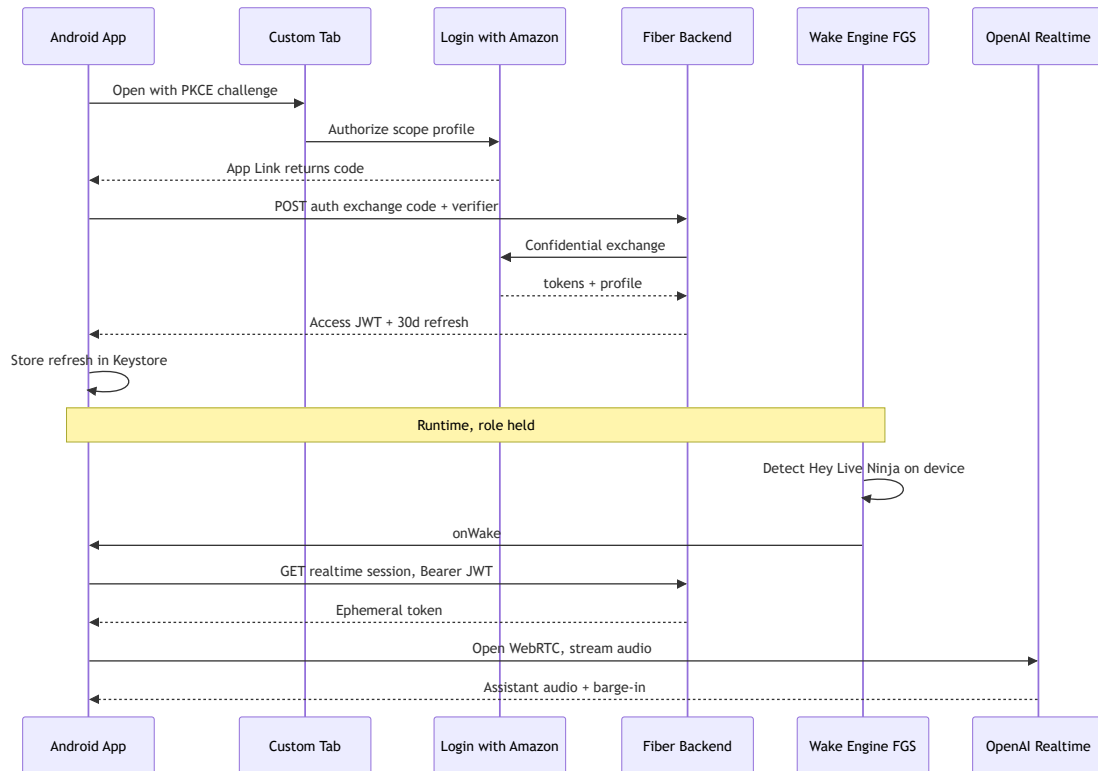
M3 — Web client [] (WS-D lead)

Definition of Done: live.jeremy.ninja serves the Fiber SSR app; unauthenticated users get server-rendered login; authed users get a conversation view with **direct WebRTC to OpenAI**, live transcript, visualizer, barge-in, and click-to-talk; a schema-driven settings page (populated controls, WCAG AA both themes); PWA with network-first HTML service worker; CSP allows only OpenAI + self. Optional WASM wake word (off by default) with guaranteed click-to-talk fallback. (FR [WEB])

Ordered tasks: - [] H — Template structure (layouts/base , partials/nav + audio_viz , pages/landing|conversation|settings|error); fingerprinted static asset generator; no-cache HTML / immutable assets. (Web §1) - [] F — realtime.mjs : RTCPeerConnection , mic track (AEC/NS/AGC), oai-events datachannel, SDP offer → /v1/realtime/calls with ephemeral secret, remote audio element. (Web §2.2) - [] F — Barge-in: on input_audio_buffer.speech_started stop/attenuate assistant audio + response.cancel , flip mic state. (Web §2.3) - [] S — Mic state machine + large primary control (idle→requesting-mic→connecting→live-listening↔live-speaking→ending , error/denied); push-to-talk + hands-free. (Web §2.5) - [] S — transcript.mjs (text-node incremental render, role="log") + visualizer.mjs (AnalyserNode → canvas, aria-hidden , prefers-reduced-motion). (Web §6) - [] S — Settings page (schema-driven, **mandatory agentic design pass first**): wake-word combobox, engine radio, sensitivity slider, persona select, voice radio+preview, turn-detection radio, theme segmented, mic-device select, sign-out/all-devices. (Web §5.3, machine UI rules) - [] S — Tool client dispatch: client-safe tools in-browser; backend tools → /api/tools/:name → function_call_output . (Web §2.4) - [] F — wakeword.mjs : openWakeWord WASM in AudioWorklet (default), Porcupine-web alt behind same interface, lazy-loaded, unsupported → click-to-talk. (Web §3) - [] S — PWA: manifest.webmanifest + sw.js (network-first HTML, SWR assets, never cache /api|/auth /OpenAI, versioned cache purge on activate, skipWaiting / clients.claim). (Web §8) - [] S — Playwright e2e (stubbed LWA + mock OpenAI WS): login, settings CRUD, wake-swap, session bootstrap; Lighthouse + axe WCAG AA. (Crosscut §7)

M4 — Android client (assistant role + wake word) [] (WS-E lead)

Definition of Done: The app installs (sideload/internal-testing), completes LWA via Custom Tabs+PKCE (30-day sliding session in Keystore), runs its **own programmable wake-word engine** (Porcupine primary / openWakeWord fallback) in a microphone FGS with a persistent notification, acquires ROLE_ASSISTANT via the resilient OEM-aware guided flow (and works even without it), and on wake opens a **WebRTC** GPT-Realtime session with AEC-backed barge-in; locked-screen sessions gate sensitive actions behind biometric. (FR [AND])



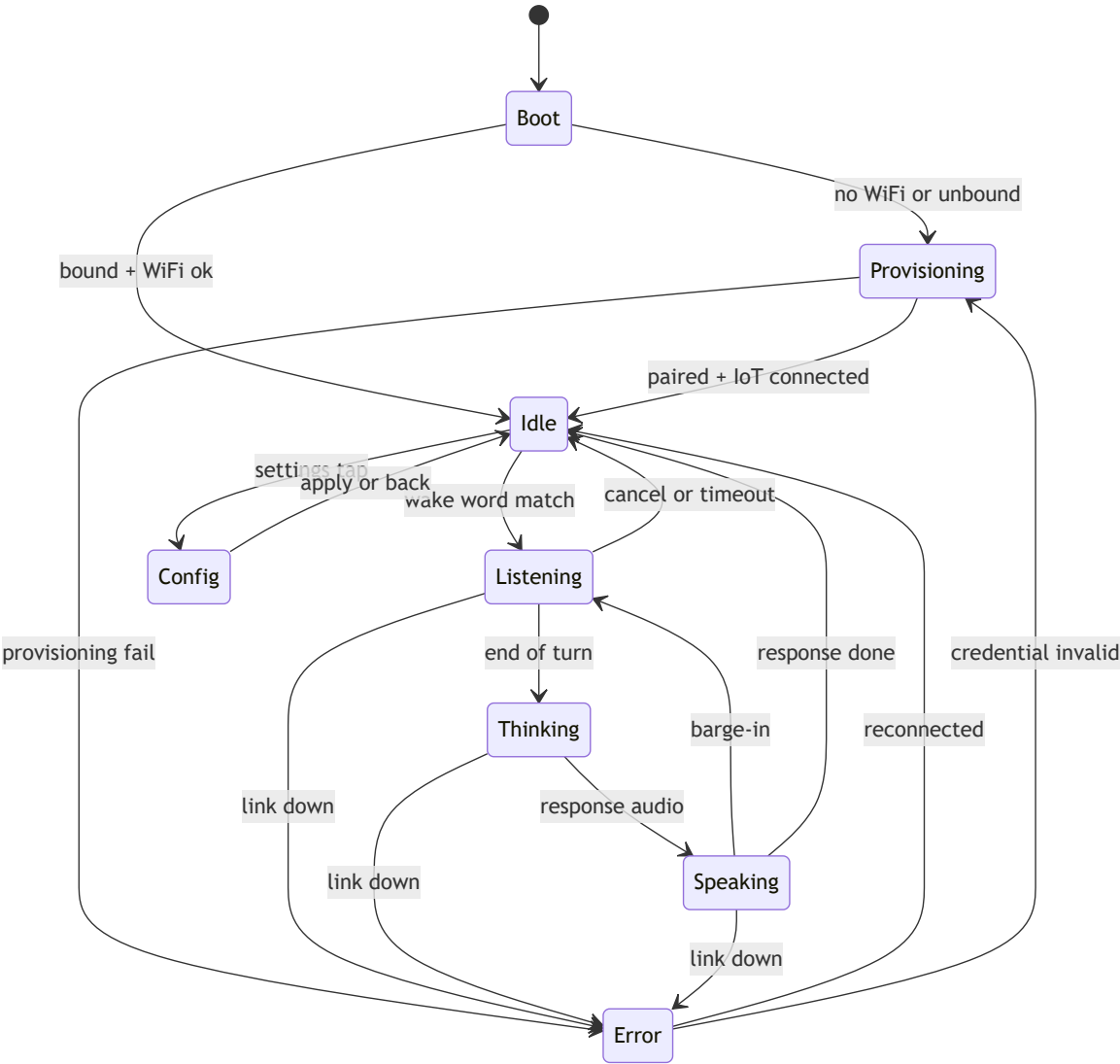
diagram

Ordered tasks: - [] **H** — Gradle module layout (:app , :core-audio , :core-realtime , :core-auth , :feature-*, :service-assistant), minSdk 29/targetSdk 35, Hilt, Compose M3. (Android §1) - [] **S** — LWA Custom Tabs + PKCE, /api/v1/auth/lwa/exchange , refresh in EncryptedSharedPreferences/Keystore, silent sliding refresh on foreground. (Android §6) - [] **F** — WakeWordEngine interface; Porcupine primary (.ppn from backend/S3) + openWakeWord fallback; AudioRecord 16kHz + VAD pre-gate. (Android §3.1) - [] **F** — WakeWordService (foregroundServiceType=microphone , persistent low-priority notification, BOOT_COMPLETED restart); battery strategy (VAD gate, no continuous wakelock, thermal/battery-saver duty-cycle, <2%/hr target). (Android §3.2/§3.3) - [] **O** — VoiceInteractionService / SessionService / Session + RecognitionService ; manifest contract; RoleManager request + **OEM-aware settings deep-link fallback walkthrough** + isRoleHeld polling; locked-session gating via requestDismissKeyguard() . (Android §2) - [] **F** — WebRTC capture chain (google-webrtc vendored .aar behind RealtimeTransport), MODE_IN_COMMUNICATION , platform+WebRTC AEC/NS/AGC, ephemeral token from backend. (Android §4) - [] **F** — Barge-in + playback (server-VAD, response.cancel , 30-50ms fade, jitter flush; half-duplex fallback on poor AEC). (Android §4.3) - [] **S** — Onboarding wizard + Settings + Wake-word management + Live overlay (Compose, populated controls, TalkBack, WCAG AA). (Android §7) - [] **S** — Permissions choreography + prominent mic disclosure/consent logging; offline/edge behavior. (Android §5/§8) - [] **S** — Tests: JUnit/Robolectric, Espresso, VoiceInteractionService instrumented, wake-engine FRR@FAR harness gated in CI. (Crosscut §7)

M5 — M5Stack firmware + IoT + on-device config / 10-yr login [] (WS-F lead)

Definition of Done: A Tab5 boots ESP-IDF firmware, onboards WiFi + Login-with-Amazons via its device-hosted config page, provisions an IoT Thing + on-chip-keypair X.509 cert (10-yr lineage, DS-peripheral-protected), does on-device ESP-SR wake detection, streams Opus audio over IoT MQTT through the Fargate

relay to GPT-Realtime with instant local barge-in, renders the LVGL state-machine UI, syncs settings via device shadow, and updates via signed A/B IoT-Jobs OTA. Device recorded in `c:\dev\fleet\esp32.md`. (FR [M5])



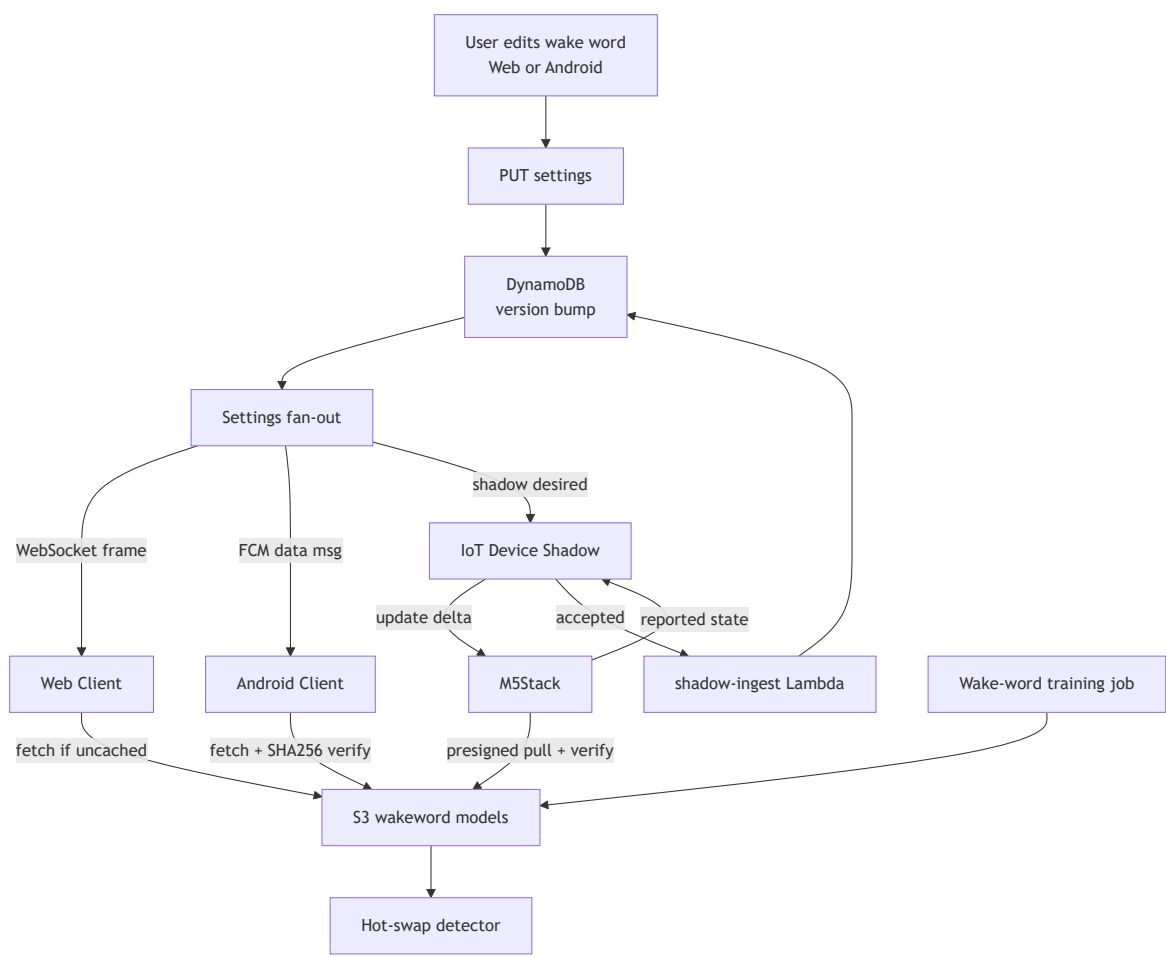
diagram

Ordered tasks: - [] **O** — ESP-IDF v5.4+ project, pinned tag; P4/C6 esp-hosted link bring-up, netif/esp-tls/mqtt over hosted transport; task partition (audio_rx , ww_infer , net_uplink/downlink , lvgl , ctrl). (M5 §1/§2) - [] **F** — Audio path: PDM mic → AFE (AEC/NS/VAD) → ESP-SR WakeNet “Hey Live Ninja” → Opus 16kHz 20ms uplink; downlink Opus decode → I2S with 60-100ms jitter buffer; local instant barge-in (stop DAC + control publish). (M5 §3/§4) - [] **S** — IoT Core: Fleet Provisioning by Claiming Certificate, on-chip keypair (DS peripheral), per-device topic policy (\${iot:Connection.Thing.ThingName}), topic map (audio/up|down , control/up|down , telemetry), classic+ config shadows. (M5 §5) - [] **O** — Device-hosted config: SoftAP captive portal (SSID scan-list-select, passphrase keyboard only), STA config page, **LWA PKCE brokered by backend**, bind token returned over IoT control/down . (M5 §6) - [] **O** — 10-yr persistence: X.509 op-cert (10-yr, rotate at yr8), encrypted NVS bind record, flash encryption + Secure Boot v2 + NVS encryption; steady-state 24h mTLS refresh; realtime session request over HTTPS → relay. (M5 §6, Auth §6) - [] **F** — `iot-ingest` Lambda: `SELECT * FROM 'liveninja/+/telemetry'` → DynamoDB `DEVICE# lastSeen/telemetry (PutItem, GSI2 DEVSEEN#)`; IoT Rules routing audio/control to relay. (§10

backend) - [] S — LVGL UI state machine (Idle/Listening/Speaking/Settings/Onboarding/Error), 720p PSRAM framebuffers + PPA dirty-rect, 48-64px targets, list-selects, "N of M", keyboard only for passphrase/name. (M5 §7) - [] F — OTA: A/B partitions, esp_https_ota, Secure Boot v2 verify + anti-rollback eFuse, IoT Jobs canary→fleet, mark-valid-after-check-in, coordinated P4↔C6 version gate. (M5 §8) - [] S — HIL rig scaffolding (bench Tab5, PlatformIO CI flash, serial+telemetry MQTT assert); record device in c:\dev\fleet\esp32.md (eFuse MAC, role, last COM). (Crosscut §7, fleet rule)

M6 — Programmable wake-word system + settings sync [] (WS-G lead)

Definition of Done: A user can create a custom wake phrase on Web/Android; a backend training pipeline produces per-platform models to S3 (SHA-256 pinned); the canonical settings doc (SETTINGS#v<n> , optimistic-concurrency) syncs across all surfaces via WebSocket/FCM/IoT-shadow with higher-version-wins reconciliation; each client SHA-verifies and hot-swaps wake models; the M5 selects from curated flashable WakeNet + oWW-ESP fallback. (FR [WAKE] , [SETTINGS])



diagram

Ordered tasks: - [] O — Settings schema + version optimistic concurrency (UpdateItem ConditionExpression), GET/PUT /settings , ?since=<v> reconcile, forward-compat unknown-field preservation. (Crosscut §3) - [] F — Fan-out: Web WebSocket settings.updated frame; Android FCM data-message nudge; IoT config shadow desired publish. (Crosscut §3.2) - [] F — shadow-ingest Lambda (IoT Rule on ... /shadow/name/config/update/accepted) → Dynamo version bump; higher-version-wins push-down. (Crosscut §3.2) - [] F — Training pipeline: wakeword-train validate

(phoneme/profanity/collision), openWakeWord on AWS Batch (arm64, conc≤2, 20-min timeout, ≤3/day/user) → int8 .onnx ; Porcupine Console API server-side per platform; output to S3, `WAKEWORD#<wwId>status=ready`, SES "ready". (Crosscut §2.2) - [] **S** — Distribution: `GET /wakeword/<wwId>/model?platform=` → presigned/CloudFront-signed URL; catalog snapshot JSON on S3+CloudFront (no live table read on public path). (Crosscut §2.3, backend §5) - [] **F** — Client hot-swap: Web/Android SHA-256 verify + live swap; M5 curated flashable WakeNet set + oWW-ESP fallback, shadow-driven model ref. (Crosscut §2.4) - [] **S** — Wake-word management UIs on Web + Android wired to catalog + custom-phrase request flow. (Web §5.3, Android §7.4) - [] **F** — Contract tests: settings schema round-trip, conflict 409 reconcile, shadow loop, wake manifest + SHA verify across client fixtures. (Crosscut §7/§8)

M7 — Hardening / observability / cost / privacy [] (WS-G lead, all WS)

Definition of Done: Full observability (structured logs, EMF, X-Ray, `live-ninja-ops` dashboard, Athena telemetry lake); all cost/quota controls enforced pre-spend with anomaly auto-suspend; privacy posture complete (on-device-wake invariant + disclosures, retention TTLs, `DELETE /account`, ZDR requested); WAF rate rules; capability negotiation + `/compat`; load tests confirm `Query`-only under load; security review passed. (FR [OBS], [COST], [PRIV], [SEC])

Ordered tasks: - [] **S** — Observability: `slog` JSON everywhere (requestId/userId/surface/route/latency, no PII), EMF metrics, X-Ray, `live-ninja-ops` dashboard; 30-day log retention. (§13 backend, Crosscut §5) - [] **S** — Telemetry lake: `/telemetry` batched+sampled (Web/Android) + M5 MQTT → IoT Rule → Firehose → S3 → Athena; event schema only, no transcript content. (Crosscut §5) - [] **O** — Cost/quota: pre-spend daily-minute + monthly-token caps, 10-min hard session cap (token TTL + server kill), per-user hourly-burn anomaly auto-suspend + alarm; per-user cost attribution rollups. (Crosscut §6, Voice §13) - [] **S** — Alarms: Lambda Errors/Throttles/p99, broker mint error rate, HTTP 5xx, SES bounce/complaint, IoT heartbeat-gap, DynamoDB read-runaway; SNS→SES. (§13) - [] **O** — Privacy: consent records (`CONSENT#`), retention TTLs (transcripts 30d default, audio off), `DELETE /account` Step Functions (Query+BatchWrite, S3 purge, IoT thing/cert delete, LWA revoke, SES confirm), OpenAI ZDR requested + documented. (Crosscut §4) - [] **S** — WAF rate-based rules on API; per-user concurrent-session + session-rate limits; IoT telemetry throttle. (Crosscut §6) - [] **S** — Versioning/compat: `/v1` path prefix, `X-LN-Client` / `X-LN-Server` headers, `GET /compat`, below-min "please update" states; content-addressed wake models with safe fallback. (Crosscut §8) - [] **F** — Load tests (k6/Vegeta + synthetic-session generator): quota/rate limits hold, `ConsumedReadCapacityUnits` flat (no Scan), relay under connection load. (Crosscut §7) - [] **O** — Security review pass over auth/broker/device/tool paths + threat-model checklist sign-off. (Auth §9, all risk tables)

M8 — Launch [] (WS-A + WS-G lead, all WS)

Definition of Done: SES production access granted (out of sandbox); Cost Allocation Tags confirmed active; all three surfaces pass end-to-end smoke on production; distribution channels live (web at `live.jeremy.ninja`, Android signed APK + `assetlinks.json` /in-app updater, M5 firmware release + fleet provisioning enabled); alarms + budgets confirmed firing to email; runbook + `/v1` long-horizon compatibility commitment documented. (FR [LAUNCH])

Ordered tasks: - [] **H** — Request SES production access; verify DKIM `@jeremy.ninja` identity, bounce/complaint SNS suppression wired. (§6 backend) - [] **H** — Confirm `Project` + `CostCenter` Cost Allocation Tags active in Billing; budgets alerting. (§12) - [] **S** — Production end-to-end smoke: web voice turn, Android wake→WebRTC turn+tool call, M5 wake→relay turn+barge-in; verify one line via `gh run`

`watch . ($14)` - [] **S** — Distribution: web live; Android signed APK + `.well-known/assetlinks.json` + `GET /v1/app/android/latest` updater; M5 firmware channel + fleet provisioning claim enabled. (*Android \$9, M5 \$8*) - [] **H** — Runbook + on-call: alarm→action mapping, credential-rotation steps (re-put SSM), device kill-switch, `/v1` compatibility lifetime commitment. (*Crosscut \$8*) - [] **O** — Launch go/no-go review against every risk table; sign off residual-risk acceptances. (*\$7*)

5. Environments & deploy

- **Production-only.** No staging/dev. Every push to `main` is a production deploy. Verify before pushing; no destructive surprises.
 - **Deploy path (only allowed one):** push/merge to `main` → GitHub Actions assumes `vars.AWS_DEPLOY_ROLE_ARN` via **OIDC** (`id-token: write`, no static keys) → `make build (arm64)` → `sam build` → `sam deploy --no-confirm-changeset --no-fail-on-empty-changeset`. No local `aws / sam deploy / sam sync`. If a deploy need has no pipeline, build the pipeline — never deploy by hand.
 - **arm64 everywhere.** All Lambdas `provided.al2023`, Architectures: `[arm64]`, built `G00S=linux GOARCH=arm64 CGO_ENABLED=0 -tags lambda.norpc -o bootstrap`. The Fargate relay is arm64. Flip architecture and build step together (never Architectures: alone).
 - **Cost tags (stack-level, once, in `samconfig.toml`):** `Project=live-ninja CostCenter=voice-ai Environment=prod ManagedBy=sam DeployedVia=github-actions Owner=jeremy`. Activate `Project + CostCenter` as Cost Allocation Tags in Billing at M0 (non-retroactive).
 - **Secrets:** SSM Parameter Store SecureString + KMS only — **no Secrets Manager/Vault**. Agents never see values; the user sets them via `scripts/set-secret.sh` (GitHub) / manual `aws ssm put-parameter` (SSM). Workflow reads only ARNs from `vars`. Broker holds the sole `ssm:GetParameter` on the OpenAI key ARN.
 - **DynamoDB discipline:** `Query / GetItem` only on every serving path; `Scan` reserved for manual one-off migrations. Read-mostly catalogs served from S3/CloudFront snapshots. `ConsumedReadCapacityUnits` alarm armed.
 - **GitHub Actions monitoring:** summarize `gh run watch` to one line; show detail only on failure.
-

6. Testing & verification strategy per milestone

Milestone	Verification (definition-of-done gate)
M0	<code>sam deploy</code> succeeds via OIDC; <code>/healthz 200</code> through CloudFront; table+GSIs+TTL present; buckets/edge exist; alarms+budgets armed; tags active. Local Fiber smoke (:8080) parity.
M1	<code>dynamodb-local</code> + mocked LWA: PKCE happy-path all surfaces; <code>aud</code> -substitution + refresh-reuse rejected; <code>tokensValidAfter</code> kill ≤ 60 s; cookie flags correct; device-claim binding; new-sign-in SES fires (mock).
M2	Mocked OpenAI REST/WS: config-bound ephemeral mint; quota gate rejects over-cap pre-spend ; tool router re-authz + confirm-before-send + idempotency; relay Opus $\rightarrow$ PCM16 + barge-in cancel; fallback cascade degrades correctly.
M3	Playwright (stub LWA, mock OpenAI WS): login $\rightarrow$ conversation bootstrap, barge-in cut-through, transcript render, settings CRUD, wake-swap, click-to-talk fallback; Lighthouse + axe WCAG AA both themes; SW network-first HTML verified (no stale HTML).
M4	Robolectric/Espresso + instrumented VoiceInteractionService; wake-engine FRR@FAR corpus gated in CI (fail on regression); role-flow fallback across <code>Build.MANUFACTURER</code> ; battery $< 2\%$ /hr screen-off measured; barge-in AEC on-device.
M5	HIL rig: CI flashes bench Tab5 (PlatformIO), speaker plays positive/negative corpus, assert wake on <code>telemetry</code> ; provisioning issues cert (key on-chip); relay round-trip audio; OTA canary $\rightarrow$ mark-valid $\rightarrow$ rollback-on-fail; shadow sync loop.
M6	Contract tests: settings schema round-trip + 409 reconcile; shadow accepted $\rightarrow$ Dynamo bump; wake manifest + SHA-256 verify across all-client fixtures; training job cost-bounded (conc ≤ 2 , timeout, ≤ 3 /day).
M7	k6/Vegeta load: <code>ConsumedReadCapacityUnits</code> flat (proves no Scan), quota/rate limits hold, relay under connection load; <code>DELETE /account</code> purges all partitions (Query+BatchWrite); security-review checklist signed.

Milestone	Verification (definition-of-done gate)
M8	Production end-to-end smoke on all three surfaces; alarms/budgets confirmed emailing; SES out of sandbox; distribution channels reachable; go/no-go against risk tables.

Cross-cutting gates (all milestones): `golangci-lint` + `go vet` clean; unit tests `testify` table-driven; JSON-schema/contract validation in CI; deploy job gated on tests; every new UI form runs the **mandatory multi-persona design pass** before code.

7. Risks & mitigations (execution / sequencing)

#	Execution risk	Mitigation (baked in)
1	WS-A slips → everything blocks. M0 is the critical path for all six workstreams.	Staff M0 with Opus+Sonnet; keep it minimal-but-real (deploy an empty stack day one). Contract freeze at M0 lets WS-B..G design against stable seams before infra is 100%.
2	Shared contracts drift between surfaces built in parallel (settings/shadow/wake-manifest/headers/quota).	Six contracts frozen in /contracts at M0 (WS-G, Opus); CI validates every client fixture against them; additive-only within /v1 .
3	Fargate relay (M2) blocks M5 — M5 audio depends on the relay existing.	Build the relay in M2 (before any device connects) so M5 firmware integrates against a live endpoint; relay tested with synthetic device JWT + mock device frames.
4	Auth (M1) is on the critical path for M2–M5 and is the hardest/riskiest.	Opus-led; ES256+KMS + rotating-refresh machinery is shared identically across surfaces — build once in M1, reuse. Threat-model checklist gates M1 done.
5	Production-only, no staging — a bad deploy hits users.	Change-set previewed in CI logs; local Fiber LWA-parity smoke pre-push; feature flags via capability negotiation; alarms + fast SES alerting; additive-only API changes; canary IoT-Jobs for firmware.
6	M5 firmware (M5) is long, hardware-gated, and last — schedule tail risk.	Sequenced to V2 after web/Android prove the loop; HIL rig scaffolded early (M5 task); P4/C6 esp-hosted link is task-1 risk-front-loaded; device recorded in fleet registry.
7	Wake-word training cost/latency (M6) could blow up or stall.	AWS Batch arm64 conc≤2, 20-min timeout, ≤3/day/user; openWakeWord (free) as default engine so never blocked on Picovoice seats; async, off request path.
8	Model-routing mis-assignment wastes budget or under-powers hard tasks.	Routing column on every task; security/auth/contracts/architecture → Opus; audio/wake/sync → Fable (→Opus if unavailable, never Sonnet); mechanical → Haiku.

#	Execution risk	Mitigation (baked in)
9	Cross-surface settings-sync conflicts (3 writers) corrupt state.	Monotonic <code>version</code> + <code>ConditionExpression</code> optimistic concurrency; Dynamo source of truth; IoT shadow transport for M5; higher-version-wins; contract tests for 409 reconcile in M6.
10	DynamoDB Scan sneaks onto a serving path during parallel dev.	"No Scan" is a review red flag; <code>ConsumedReadCapacityUnits</code> alarm from M0; load test in M7 proves flat read units; catalogs from S3 snapshots.
11	Subagent stall/timeout during autonomous execution.	Per house policy: 6-min inactivity / 30-min completion timeouts, auto-retry ≤ 2 (3 total), incorporate partial output on retry, escalate to user after 2 failed restarts.
12	Long-lived M5 devices outrun API changes (10-yr field life).	<code>/v1</code> kept alive for field lifetime; capability negotiation + <code>/compat</code> ; forward-compatible settings schema (unknown fields preserved); signed IoT-Jobs OTA.

8. Implementation Notes

Per house style, append **verbose** notes here (and inline under each task/milestone) as work proceeds — decisions made, files touched, commands run, gotchas hit, blockers and how they were resolved. Keep it detailed enough that a fresh agent can resume from this plan alone. Update the status markers in §4 in place.

M0 — Bootstrap / Infrastructure

(no notes yet)

M1 — Auth

(no notes yet)

M2 — Realtime voice backend

(no notes yet)

M3 — Web client

(no notes yet)

M4 — Android client

(no notes yet)

M5 — M5Stack firmware + IoT

(no notes yet)

M6 — Programmable wake-word + settings sync

(no notes yet)

M7 — Hardening / observability / cost / privacy

(no notes yet)

M8 — Launch

(no notes yet)